

Advanced Programming in the Linux Environment

Volume III

Environment and System Databases

Marco Bruno

Volume III — Environment and System Databases

This chapter explores the process environment and its interaction with the operating system. It includes functions for handling time and date, environment variables, and localization (locale). It also provides a detailed overview of system databases, such as user and group information, along with login records and system identification (*uname*). The chapter concludes with a dedicated section on Pluggable Authentication Modules (PAM).

Table of contents

Volume III – Environment and system databases

III.1 Time functions

III.1.1 Getting the time: `time`, `gettimeofday` and `settimeofday`

III.1.2 Setting time with `clock_gettime` and `clock_settime`

III.1.3 Clocks on Linux

III.1.4 Setting a new time: `tm`

III.1.5 Time conversion functions

III.1.6 Formatting date and time: `strftime` and `strptime`

III.1.7 Time conversion: `getdate`

III.2 The environment

III.2.1 Working with environment variables: `getenv` and `setenv`

III.2.2 Environment during an `execve`

III.3 Locale

III.3.1 Setting locale: `setlocale`

III.3.2 Working with locale: `localeconv`, `newlocale` and `uselocale`

III.3.3 Duplicating a locale object: `duplocale`

III.4 The User Database: `/etc/passwd`

III.4.1 User and group files

III.4.2 The `/etc/passwd` file

III.4.3 Iterating the user database : `getpwent` and `setpwent`

III.4.4 Working with records : `fgetpwent` and `putpwent`

III.5 The Group Database: `/etc/group`

III.5.1 Reading the group database: `getgrnam` and `getgrgid`

III.5.2 Working with the database: `getgrent` and `setgrent`

III.5.3 Retrieving the group list: `getgrouplist`

III.5.4 Supplementary groups: `getgroups`, `setgroups` and `initgroups`

III.5.5 Real and effective group ID: `getgid`, `getegid`, `setgid`, `setegid`

III.5.6 Saved Set-User-Id and Set-Group-Id

III.5.7 Checking whether a process is in a group: `group_member`

III.6 Users functions

III.6.1 Setting user ID: `getuid`, `geteuid`, `setuid` and `seteuid`

III.6.2 `getresgid` and `getresuid`

III.7 The Password Database: `/etc/shadow`

III.7.1 Creating an encrypted passphrase: `crypt`

III.7.2 Generating a salt: `crypt_gensalt`

III.7.3 Hiding the input with BSD `readpassphrase`

III.8 Login

III.8.1 Working with the `utmp` file: `getutent`, `getutid`, `setutent`

III.8.2 The login: `login` and `logout`

III.9 The host, network, protocol and service databases

III.9.1 The host database: `/etc/hosts`

III.9.2 The network database: `/etc/networks`

III.9.3 The protocol database: `/etc/protocols`

III.9.4 The service database: `/etc/services`

III.10 System identification

III.10.1 Getting system information: `uname`

III.10.2 Getting and setting hostname: `gethostname` and `sethostname`

III.11 Pluggable Authentication Modules

III.11.1 PAM Authentication: `pam_authenticate`

III.11.2 PAM Session: `pam_open_session` and `pam_close_session`

III.11.3 Process environment and PAM environment: `pam_getenv`, `pam_putenv`

III.11.4 PAM items: `pam_get_item` and `pam_set_item`

III.11.5 Changing password: `pam_chauthtok`

III.12 Theory and insights

III.12.1 The Process Execution Environment

III.12.2 The `intmax_t` and `uintmax_t` data types

III.12.3 Using `initgroups` to set up the supplementary groups

III.12.4 Command line tools for the groups

III.12.5 The *Name Service Switch*

III.12.6 Command line tools for the users

III.12.7 The login process

III.12.8 PAM functions summary

List of tables

List of figures

Bibliography

III.1 Time functions

When we work on Linux systems it is a good practice to know how the system deals with dates and times, how we can retrieve these information and how we can use them.

We have already seen how to retrieve the current time with the command `date`. We examine some options of this command.

```
[linux@fidir] date
Fri Apr 25 05:37:42 +07 2025

[linux@fidir] date -u
Thu Apr 24 22:38:37 UTC 2025

[linux@fidir] date +%D
04/25/25

[linux@fidir] date +%T
05:41:06

[linux@fidir] date +%s
1745534529
```

Without any options, `date` returns the current date and time, passing the option `-u`, it returns the UTC time. The `+` sign is used to specify which format or part of the date the command must return: `D` returns the current date in local format, `T` the current time, `s` the timestamp from the Epoch.

In the `/etc` folder, the symbolic link `localtime` points to the file containing the data of the local timezone, which is in the folder `/usr/share/zoneinfo`.

```
[linux@fidir] ls -l /etc/localtime
lrwxrwxrwx 1 root root 35 Mar 26 20:31 /etc/localtime -> /usr/share/zoneinfo/Asia/Phnom_Penh

[linux@fidir] file /usr/share/zoneinfo/Asia/Phnom_Penh
/usr/share/zoneinfo/Asia/Phnom_Penh: timezone data (fat), version 2, no gmt time flags, no std time flags, no leap seconds, 5 transition times, 6 local time types, 21 abbreviation chars
```

The pointed file is a *timezone data* file (binary file).

In addition, in the `/etc` folder, there is the file containing the information of the current timezone.

```
[linux@fidir] ls -l /etc/timezone
-rw-r--r-- 1 root root 16 Apr 17 08:13 timezone

[linux@fidir] cat /etc/timezone
Asia/Phnom_Penh
```

The `/usr/share/zoneinfo` directories contains files for the various time zones available on the system.

III.1.1 Getting the time: `time`, `gettimeofday` and `settimeofday`

Linux provides many functions for handling date and time. The `time`¹ function returns the time as the number of seconds elapsed since the Epoch.

```
#include <time.h>

time_t time(time_t *_Nullable tloc);

Returns: the number of seconds if success, -1 if error and errno is set
```

If `tloc` is not `NULL`, the returned value is stored in it. The number returned by this calls, represents the number of seconds since the date “1970-01-01 00:00:00 +0000 (UTC)”.

To retrieve and set the time, the following functions² are available.

```
#include <sys/time.h>

int gettimeofday(struct timeval *restrict tv, struct timezone *_Nullable
                 restrict tz);

int settimeofday(const struct timeval *tv, const struct timezone
                 *_Nullable tz);

Returns: 0 if success, -1 if error and errno is set
```

The `timeval` structure is defined in the `sys/time.h` file header as follows:

```
struct timeval {
    time_t tv_sec;           /* seconds */
    suseconds_t tv_usec;     /* microseconds */
};
```

The `timezone` structure is defined as follows:

```
struct timezone {
    int tz_minuteswest;     /* minutes west of Greenwich */
    int tz_dsttime;         /* type of DST correction */
};
```

Since the `tz` argument is obsolete, it is usually specified as `NULL`.

The `gettimeofday.c` program uses `gettimeofday` and `time` to retrieve the current time.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
```

1 See the `time(2)` manual page

2 See the man page `gettimeofday(2)` manual page

```
4 #include <sys/time.h>

5 int main(int argc, char *argv[]) {
6     struct timeval *tv;
7     int curt;

8     if (gettimeofday(tv, NULL) == -1) {
9         perror("gettimeofday");
10        exit(EXIT_FAILURE);
11    }

12    printf("\nCurrent: %d", tv->tv_sec);
13    if ((curt = time(NULL)) == -1) {
14        perror("time");
15    }

16    printf("\nCurrent: %d", curt);
17 }
```

gettimeofday.c

We run the program and then check the result using the `date` command.

```
[linux@fidir] ./gettimeofday
Current: 1745537558
Current: 1745537558

[linux@fidir] date +%s
1745537560
```

`time` and `gettimeofday` have returned the same value. The execution of `date` has returned a similar value (two seconds greater).

III.1.2 Setting time with `clock_gettime` and `clock_settime`

In addition to the aforementioned functions, the following functions are also available for retrieving and setting the time³.

```
#include <time.h>

int clock_getres(clockid_t clockid, struct timespec *_Nullable res);

int clock_gettime(clockid_t clockid, struct timespec *tp);

int clock_settime(clockid_t clockid, const struct timespec *tp);

Return: 0 if success, -1 if error and errno is set
```

The `clock_gettime` and `clock_settime` functions, retrieve and set the time of the clock specified by the argument `clockid`. The `clockid` argument is of the type `clockid_t`⁴, which is defined in the `sys/types.h`⁵ header file, and represents the clock on which to act.

A clock can be either system-wide, making it accessible to all processes, or process-specific, where it measures time only within a single process.

All the implementations support the system-wide *real-time clock*, which is defined by the `CLOCK_REALTIME` constant, which represents seconds and nanoseconds since the Epoch. The following table lists the clocks supported by the *glibc* and Linux kernel.

<code>CLOCK_REALTIME</code>	System-wide clock that measures real time, to set it it requires appropriate privileges
<code>CLOCK_REALTIME_ALARM</code>	Like <code>CLOCK_REALTIME</code> but not settable
<code>CLOCK_REALTIME_COARSE</code>	Faster and less precise than <code>CLOCK_REALTIME</code> , not settable
<code>CLOCK_TAI</code>	Nonsettable system-wide clock derived by <code>CLOCK_REALTIME</code> but counting leap seconds ⁶
<code>CLOCK_MONOTONIC</code>	Nonsettable system-wide clock representing monotonic time since an unspecified point in the past, on Linux the value is the number of seconds from when the system booted. It does not count time that the system is suspended
<code>CLOCK_MONOTONIC_COARSE</code>	Faster but less precise version of <code>CLOCK_MONOTONIC</code>
<code>CLOCK_MONOTONIC_RAW</code>	Like <code>CLOCK_MONOTONIC</code> but provides access to hardware-based time
<code>CLOCK_BOOTTIME</code>	Nonsettable system-wide clock identical to <code>CLOCK_MONOTONIC</code> but includes the time that the system

³ See the `clock_getres(2)` manual page

⁴ `clockid_t` is an arithmetic type, see the `clockid_t(3)` manual page

⁵ On Debian-based systems, the header file is `/usr/include/x86_64-linux-gnu/bits/types/clockid_t.h`

⁶ It does not have discontinuities or frequency adjustments caused by inserting leap seconds. TAI is an acronym of International Atomic Time

	is suspended
CLOCK_BOOTTIME_ALARM	Like CLOCK_BOOTTIME
CLOCK_PROCESS_CPUTIME_ID	It measures the CPU time consumed by the process, not settable
CLOCK_THREAD_CPUTIME_ID	It measures the CPU time consumed by the thread, not settable

Tab 1. Clocks

These clock IDs are hard-coded in the System V style, and Linux also supports POSIX clock operations on certain character devices known as *dynamic clocks*.

With appropriate macros we can convert open file descriptors into clock IDs⁷ and pass them to `clock_gettime`, `clock_settime`, `clock_adjtime`.

The `clock_getres` function finds the resolution of the specified clock and stores it in the `res` argument.

The resolution of a `clock` depends on its implementation. Since the clock resolution defines the unit of time measurement, if the value of the `tp` argument passed to the `clock_settime` function is not a multiple of the `res` value returned by `clock_getres`, it will be truncated to the nearest multiple of `res`.

The `clock_times.c` program, taken from the `clock_gettime(2)` manual page, demonstrates how to use the POSIX functions `clock_gettime` and `clock_getres` with different system clocks available in Linux.

```

1 #define _XOPEN_SOURCE 600
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <time.h>
7 #define SECS_IN_DAY (24 * 60 * 60)

8 static void displayClock(clockid_t clock, const char *name, bool
                           showRes)
9 {
10     long days;
11     struct timespec ts;

12     if (clock_gettime(clock, &ts) == -1) {
13         perror("clock_gettime");
14         exit(EXIT_FAILURE);
15     }

16     printf("%-15s: %10jd.%03ld (", name, (intmax_t) ts.tv_sec,
            ts.tv_nsec / 1000000);

```

⁷ An example of this technique is available in the `clock_getres(2)` manual page

```

17  days = ts.tv_sec / SECS_IN_DAY;
18  if (days > 0)
19      printf("%ld days + ", days);

20  printf("%2dh %2dm %2ds", (int) (ts.tv_sec % SECS_IN_DAY) / 3600,
          (int) (ts.tv_sec % 3600) / 60, (int) ts.tv_sec % 60);
21  printf("\n");

22  if (clock_getres(clock, &ts) == -1) {
23      perror("clock_getres");
24      exit(EXIT_FAILURE);
25  }

26  if (showRes)
27      printf("resolution: %10jd.%09ld\n", (intmax_t) ts.tv_sec,
            ts.tv_nsec);
28  }

29 int main(int argc, char *argv[])
30 {
31     bool showRes = argc > 1;
32     displayClock(CLOCK_REALTIME, "CLOCK_REALTIME", showRes);

33     #ifdef CLOCK_TAI
34         displayClock(CLOCK_TAI, "CLOCK_TAI", showRes);
35     #endif
36     displayClock(CLOCK_MONOTONIC, "CLOCK_MONOTONIC", showRes);
37     #ifdef CLOCK_BOOTTIME
38         displayClock(CLOCK_BOOTTIME, "CLOCK_BOOTTIME", showRes);
39     #endif
40     exit(EXIT_SUCCESS);
41 }

```

clock_times.c

- Line 1. Defines the feature test macro `_XOPEN_SOURCE 600`, enabling POSIX.1-2001 functionality required for the use of `clock_gettime` and `clock_getres`*
- Line 7. Defines the constant `SECS_IN_DAY`, representing the total number of seconds in one day ($24 \times 60 \times 60 = 86400$)*
- Line 8. Defines the function `displayClock`, which receives: a clock identifier (`clockid_t`), the clock name as a string, a boolean flag indicating whether the clock resolution must be displayed*
- Line 10-11. Declares: a variable `days` used to store the number of elapsed days, a `timespec` structure used to store seconds and nanoseconds returned by the clock functions*
- Line 12-15. Calls `clock_gettime` to retrieve the current value of the specified clock.*
- Line 16. Prints: the clock name, the number of elapsed seconds (`tv_sec`), the milliseconds obtained from the nanoseconds field (`tv_nsec / 1000000`).*
- Line 17-19. Calculates the number of complete days contained in the elapsed seconds and prints the result if the value is greater than zero*
- Line 20-21. Converts the remaining seconds into: hours, minutes, seconds, using modulo arithmetic*

and prints the time in human-readable format.

Line 22-25. Calls `clock_getres` to obtain the resolution (precision) of the specified clock.

If the function fails, the program prints an error message and exits

Line 26-27. If the boolean variable `showRes` is true, prints the clock resolution in seconds and nanoseconds

Line 29. Defines the `main` function, which represents the entry point of the program

Line 31. Defines the boolean variable `showRes`. The variable becomes `true` if at least one command-line argument is provided (`argc > 1`)

Line 32. Calls `displayClock` with `CLOCK_REALTIME`, which represents the system real-time clock

Line 33-35. Conditionally calls `displayClock` with `CLOCK_TAI` if the system supports this clock

Line 36. Calls `displayClock` with `CLOCK_MONOTONIC`, a monotonic clock not affected by system time changes

Line 37-39. Conditionally calls `displayClock` with `CLOCK_BOOTTIME`, which also includes time spent while the system is suspended

We execute the program

```
[linux@fidir] ./clock_times
CLOCK_REALTIME      : 1745717821.669 (20205 days + 1h 37m 1s)
CLOCK_TAI           : 1745717821.669 (20205 days + 1h 37m 1s)
CLOCK_MONOTONIC     : 125659.771 (1 days + 10h 54m 19s)
CLOCK_BOOTTIME      : 125659.771 (1 days + 10h 54m 19s)
```

This execution shows the current values returned by four different Linux clocks.

- `CLOCK_REALTIME` and `CLOCK_TAI` return the current calendar time expressed as the number of seconds elapsed since the Unix Epoch (1 January 1970). The output also converts this value into a human-readable format showing days, hours, minutes, and seconds
- `CLOCK_MONOTONIC` returns the time elapsed since the system boot. This clock increases continuously and is not affected by manual system date changes or NTP adjustments
- `CLOCK_BOOTTIME` is similar to `CLOCK_MONOTONIC`, but it also includes the time during which the system was suspended or in sleep mode

In this execution the system has been running for approximately 1 day, 10 hours, 54 minutes, and 19 seconds. Both monotonic clocks return the same value, indicating that the system has not been suspended since boot. The milliseconds displayed after the decimal point correspond to the nanoseconds field converted to milliseconds.

We pass an argument to retrieve the resolution of the clock.

```
[linux@fidir] ./clock_times 1
CLOCK_REALTIME :      1745717909.375 (20205 days + 1h 38m 29s)
resolution:      0.000000001
CLOCK_TAI :      1745717909.375 (20205 days + 1h 38m 29s)
resolution:      0.000000001
CLOCK_MONOTONIC:    125747.477 (1 days + 10h 55m 47s)
resolution:      0.000000001
```

CLOCK_BOOTTIME :	125747.477 (1 days + 10h 55m 47s)
resolution:	0.000000001

When the program is executed with a command-line argument, the variable `showRes` becomes `true`, causing the program to also display the resolution of each clock using `clock_getres`. The output shows that all the clocks have a resolution of `0.000000001`, which corresponds to 1 nanosecond.

This means the kernel is capable of representing time values with nanosecond precision.

`CLOCK_REALTIME` and `CLOCK_TAI` return the current system time expressed as the number of seconds elapsed since the Unix Epoch (1 January 1970 00:00:00 UTC).

The retrieved value is based on UTC time and is independent from the local timezone configured on the system. The timezone only affects how the timestamp is converted into a human-readable date and time representation. As a consequence, the timestamp value is theoretically identical on all systems, differences may occur only if system clocks are not correctly synchronized.

`CLOCK_MONOTONIC` and `CLOCK_BOOTTIME` instead represent the elapsed time since system startup rather than calendar time.

III.1.3 Clocks on Linux

On Linux systems, the current date, time, timezone, and clock synchronization status can be displayed using the `timedatectl`⁸ command.

```
[linux@fidir] timedatectl
      Local time: Sun 2025-04-27 09:07:48 +07
      Universal time: Sun 2025-04-27 02:07:48 UTC
      RTC time: Sun 2025-04-27 02:07:48
      Time zone: Asia/Phnom_Penh (+07, +0700)
System clock synchronized: yes
      NTP service: active
      RTC in local TZ: no
```

Obviously, the *local time* and the *universal time* (UTC) differ because the local time includes the *timezone offset* configured on the system.

The value returned by `clock_gettime` with `CLOCK_REALTIME` is based on the Unix Epoch (1 January 1970 00:00:00 UTC). Therefore, the returned timestamp is internally represented in UTC and is independent from the *local timezone*.⁹ To demonstrate this behavior we can use the following sequence of commands.

```
[linux@fidir] date +%s
1745720874

[linux@fidir] date -d '1970-01-01 00:00:00' +%s
-25200
[linux@fidir] echo $(( $(date +%s) + -25200 ))
1745695689

[linux@fidir] ./clock_times
CLOCK_REALTIME : 1745720890.867 (20205 days + 2h 28m 10s)
CLOCK_TAI : 1745720890.867 (20205 days + 2h 28m 10s)
CLOCK_MONOTONIC: 2664.448 ( 0h 44m 24s)
CLOCK_BOOTTIME : 2664.448 ( 0h 44m 24s)
```

The first command returns the current timestamp expressed as the number of seconds elapsed since the Unix Epoch. The second command calculates the difference, in seconds, between the Unix Epoch in UTC and the same date interpreted in the local timezone¹⁰. The returned value (25200) corresponds to a timezone offset of $25200 / 60 / 60 = 7$ hours, which means the system timezone is UTC+7.

The third command executes the `clock_times.c` program and displays the values returned by the different clocks. The value returned by `CLOCK_REALTIME` (1745720890.867) is very close to the timestamp produced by `date +%s`, confirming that `CLOCK_REALTIME` internally uses UTC-based Unix time. The small difference of a few seconds is caused simply by the time required to type and execute the commands.

As a result we understand that Linux internally stores time as UTC, the Unix timestamp is independent from the local timezone, the timezone is only applied when converting timestamps into human-readable local dates and times.

⁸ See the `timedatectl(1)` manual page

⁹ The option `-d` of `date` allows to specify a date and time

¹⁰ As we were in the Epoch date and time in a different timezone on the earth

On Linux systems, there are two different types of clocks:

Hardware Clock

Independent hardware device with its own power supplier. It operates when the machine is powered off or even unplugged. Also called “Real time clock” (RTC), BIOS clock, CMOS clock.

System Clock

Is part of the Linux kernel and is driven by a timer interrupt, it exists and has meaning only while Linux is running. It is the number of seconds from the Epoch. This is initialized from the Hardware Clock at boot.

To retrieve information from the *Hardware clock*, we can use the `hwclock`¹¹ utility.

```
[linux@fidir] sudo hwclock
2025-04-27 09:49:11.767010+07:00
```

The command displays:

- the current date
- the current time with fractions of seconds stored by the hardware clock
- the timezone offset

The value shown is retrieved directly from the RTC (Real-Time Clock) located on the motherboard.

¹¹ See the `hwclock(1)` manual page

III.1.4 Setting a new time: tm

We present a simple example of the use of the `settimeofday` system call. The `setnewtime.c` program first retrieves the current system time and then sets a new date and time value. To define the new date and time, it uses the `tm` structure, which is defined in the `sys/types.h`¹² header file as follows.

```
struct tm
{
    int tm_sec;           /* Seconds. [0-60] (1 leap second) */
    int tm_min;           /* Minutes. [0-59] */
    int tm_hour;          /* Hours. [0-23] */
    int tm_mday;          /* Day. [1-31] */
    int tm_mon;           /* Month. [0-11] */
    int tm_year;          /* Year - 1900. */
    int tm_wday;          /* Day of week. [0-6] */
    int tm_yday;          /* Days in year. [0-365] */
    int tm_isdst;         /* DST. [-1/0/1] */
#ifdef __USE_MISC
    long int tm_gmtoff;    /* Seconds east of UTC. */
    const char *tm_zone;   /* Timezone abbreviation. */
#else
    long int __tm_gmtoff;   /* Seconds east of UTC. */
    const char *__tm_zone; /* Timezone abbreviation. */
#endif
};
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/time.h>
5 #include <time.h>
6 #include <errno.h>

7 int main(int argc, char *argv[]) {
8     struct timeval new_time;
9     time_t t;

10    if (argc != 7) {
11        printf("\nUsage: %s year month day hour min sec", argv[0]);
12        exit(EXIT_FAILURE);
13    }

14    gettimeofday(&new_time, NULL);
15    t = new_time.tv_sec;
16    printf("\nCurrent system time: %s", ctime(&t));
```

12 On Debian-based systems: `/usr/include/x86_64-linux-gnu/bits/types/struct_tm.h`

```

17  struct tm tm_time = {
18      .tm_year = atoi(argv[1]) - 1900,
19      .tm_mon = atoi(argv[2]) - 1,
20      .tm_mday = atoi(argv[3]),
21      .tm_hour = atoi(argv[4]),
22      .tm_min = atoi(argv[5]),
23      .tm_sec = atoi(argv[6]),
24  };

25  time_t new_epoch = mktime(&tm_time);
26  if (new_epoch == -1) {
27      perror("mktime failed");
28      exit(EXIT_FAILURE);
29  }

30  new_time.tv_sec = new_epoch;
31  new_time.tv_usec = 0;

32  if (settimeofday(&new_time, NULL) == -1) {
33      perror("settimeofday");
34      exit(EXIT_FAILURE);
35  }

36  gettimeofday(&new_time, NULL);
37  t = new_time.tv_sec;
38  printf("New system time: %s", ctime(&t));
39  exit(EXIT_SUCCESS);
40 }

```

setnewtime.c

Line 8. Declares a struct timeval named new_time used to hold time in seconds and microseconds¹³. This structure is required by gettimeofday and settimeofday, which operate with higher precision than time_t

Line 9. Declares a variable of type time_t used to store Unix epoch time¹⁴

Line 14–16. Retrieves the current system time using gettimeofday, which fills new_time with both seconds (tv_sec) and microseconds (tv_usec). Only the seconds portion is copied into t, then converted to a human-readable string using ctime(&t)¹⁵

Line 17–24. Declares and initializes a struct tm representing the target time provided via command-line arguments. The fields are populated using atoi¹⁶. The year is adjusted by subtracting 1900 and the month by subtracting 1, as required by the struct tm format¹⁷

Line 25. Converts the populated struct tm into a time_t (epoch timestamp) using mktime¹⁸

¹³ POSIX time format

¹⁴ This is the standard scalar representation of system time used by many C library functions

¹⁵ ctime returns a pointer to a statically allocated string that includes a trailing newline, and it is not thread-safe, ctime_r would be safer in multithreaded contexts

¹⁶ atoi performs no validation, invalid input (non-numeric strings) will silently become 0.

¹⁷ This structure represents *local time*, meaning results depend on the system timezone and daylight saving rules

¹⁸ mktime interprets the input as local time and may normalize invalid date values

Line 30–31. Assigns the computed epoch time (*new_epoch*) to *new_time.tv_sec* and explicitly sets microseconds to 0. This prepares the structure for use with *settimeofday*, which requires both seconds and microseconds fields

Line 32–35. Attempts to set the system time using *settimeofday*. This operation typically requires root/administrator privileges; otherwise, it will fail with *EPERM*. It directly modifies the kernel clock, so it affects the entire system¹⁹

Line 36–38. Retrieves the updated system time again using *gettimeofday* to verify the change. The new seconds value is converted to a human-readable string using *ctime*

We execute the program using `sudo`

```
[linux@fidir] sudo ./setnewtime 2025 5 3 17 21 10
```

```
Current system time: Thu Apr  3 17:33:07 2025
```

```
New system time: Sat May  3 17:21:10 2025
```

```
[linux@fidir] date
```

```
Sat May  3 17:21:13 +07 2025
```

There are some things to consider:

1. If we run the program without `sudo`, we get "Operation not permitted". This happens because changing the system clock via *settimeofday* is a *privileged operation*
2. The field `tm_year` of the `tm` structure must be expressed as the number of years since 1900. This is part of the historical POSIX/C design of `struct tm`, where: `tm_year = actual_year - 1900`. This is why we subtract 1900 from the user-provided year to correctly map it into the expected internal representation used by `mktime`
3. Months in `tm` are zero-indexed (January = 0, December = 11). This is another legacy design choice in the *C time API*. Therefore `tm_mon = actual_month - 1` is required to align user-friendly input (1–12) with the internal representation expected by `mktime`
4. If NTP (Network Time Protocol) is active on the system, the time may automatically be corrected after running the program. Modern Linux systems often run a time synchronization service (e.g., `systemd-timesyncd`, `ntpd`, or `chronyd`). These services periodically adjust the system clock to match external time servers. As a result, manual changes made by *settimeofday* can be: quickly overwritten, or gradually corrected (slewed back to the correct time, depending on configuration)
5. The program is *not thread-safe*. Several functions used are inherently unsafe in multithreaded environments: `ctime` returns a pointer to a static internal buffer, which can be overwritten by concurrent calls. `mktime` may also rely on shared internal timezone state. Global time-related state (like timezone settings) may be modified implicitly.

¹⁹ `settimeofday` is considered obsolete on some systems in favor of `clock_settime`

III.1.5 Time conversion functions

The `mktime` function converts a broken-down time structure (`struct tm`) into a calendar time representation (`time_t`), expressed as seconds since the Unix epoch. It ignores the values provided in the `tm_wday` (day of the week) and `tm_yday` (day of the year) fields, since these can be automatically computed from the other fields (year, month, and day).

The `tm_isdst` field indicates whether Daylight Saving Time (DST) is in effect for the given time:

- A positive value means DST is in effect
- A value of 0 means DST is not in effect
- A negative value tells `mktime` to automatically determine whether DST should be applied based on the system's timezone rules

When `tm_isdst` is set to `-1`, the function consults the system's timezone database and adjusts the resulting `time_t` value accordingly if DST applies.

The following functions²⁰ can be used to transform date and time to broken-down time or ASCII.

```
#include <time.h>

char *asctime(const struct tm *tm);
char *asctime_r(const struct tm *restrict tm, char buf[restrict 26]);

char *ctime(const time_t *timep);
char *ctime_r(const time_t *restrict timep, char buf[restrict 26]);

struct tm *gmtime(const time_t *timep);
struct tm *gmtime_r(const time_t *restrict timep, struct tm *restrict
                    result);

struct tm *localtime(const time_t *timep);
struct tm *localtime_r(const time_t *restrict timep, struct tm *restrict
                      result);

time_t mktime(struct tm *tm);

Return:
    gmtime and localtime return a pointer to a struct tm, NULL on error
    gmtime_r and localtime_r return the address of the structure pointed to by
        result, NULL on error
    asctime and ctime return a pointer to a string, NULL on error
    asctime_r and ctime_r return a pointer to the string pointed to by buf, NULL
        on error
    mktime returns a calendar time (seconds since Epoch) or -1 if error
```

The functions `ctime`, `gmtime` and `localtime`, take an argument of type `time_t` representing a calendar time, when interpreted as an absolute value, it represent the seconds passed since Epoch. `asctime` and `mktime` take an argument representing broken-down time stored in a structure of type `tm`.

²⁰ See the `ctime(3)` manual page

A call to `ctime` is equivalent to call `asctime(localtime(t))` and returns a null-terminated string representing the current time.

The `gmtime` function converts the calendar time `timep` to broken-down representation expressed in UTC.

`gmtime_r` works like `gmtime` but stores the value in an user-supplied structure passed as second argument.

`localtime` converts the calendar time `timep` to broken-down time representation, expressed relative to the user timezone. This function also sets the external variables `tzname`, `timezone` and `daylight`.

`localtime_r` does the same but stores the result in a user-supplied structure passed as second argument.

`asctime` converts the broken-down time value `tm` into a string with the same format as `ctime`.

`asctime_r` does the same but stores the result in an user-supplied structure passed as second argument.

The `mktime` function modifies the fields `tm_wday` and `tm_yday` calculating them based on the values of the other fields. `mktime` also sets the external variables `tzname`, `timezone` and `daylight`.

The variables `tzname`, `timezone` and `daylight` are usually initialized by the function `tzset`²¹. This function is automatically called by the *time conversion functions* that depend on the timezone. The variables are defined as follows.

```
#include <time.h>

void tzset(void);

extern char *tzname[2];
extern long timezone;
extern int daylight;
```

`tzset` initializes `tzname` from the `TZ` environment variable, if `TZ` is not defined in the environment the system timezone is used. The system timezone is configured copying or linking a file in the `tzfile`²² format to the file `/etc/localtime`. If `TZ` is in the environment but it has no value then UTC is used. The `tzfile` format is used to store the timezone data, the files for the several timezones are in the folder `/usr/share/zoneinfo`.

```
[linux@fidir] ls /usr/share/zoneinfo
Africa      Cuba      GMT+0      Kwajalein  Poland    Zulu
America     EET      GMT-0      Libya      Portugal  iso3166.tab
Antarctica  EST      GMT0       MET        ROC       leap-seconds.list
Arctic      EST5EDT  Greenwich MST        ROK       leapseconds
Asia        Egypt    HST        MST7MDT    Singapore localtime
Atlantic    Eire     Hongkong   Mexico     Turkey    posixrules
Australia   Etc      Iceland    NZ         UCT       right
Brazil      Europe   Indian     NZ-CHAT    US        tzdata.zi
CET         Factory  Iran       Navajo     UTC       zone.tab
CST6CDT     GB       Israel     PRC        Universal zone1970.tab
Canada      GB-Eire  Jamaica    PST8PDT    W-SU     zonenow.tab
Chile       GMT      Japan      Pacific    WET
```

²¹ See the `tzset(3)` manual page

²² See the `tzfile(5)` manual page

III.1.6 Formatting date and time: `strftime` and `strptime`

When working with date and time, it is often necessary to represent them in a specific format. For this purpose, the functions `strftime` and `strptime`²³ are available.

```
#include <time.h>

size_t strftime(char s[restrict .max], size_t max, const char *restrict
                format, const struct tm *restrict tm);

size_t strftime_l(char s[restrict .max], size_t max, const char *restrict
                  format, const struct tm *restrict tm, locale_t locale);

Return: the number of bytes placed in the s array, 0 if the length of the
string exceed max
```

The `strftime` function formats the broken-down time `tm` in the way defined in the `format` argument, and stores the result in the `s` array of size `max`. The format specification is a string that can contain special character sequences called *conversion specifications*, each of which is introduced by a “%” in the same way of the function `printf`.

The function `strftime_l` works exactly as `strftime` but uses the specified `locale` at the place of the current locale. If the length of the string exceed `max` bytes, then the function returns 0.

The *glibc* provides some extensions for conversion specifications that can be used between the “%” and the conversion specifier character. The following table lists the flags that can be used.

<code>_</code> (underscore)	Pad a numeric result string with spaces
<code>-</code> (dash)	Do not pad a numeric result string
<code>0</code>	Pad a numeric result string with zeros even if the conversion specifier character uses space-padding by default
<code>^</code>	Convert alphabetic characters in result string to uppercase
<code>#</code>	Swap the case of the result string, works only with certain conversion specifier characters

Tab 2. Conversion specifier extensions of `strftime`

The `timeformat.c` program, adapted from the `strftime(3)` manual page, prints the date and time according to the format specified on the command line.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>

4 int main(int argc, char *argv[])
5 {
6     char outstr[200];
7     time_t t;
```

23 See the `strftime(3)` manual page

```

8   struct tm *tmp;

9   if (argc != 2) {
10      printf("\nUsage: %s <specifier>", argv[0]);
11      exit(EXIT_FAILURE);
12  }

13  t = time(NULL);
14  tmp = localtime(&t);
15  if (tmp == NULL) {
16      perror("localtime");
17      exit(EXIT_FAILURE);
18  }

19  if (strftime(outstr, sizeof(outstr), argv[1], tmp) == 0) {
20      fprintf(stderr, "strftime returned 0");
21      exit(EXIT_FAILURE);
22  }

23  printf("Result string is \"%s\"\n", ostr);
24  exit(EXIT_SUCCESS);
25 }

```

timeformat.c

Line 13. Retrieves the current system time as a `time_t` value (Unix timestamp), representing the number of seconds since the epoch

Line 14. Converts the timestamp `t` into a broken-down local time representation (`struct tm`) using `localtime`, which fills the structure with human-readable components such as year, month, day, hour, and minute according to the system's local timezone

Line 19. Uses `strftime` to format the `tm` structure according to the format specifier provided via the command line. The resulting formatted string is stored in `ostr`

We execute the program passing a string with some specifiers²⁴.

```
[linux@fidir] ./timeformat "%d %h %Y - %H:%M"
```

```
Result string is "29 Apr 2025 - 11:23"
```

```
[linux@fidir] ./timeformat "%d/%m/%Y - %H:%M"
```

```
Result string is "29/04/2025 - 11:31"
```

In these examples, the program interprets the format string provided on the command line and applies it to the current local time obtained via `localtime`. The `strftime` function then replaces each conversion specifier with the corresponding value from the `struct tm`.

- `%d` → day of the month (01–31)
- `%m` → month as a number (01–12)
- `%h` → abbreviated month name (e.g., Apr)
- `%Y` → full year (e.g., 2025)

²⁴ The complete list of the specifiers is available in the `strftime(3)` manual page

- %H → hour in 24-hour format (00–23)
- %M → minutes (00–59)

To perform the reverse operation, that is, converting a date and time string into a `tm` structure representation, we can use the following function²⁵.

```
#define _XOPEN_SOURCE
#include <time.h>

char *strptime(const char *restrict s, const char *restrict format,
               struct tm *restrict tm);

Returns: a pointer to the first character not processed, NULL if error
```

The program `strtotime.c` converts a date-time string into a `tm` structure according to the format specifiers provided.

```
1 #define _XOPEN_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>

5 int main(int argc, char *argv[])
6 {
7     struct tm tm;
8     if (argc != 3) {
9         printf("\nUsage: %s <datetime> <specifiers>", argv[0]);
10        exit(EXIT_FAILURE);
11    }

12    strptime(argv[1], argv[2], &tm);
13    printf("\nDAY: %d", tm.tm_mday);
14    printf("\nMONTH: %d", tm.tm_mon + 1);
15    printf("\nYEAR: %d", tm.tm_year + 1900);
16    printf("\nHOUR: %d", tm.tm_hour);
17    printf("\nMIN: %d", tm.tm_min);
18    exit(EXIT_SUCCESS);
19 }
```

strtotime.c

The line 12 calls `strptime`, which parses the input string (`argv[1]`) according to the format string (`argv[2]`). The function fills the corresponding fields of the `tm` structure based on the provided format specifiers, converting the textual representation of date and time into a structured format that can be further processed in C.

25 See the `strptime(3)` manual page

We execute the program

```
[linux@fidir] ./strptime "29/04/2025 11:50" "%d/%m/%Y %H:%M"  
DAY: 29  
MONTH: 4  
YEAR: 2025  
HOUR: 11  
MIN: 50
```

the program has correctly parsed the input string "29/04/2025 11:50" according to the format specifier "%d/%m/%Y %H:%M" and populated the corresponding fields of the `tm` structure. The values stored in `tm` match the expected date and time components, confirming that `strptime` successfully interpreted the input and converted it into a structured representation.

Once the structure `tm` has been populated, we can pass it to `strptime` to print the time in another format (e.g adding the following code after line 17)

```
char outstr[200];  
if (strptime(outstr, sizeof(outstr), "Time: %Y-%m-%d", &tm) == 0) {  
    fprintf(stderr, "strptime returned 0");  
    exit(EXIT_FAILURE);  
}  
printf("\n%s", outstr);
```

III.1.7 Time conversion: `getdate`

The `getdate`²⁶ and `getdate_r` functions are used to convert a *date-plus-time* string to *broken-down time*.

```
#include <time.h>

struct tm *getdate(const char *string);

extern int getdate_err;

int getdate_r(const char *restrict string, struct tm *restrict res);

    Return: a pointer to a struct tm, on error NULL and sets getdate_err
           0 on success, the error on fail
```

The `getdate` function converts the string passed in the `string` argument into a broken-down time. The broken-down time is stored in a `tm` structure. To specify the format of the input string, `getdate` uses a file that is specified in the `DATMSK` environment variable. To use this file, we need to set this variable in the environment before calling `getdate`. If the function fails, it sets the error in the `getdate_err` global variable.

The `getdate_r` function is a GNU extension that does not use a global variable to report errors but the return value, and that store the broken-down time in a the second parameter `res`.

The following table lists the errors `getdate_r` may return.

1	DATMSK is not defined
2	The file specified in DATMSK cannot be opened for reading
3	Failed to get file status information
4	The file is not a regular file
5	Error during the reading of the file
6	Not enough memory available to allocate
7	No line in the file matches the input
8	Invalid input specification

Tab 3. Error of `getdate_r`

The program `getdate.c` shows an example.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5 #include <errno.h>
```

26 See the `getdate(3)` manual page

```

6 int main(int argc, char *argv[])
7 {
8     if (argc != 2) {
9         printf("\nUsage %s '1 January 1980 15:00' ", argv[0]);
10        exit(EXIT_FAILURE);
11    }

12    setenv("DATEMSK", "./datemsk.txt", 1);
13    const char *date_str = argv[1];
14    struct tm *tm_ptr = getdate(date_str);

15    if (tm_ptr == NULL) {
16        perror("getdate");
17        fprintf(stderr, "getdate error code: %d\n", getdate_err);
18        return 1;
19    }

20    printf("Parsed date and time:\n");
21    printf("Year: %d\n", tm_ptr->tm_year + 1900);
22    printf("Month: %d\n", tm_ptr->tm_mon + 1);
23    printf("Day: %d\n", tm_ptr->tm_mday);
24    printf("Hour: %d\n", tm_ptr->tm_hour);
25    printf("Minute: %d\n", tm_ptr->tm_min);
26    return 0;
27 }

```

getdate.c

- Line 12. Sets the DATEMSK environment variable to ./datemsk.txt, which specifies the path to a format template file. This file contains one or more valid date/time format strings used by getdate to interpret the input*
- Line 14. Calls getdate to convert the input string (argv[1]) into a struct tm. The conversion is performed by matching the input against the format(s) defined in the file referenced by DATEMSK. If a valid match is found, a pointer to a statically allocated struct tm is returned*
- Line 15–19. Checks whether getdate returned NULL, indicating a parsing failure. If an error occurs, perror prints a system error message, and getdate_err provides a more specific error code describing why the conversion failed (e.g., no valid match in the template file or invalid input format)*
- Line 20–25. Prints the parsed date and time fields stored in the resulting struct tm, converting them into human-readable values. Adjustments are applied for tm_year (offset from 1900) and tm_mon (zero-based indexing)*

We create the file and execute the program.

```
[linux@users] echo %d %B %Y %H:%M > datemsk.txt
```

```
[linux@users] cat datemsk.txt
```

```
[linux@users] ./getdate '16 June 2025 13:00'
```

Parsed date and time:

Year: 2025

Month: 6

Day: 16

Hour: 13

Minute: 0

The program successfully reads the format patterns from the `datemsk.txt` file and uses them to parse the input string "16 June 2025 13:00" via `getdate`. The function matches the string against the specified format, converts it into a `tm` structure, and correctly extracts each field (day, month, year, hour, and minute). The printed output confirms that the parsing operation was successful and that the date-time string was accurately interpreted according to the external template.

III.2 The environment

On Linux systems we have always to deal with the environment. This is composed by the configuration related to the shell, the processes, the file system, the users and groups and many other settings. The behavior of the applications can change or can be modified based on the environment settings. The most of these setting are related to the user-level and affect the way a user, process or group behave executing programs and acting on the filesystem.

The most important part of the environment is the set of the environment variables that we can display with the `env`²⁷ command.

```
[linux@users] env  
...output...
```

The following table lists some of the most common variables²⁸.

LANG	The language of the system
USER	Username of the user currently logged in
HOME	The home folder of the user
PWD ²⁹	The current working directory
PATH	The paths list where the shell search for the commands entered on the command line
LOGNAME	Login name (POSIX)
SHELL	Default shell
TERM	Default terminal
DISPLAY	Default display
OLDPWD	Previous CWD

Tab 4. Common environment variables

²⁷ See the `env(1)` manual page

²⁸ See the `environ(7)` manual page

²⁹ PWD stands for *Process Working Directory*, in the shell it usually corresponds to CWD

III.2.1 Working with environment variables: `getenv` and `setenv`

Environment variables can be set through the shell or stored in files that are read during system boot or user login.

When a program is executed, it runs within the environment defined by these variables and other system settings. In C, the `main` function can accept the environment as a third parameter. Similarly, the `execve` function takes the environment as an argument when executing an external program.

The program `showenv.c` displays all the environment variables using the `envp` parameter of the `main` function.

```
1 #include <stdio.h>

2 int main(int argc, char *argv[], char *envp[]) {
3     printf("Environment variables:\n\n");

4     for (int i = 0; envp[i] != NULL; i++) {
5         printf("%s\n", envp[i]);
6     }
7     return 0;
8 }
```

showenv.c

The `envp` argument is automatically populated with the environment variables. The `for` loop at line 4, retrieves and prints all the elements of the `envp` array.

```
[linux@users] ./showenv
```

Environment variables:

HOME=/home/users

LANG=en_US.UTF-8

LOGNAME=users

PATH=/home/users/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/bin:/bin:/usr/local/

...output...

To retrieve the value of a variable from the environment we can use the following functions³⁰.

```
#include <stdlib.h>

char *getenv(const char *name);

char *secure_getenv(const char *name);

Return: a pointer to the value, NULL on error
```

30 See the `getenv(3)` manual page

The `getenv` function returns the value of the variable `name`. The `secure_getenv` function returns `NULL` when the *secure execution* is required. The *secure execution* is required when one of the following conditions is true

1. The process's effective user ID did not match its real user ID or the process effective group ID did not match its real group ID (set-user-ID, set-group-ID programs).
2. The *effective capability bit* is set on the executable file
3. The process has a nonempty capability set

To set the value of an environment variable we can use the following functions³¹.

```
#include <stdlib.h>

int setenv(const char *name, const char *value, int overwrite);

int unsetenv(const char *name);

Return: 0 on success, -1 on error and sets errno
```

The `setenv` function sets the value `value` of the `name` environment variable and overwrite it if `overwrite` is nonzero. This function makes copies of the strings pointed to by `name` and `value`. The `unsetenv` function deletes the variable from the environment.

In addition, the `putenv`³² function can be used to set/change the value of an environment variable.

```
#include <stdlib.h>

int putenv(char *string);

Returns: 0 on success, nonzero on error and sets errno
```

The `string` argument is in the form `name=value`. If `name` does not already exist in the environment is added to it. If `string` does not include an equal (=) sign then the variable `name` is removed from the environment. `putenv` does not make copies of the string pointed to by `string`. The string pointed to by `string` becomes part of the environment, so altering the string changes the environment.

The `clearenv`³³ function clears the environment of all *name-value* pairs and sets the value of the external variable `environ` to `NULL`.

```
#include <stdlib.h>

int clearenv(void);

Returns: 0 on success, nonzero on error
```

³¹ See the `setenv(3)` manual page

³² See the `putenv(3)` manual page

³³ See the `clearenv(3)` manual page

On Linux, the environment is explained in the `environ(7)` manual page. It is defined as a pointer to an array of pointers to strings named `environ`.³⁴

```
extern char **environ;
```

To print the environment is also available the `printenv`³⁵ command.

```
[linux@users] printenv  
...output...
```

The `env_manage.c` program demonstrates how to manipulate and inspect environment variables from within a C program.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>

4 int main(int argc, char *argv[], char *envp[]) {
5     if (argc < 2 || (argc != 3 && strcmp(argv[1], "print") != 0)) {
6         fprintf(stderr, "Usage:\n");
7         fprintf(stderr, " %s add <VAR_NAME> <VALUE>\n", argv[0]);
8         fprintf(stderr, " %s change <VAR_NAME> <NEW_VALUE>\n",
9                 argv[0]);
10        fprintf(stderr, " %s delete <VAR_NAME>\n", argv[0]);
11        fprintf(stderr, " %s print\n", argv[0]);
12        return 1;
13    }

14    const char *action = argv[1];
15    const char *var_name = argv[2];
16    const char *var_value = (argc >= 4) ? argv[3] : NULL;

17    if (strcmp(action, "add") == 0) {
18        if (var_value == NULL) {
19            fprintf(stderr, "Missing value for 'add'\n");
20            return 1;
21        }
22        if (getenv(var_name)) {
23            printf("Variable %s already exists: %s\n", var_name,
24                  getenv(var_name));
25        } else {
26            if (setenv(var_name, var_value, 0) != 0) {
27                perror("setenv");
28                return 1;
29            }
30            printf("Added %s=%s\n", var_name, getenv(var_name));
31        }
32    }
33 }
```

³⁴ The reader is encouraged to read also the `environ(3POSIX)` POSIX manual page:

³⁵ See the `printenv(1)` manual page

```

30     } else if (strcmp(action, "change") == 0) {
31         if (var_value == NULL) {
32             fprintf(stderr, "Missing value for 'change'\n");
33             return 1;
34         }
35         if (setenv(var_name, var_value, 1) != 0) {
36             perror("setenv");
37             return 1;
38         }
39         printf("Changed %s=%s\n", var_name, getenv(var_name));
40     } else if (strcmp(action, "delete") == 0) {
41         if (unsetenv(var_name) != 0) {
42             perror("unsetenv");
43             return 1;
44         }
45         printf("Deleted %s (current value: %s)\n", var_name,
46               getenv(var_name) ? getenv(var_name) : "NULL");
47     } else if (strcmp(action, "print") == 0) {
48         for (int i = 0; envp[i] != NULL; i++) {
49             printf("%s\n", envp[i]);
50         }
51     } else {
52         fprintf(stderr, "Unknown action: %s (use add, change, delete
53               or print)\n", action);
54         return 1;
55     }
56     return 0;

```

env_manage.c

- Line 16–29. Adds a new environment variable if it does not already exist. It first checks whether the variable is already defined using `getenv`. If it exists, the program prints its current value. Otherwise, it uses `setenv` with the `overwrite` flag set to 0 to create the variable without replacing an existing one*
- Line 30–39. Changes the value of an existing environment variable. The function `setenv` is called with the `overwrite` flag set to 1, which forces the update even if the variable already exists. The updated value is then retrieved using `getenv` and displayed*
- Line 40–46. Deletes an existing environment variable. The function `unsetenv` removes the variable from the environment. After deletion, the program attempts to print its value, which should return `NULL` since the variable no longer exists*
- Line 47–50. Prints all environment variables. The program iterates through the `envp` array, which contains all environment strings in the form `NAME=VALUE`, and prints each entry line by line*

```
[linux@users] ./env_manage
```

Usage:

```
./env_manage add <VAR_NAME>
```

```
./env_manage change <VAR_NAME> <NEW_VALUE>
```

```
./env_manage delete <VAR_NAME>
```

The reader is encouraged to run the program in order to test the different available options and observe how each operation affects the environment variables in practice.

III.2.2 Environment during an `execve`

When using `execve`, the child process can be executed with different environments depending on the arguments provided. The child:

- may inherit exactly the same environment as the parent process
- run with a completely custom environment
- execute with the parent's environment extended by additional user-defined variables

This flexibility allows programs to control how external applications access configuration settings and environment variables.

The `getvar.c` program retrieves the value of the environment variable passed as its first argument.

```
1 #include <stdio.h>
2 #include <stdlib.h>

3 int main (int argc, char *argv[]) {
4     if (argc != 2) {
5         printf("\nUsage %s <env_var>", argv[0]);
6         exit(EXIT_FAILURE);
7     }

8     char *value;
9     value = getenv(argv[1]);
10    printf("\nVAR: %s=%s", argv[1], value);

11    exit(EXIT_SUCCESS);
12 }
```

getvar.c

The function `getenv` searches the process environment for the variable name provided in `argv[1]` and returns a pointer to its value if it exists.

Custom environment

The program `exec_with_env.c` executes `getvar.c` using `execve`³⁶, explicitly providing a custom environment.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>

4 int main(int argc, char *argv[]) {
```

³⁶ See the `execve(2)` manual page

```

5     if (argc != 2) {
6         printf("\nUsage: %s <env_var>", argv[0]);
7         exit(EXIT_FAILURE);
8     }

9     char *path = "./getvar";
10    char *const args[] = { "./getvar", argv[1], NULL };

11    char *const envp[] = {
12        "USER=example_user",
13        "HOME=/home/example_user",
14        "MY_VAR=HelloWorld",
15        NULL
16    };

17    if (execve(path, args, envp) == -1) {
18        perror("execve");
19        exit(EXIT_FAILURE);
20    }

21    exit(EXIT_SUCCESS);
22 }

```

exec_with_env.c

Line 9. Defines the pathname of the executable to be run (`getvar`)

Line 10. Defines the argument list passed to the new program, including the program name and the environment variable to retrieve

Line 16. Defines a completely new environment (`envp`) that will replace the current process environment when executing `getvar.c`

Line 17. Calls `execve`, passing the program path, arguments, and the custom environment. If successful, the current process image is replaced by `getvar`, which then runs using only the provided environment variables

We execute the program.

```

[linux@users] ./exec_with_env USER
VAR: USER=example_user

```

```

[linux@users] ./exec_with_env HOME
VAR: HOME=/home/example_user

```

```

[linux@users] ./exec_with_env MY_VAR
VAR: MY_VAR=HelloWorld

```

```

[linux@users] ./exec_with_env DISPLAY
VAR: DISPLAY=(null)

```

```

[linux@users] ./exec_with_env LOGNAME
VAR: LOGNAME=(null)

```

When `getvar` is executed, it inherits the environment explicitly provided by the `execve` call in the parent process (`exec_with_env`). When it attempts to retrieve the values of the `USER`, `HOME`, `MY_VAR` environment variables, it succeeds and prints the value. If we pass other variables, such as `DISPLAY`, `LOGNAME` etc., it will print `null`. This because these variables do not exist in the environment of `getvar`. The program is running with his own environment that does not contain these variables.

Environment inherited from the parent

If we want a program to inherit the environment of the parent we can pass the `envp` parameter of the main function to `execve`. To do this we define `main` as follows

```
int main(int argc, char *argv[], char *envp[]) {
```

next, we comment out the lines 11-16

```
/*char *const envp[] = {
    "USER=example_user",
    "HOME=/home/example_user",
    "MY_VAR=HelloWorld",
    NULL
};*/
```

Now, the `execve` call at line 17, will pass to `getvar.c` the `envp` of `main` instead of a custom defined environment.

```
if (execve(path, args, envp) == -1) {
```

```
[linux@users] ./exec_with_env USER
VAR: USER=users

[linux@users] ./exec_with_env DISPLAY
VAR: DISPLAY=:0

[linux@users] ./exec_with_env LOGNAME
VAR: LOGNAME=users
```

The `USER` environment variable contains the name of the current user (`users`), while `DISPLAY` and `LOGNAME` are resolved to their respective values.

The `getvar.c` program is executed within the same environment as `exec_with_env.c`, namely the environment of the user `users`.

Extended inherited environment

If we want to extend the parent's environment with additional variables instead of replacing it completely, we must first copy the existing `envp` array into a new array, then append the new variables to it. Finally, this updated environment array can be passed to `execve`, ensuring that both the original and newly added variables are available in the executed program.

The `exec_add_env.c` program demonstrates how to extend the current environment by copying it into a new array, adding additional variables, and then passing the modified environment to `execve`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>

5 #define EXTRA_VARS 2
6 int count_env(char *envp[]) {
7     int count = 0;
8     while (envp[count] != NULL) count++;
9     return count;
10 }

11 int main(int argc, char *argv[], char *envp[]) {
12     if (argc < 2) {
13         printf("\nUsage: %s <var> [value]", argv[0]);
14         exit(EXIT_FAILURE);
15     }

16     int env_count = count_env(envp);
17     char *result;
18     int total_env = env_count + EXTRA_VARS;
19     char **new_envp = malloc((total_env + 1) * sizeof(char *));

20     if (!new_envp) {
21         perror("malloc");
22         return 1;
23     }
24     for (int i = 0; i < env_count; i++) {
25         new_envp[i] = envp[i];
26     }

27     if (argv[2]) {
28         size_t len = strlen(argv[1]) + strlen("=") + strlen(argv[2]) + 1;
29         result = malloc(len);
30         if (!result) {
31             perror("malloc");
32             return 1;
33         }
34         strcpy(result, argv[1]);
35         strcat(result, "=");
36         strcat(result, argv[2]);
37     }
38     new_envp[env_count] = argv[2] ? result : argv[1];
39     new_envp[env_count + 1] = "DEBUG=1";
40     new_envp[env_count + 2] = NULL;
```

```

41  char *program = "./getvar";
42  char *const new_argv[] = { "./getvar", argv[1], NULL };
43  if (execve(program, new_argv, new_envp) == -1) {
44      perror("execve");
45      free(new_envp);
46      return 1;
47  }
48  exit(EXIT_SUCCESS);
49 }

```

exec_add_env.c

- Line 6–10. Defines a helper function that counts the number of environment variables by iterating through the `envp` array until the `NULL` terminator is reached*
- Line 16. Calls the function to determine the size of the current environment*
- Line 18. Increases the environment size by 2 to make space for additional variables*
- Line 19. Allocates memory for the new environment array, including one extra slot for the `NULL` terminator*
- Line 24–26. Copies all existing environment variables from `envp` into the newly allocated `new_envp` array*
- Line 27–37. If a third argument is provided (the value of the new variable), a new string of the form `name=value` is dynamically constructed using `malloc`, `strcpy`, and `strcat`*
- Line 38. Adds the new variable to the environment. If a value was provided, the entry is `name=value`; otherwise, only the variable name is stored*
- Line 39. Adds an additional environment variable (`DEBUG=1`) to the new environment*
- Line 40. Adds a `NULL` pointer to terminate the new environment array, as required by `execve`*
- Line 41–42. Prepares the executable path (`getvar`) and the argument list to pass to the new program*
- Line 43. Calls `execve` to execute `getvar` with the modified environment. The current process image is replaced by the new program*
- Line 45. Frees allocated memory only if `execve` fails, since on success the process memory is replaced and this code is never executed*
- Line 48. This line is never reached because a successful `execve` does not return*

We execute the program.

```

[linux@users] ./exec_add_env USER
VAR: USER=users

[linux@users] ./exec_add_env DEBUG
VAR: DEBUG=1

```

Both the inherited `USER` variable and the newly added `DEBUG` variable are correctly resolved, confirming that standard environment variables are preserved across `execve`, while newly defined variables are also properly set in the resulting process environment.

We pass a new variable without value.

```

[linux@users] ./exec_add_env MY_VAR
VAR: MY_VAR=(null)

```

When passing a new variable without a value (`MY_VAR`), the variable is added to the custom environment but has no associated value, resulting in `(null)` when `getenv` is called in `getvar`.

We passing a new variable with a value.

```
[linux@users] ./exec_add_env NEW_VAR hellofromgetvar  
VAR: NEW_VAR=hellofromgetvar
```

When passing a new variable with an explicit value (`NEW_VAR hellofromgetvar`), the program correctly constructs the `name=value` pair, adds it to the environment, and `getvar` successfully retrieves and prints the assigned value.

These executions show that by building a custom `envp` array and passing it to `execve`, it is possible to run a program with a fully controlled environment, including inherited, modified, or newly created variables.

III.3 Locale

On Linux systems, the *locale* represents a set of rules that define regional and language-specific behavior for a program's runtime environment. With the `locale`³⁷ command we can see the current locale settings.

```
[linux@users] locale
LANG=en_US.UTF-8
LANGUAGE=
LC_CTYPE="en_US.UTF-8"
LC_NUMERIC="en_US.UTF-8"
LC_TIME="en_US.UTF-8"
LC_COLLATE="en_US.UTF-8"
LC_MONETARY="en_US.UTF-8"
LC_MESSAGES="en_US.UTF-8"
LC_PAPER="en_US.UTF-8"
LC_NAME="en_US.UTF-8"
LC_ADDRESS="en_US.UTF-8"
LC_TELEPHONE="en_US.UTF-8"
LC_MEASUREMENT="en_US.UTF-8"
LC_IDENTIFICATION="en_US.UTF-8"
LC_ALL=
```

The following table lists some of these settings.

Category	Description	Example
LC_CTYPE	Character classification and case conversion	Determines if ä is a letter
LC_COLLATE	String sorting rules	Sort order: ä vs z
LC_TIME	Date and time formats	15.06.2025 vs 06/15/2025
LC_NUMERIC	Decimal and thousands separators	1.000,50 vs 1,000.50
LC_MONETARY	Currency format	€1.234,56 vs \$1,234.56
LC_MESSAGES	Language used for system messages	"Permission denied" in French
LC_ALL	Overrides all of the above	Used for testing or global set
LANG	Default locale if no LC_* is defined	Used as a fallback

Tab 5. Locale settings

These settings can be specified through environment variables, as follows.

```
[linux@users] export LANG=en_US.UTF-8
[linux@users] export LC_TIME=it_IT.UTF-8
```

³⁷ See the `locale(1)` manual page

III.3.1 Setting locale: `setlocale`

We can use the `setlocale`³⁸ function to set or query the program's current locale.

```
#include <locale.h>

char *setlocale(int category, const char *locale);

Returns: string corresponding to the locale set, NULL on error
```

The `category` argument may take one of the following values.

LC_ALL	All of the locale
LC_ADDRESS	Formatting of addresses and geography-related items (*)
LC_COLLATE	String collation
LC_CTYPE	Character classification
LC_IDENTIFICATION	Metadata describing the locale (*)
LC_MEASUREMENT	Settings related to measurements (metric versus US customary) (*)
LC_MESSAGES	Localizable natural-language messages
LC_MONETARY	Formatting of monetary values
LC_NAME	Formatting of salutations for persons (*)
LC_NUMERIC	Formatting of nonmonetary numeric values
LC_PAPER	Settings related to the standard paper size (*)
LC_TELEPHONE	Formats to be used with telephone services (*)
LC_TIME	Formatting of date and time values

Tab 6. Locale categories

The categories marked with an asterisk are GNU extensions. Many details about these categories are available in the `locale(7)` manual page. The data types and functions are defined in the `locale.h` header file. The following table shows a list of the current implemented functions.

<code>setlocale</code>	Set/return the current locale
<code>localeconv</code>	Returns the numeric/monetary information for the current locale
<code>newlocale</code>	Returns a reference to a data structure representing a set of locale dataset
<code>duplocale</code>	Return a duplicate of the set of locale
<code>freelocale</code>	Free the data associated with a locale dataset previously returned by <code>setlocale_r</code>
<code>uselocale</code>	Switch the current thread's locale to a new one

Tab 7. Locale functions

³⁸ See the `setlocale(3)` manual page

To handle numeric and monetary formatting, C provides the `lconv` structure, defined as follows.

```
struct lconv
{
    /* Numeric (non-monetary) information. */
    char *decimal_point; /* Decimal point character. */
    char *thousands_sep; /* Thousands separator. */
    char *grouping;
    /* Monetary information. */
    char *int_curr_symbol;
    char *currency_symbol; /* Local currency symbol. */
    char *mon_decimal_point; /* Decimal point character. */
    char *mon_thousands_sep; /* Thousands separator. */
    char *mon_grouping; /* Like `grouping' element (above). */
    char *positive_sign; /* Sign for positive values. */
    char *negative_sign; /* Sign for negative values. */
    char int_frac_digits; /* Int'l fractional digits. */
    char frac_digits; /* Local fractional digits. */
    /* 1 if currency_symbol precedes a positive value, 0 if succeeds. */
    char p_cs_precedes;
    /* 1 if a space separates currency_symbol from a positive value. */
    char p_sep_by_space;
    /* 1 if currency_symbol precedes a negative value, 0 if succeeds. */
    char n_cs_precedes;
    /* 1 iff a space separates currency_symbol from a negative value. */
    char n_sep_by_space;
    /* Positive and negative sign positions:
       char p_sign_posn;
       char n_sign_posn;
    */
#ifdef __USE_ISOC99
    /* 1 if int_curr_symbol precedes a positive value, 0 if succeeds. */
    char int_p_cs_precedes;
    /* 1 if a space separates int_curr_symbol from a positive value. */
    char int_p_sep_by_space;
    /* 1 if int_curr_symbol precedes a negative value, 0 if succeeds. */
    char int_n_cs_precedes;
    /* 1 if a space separates int_curr_symbol from a negative value. */
    char int_n_sep_by_space;
    /* Positive and negative sign positions:
       char int_p_sign_posn;
       char int_n_sign_posn;
    */
#else
    char __int_p_cs_precedes;
    char __int_p_sep_by_space;
    char __int_n_cs_precedes;
    char __int_n_sep_by_space;
    char __int_p_sign_posn;
    char __int_n_sign_posn;
#endif
};
```

III.3.2 Working with locale: `localeconv`, `newlocale` and `uselocale`

The function `localeconv`³⁹ returns a pointer to a struct `lconv` for the current locale that contains all values associated with the locale categories `LC_NUMERIC` and `LC_MONETARY`

```
#include <locale.h>
```

```
struct lconv *localeconv(void);
```

Returns: a pointer to a filled in struct `lconv`

The program `locale_info.c` shows an example.

```
1 #include <stdio.h>
2 #include <locale.h>
3 #include <stdlib.h>

4 int main(int argc, char *argv[]) {

5     if (argc != 2) {
6         printf("\nUsage: %s <LANG>\n", argv[0]);
7         printf("\nExample:\n");
8         printf("%s en_US.UTF-8\n", argv[0]);
9         printf("%s fr_FR.UTF-8\n", argv[0]);
10        exit(EXIT_FAILURE);
11    }

12    if (!setlocale(LC_ALL, argv[1])) {
13        perror("setlocale");
14        exit(EXIT_FAILURE);
15    }

16    struct lconv *lc = localeconv();
17    printf("Locale settings:\n");
18    printf("Decimal point : '%s'\n", lc->decimal_point);
19    printf("Thousands separator : '%s'\n", lc->thousands_sep);
20    printf("Currency symbol : '%s'\n", lc->currency_symbol);
21    printf("Positive sign : '%s'\n", lc->positive_sign);
22    printf("Negative sign : '%s'\n", lc->negative_sign);
23    printf("International currency symbol: '%s'\n",
24           lc->int_curr_symbol);

24    exit(EXIT_SUCCESS);
25 }
```

locale_info.c

³⁹ See the `localeconv(3)` manual page

We execute the program passing different locales.

```
[linux@users] ./locale_info en_US.UTF-8
Locale settings:
Decimal point      : '.'
Thousands separator : ','
Currency symbol    : '$'
Positive sign      : ''
Negative sign      : '-'
International currency symbol: 'USD '

[linux@users] ./locale_info fr_FR.UTF-8
Locale settings:
Decimal point      : ','
Thousands separator : ''
Currency symbol    : '€'
Positive sign      : ''
Negative sign      : '-'
International currency symbol: 'EUR '

[linux@users] ./locale_info ja_JP.utf8
Locale settings:
Decimal point      : '.'
Thousands separator : ','
Currency symbol    : '¥'
Positive sign      : ''
Negative sign      : '-'
International currency symbol: 'JPY '
```

We pass three different locale identifiers: `en_US.UTF-8`, `fr_FR.UTF-8`, and `ja_JP.utf8`. The output of the program changes depending on the locale selected at runtime.

Program behavior is influenced by locale settings, which determine how language- and region-specific conventions are applied. Formatting details such as the decimal point, thousands separator, currency symbol, and international currency representation vary across locales.

As observed, the same program produces different outputs for each locale, confirming that execution is context-dependent and relies on the environment in which it runs.

To retrieve the full list of available locales on the system, we can use the `locale` command with the `-a` option.

```
[linux@users] locale -a
C
C.utf8
POSIX
aa_DJ
aa_DJ.utf8
aa_ER
...output...
```

C, C.UTF-8, and POSIX are standard or special-purpose base locales that define default system-wide formatting and behavior. The C locale is the original default locale defined by the C standard. POSIX is functionally equivalent to the C locale, while C.UTF-8 is a variant that extends the C locale by supporting UTF-8 encoding.

The functions `nl_langinfo`⁴⁰ and `nl_langinfo_l` are used to query language and locale information in a more flexible way. They are defined as follows.

```
#include <langinfo.h>

char *nl_langinfo(nl_item item);

char *nl_langinfo_l(nl_item item, locale_t locale);

Return: a pointer to a string with the value of item, if no locale has been
        selected by setlocale for the appropriate category, return a pointer to
        the corresponding string in the "C" locale
```

The function `nl_langinfo` returns a string with the value corresponding to `item` in the program's current locale. The function `nl_langinfo_l` returns a string with the value corresponding to `item` for the locale identified by the `locale` parameter, which was previously created by a call to `newlocale`.

The `newlocale`⁴¹ function creates a new *locale object*, while `freelocale` releases the memory associated with a locale object. These two functions are defined as follows.

```
#include <locale.h>

locale_t newlocale(int category_mask, const char *locale, locale_t
                  base);

void freelocale(locale_t locobj);

Return: a handle of locale_t type
```

Concerning the `nl_langinfo` function, the values that can be specified in the `item` argument is listed in the following table.

CODESET	Return a string with the name of the character encoding used in the selected locale, such as "UTF-8", "ISO-8859-1", or "ANSI_X3.4-1968"
D_T_FMT	Return a string that can be used as a format string for <code>strftime</code> to represent time and date in a locale-specific way
D_FMT	Return a string that can be used as a format string for <code>strftime</code> to represent a date in a locale-specific way
T_FMT	Return a string that can be used as a format string for <code>strftime</code> to represent a time in a locale-specific way

⁴⁰ See the `nl_langinfo(3)` manual page

⁴¹ See the `newlocale(3)` manual page

AM_STR	Return a string that represents affix for ante meridiem (before noon, "AM") time
PM_STR	Return a string that represents affix for post meridiem (before midnight, "PM") time
T_FMT_AMP	Return a string that can be used as a format string for <code>strftime</code> to represent a time in a.m. or p.m. notation in a locale-specific way
ERA	Return era description, which contains information about how years are counted and displayed for each era in a locale. The format is : direction:offset:start_date:end_date:era_name:era_format
ERA_D_T_FMT	Return a string that can be used as a format string for <code>strftime</code> for alternative representation of time and date in a locale-specific way
ERA_D_FMT	Return a string that can be used as a format string for <code>strftime</code> for alternative representation of a date in a locale-specific way
ERA_T_FMT	Return a string that can be used as a format string for <code>strftime</code> for alternative representation of a time in a locale-specific way
DAY_{1-7}	Return name of the n-th day of the week
ABDAY_{1-7}	Return abbreviated name of the n-th day of the week
MON_{1-12}	Return name of the n-th month
ABMON_{1-12}	Return abbreviated name of the n-th month
RADIXCHAR	Return radix character (decimal dot, decimal comma, etc.)
THOUSEP	Return separator character for thousands (groups of three digits)
YESEXPR	Return a regular expression that can be used with the <code>regex</code> function to recognize a positive response to a yes/no question
NOEXPR	Return a regular expression that can be used with the <code>regex</code> function to recognize a negative response to a yes/no question
CRNCYSTR	Return the currency symbol, preceded by "-" if the symbol should appear before the value, "+" if the symbol should appear after the value, or "." if the symbol should replace the radix character

Tab 8. Values for the parameter `item` of `nl_langinfo`

The `nlocale.c` program demonstrates how to use `newlocale` and `freelocale` to create and manage locale objects, and how locale settings affect formatted output.

```

1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <locale.h>
4 #include <langinfo.h>
5 #include <stdlib.h>

6 int main(int argc, char *argv[]) {
7     if (argc != 2) {
8         printf("\nUsage: %s <locale>", argv[0]);
9         exit(EXIT_FAILURE);

```

```

10     }

11     const char *locale_name = argv[1];
12     locale_t new_locale = newlocale(LC_ALL_MASK, locale_name, NULL);
13     if (new_locale == (locale_t)0) {
14         perror("newlocale");
15         return 1;
16     }

17     locale_t old_locale = uselocale(new_locale);
18     printf("Locale: %s\n", locale_name);
19     printf("Day 1: %s\n", nl_langinfo(DAY_1));
20     printf("Abbrev month 1: %s\n", nl_langinfo(ABMON_1));
21     printf("Currency symbol: %s\n", nl_langinfo(CURRENCY_SYMBOL));
22     printf("Decimal point: %s\n", nl_langinfo(RADIXCHAR));
23     uselocale(old_locale);
24     freelocale(new_locale);
25     return 0;
26 }

```

nlocale.c

- Line 12. Creates a new `locale_t` object using `newlocale`, based on the locale name provided as a command-line argument. This object defines a set of locale-specific rules for formatting dates, numbers, currency, and language-dependent strings*
- Line 17. Switches the current thread's locale to the newly created locale using `uselocale`, while storing the previous locale so it can be restored later*
- Line 18–22. Prints various locale-dependent values using `nl_langinfo`, such as the name of the first day of the week, abbreviated month names, currency symbol, and decimal separator. These values change depending on the active locale*
- Line 23. Restores the previous locale to ensure that further operations are not affected by the temporary locale change*
- Line 24. Frees the memory associated with the newly created locale object using `freelocale`, preventing memory leaks*

We execute the program passing different locales.

```
[linux@users] ./nlocale en_US.UTF-8
```

```
Locale: en_US.UTF-8
```

```
Day 1:      Sunday
```

```
Abbrev month 1: Jan
```

```
Currency symbol: $
```

```
Decimal point: .
```

```
[linux@users] ./nlocale zh_CN.utf8
```

```
Locale: zh_CN.utf8
```

```
Day 1:      星期日
```

```
Abbrev month 1: 1月
```

```
Currency symbol: ¥
```

```
Decimal point: .
```

At runtime, the program creates and activates the requested locale using `newlocale` and `uselocale`, and then retrieves locale-dependent information through `nl_langinfo`. The `en_US.UTF-8` locale returns properly formatted English values, while `zh_CN.utf8` displays the corresponding values in the respective language.

The `uselocale`⁴² function is used to set the current locale for the process and is defined as follows.

```
#include <locale.h>

locale_t uselocale(locale_t newloc);

Returns: the locale handle, LC_GLOBAL_LOCALE if there was no a previous call,
on error (locale_t)0 and sets errno
```

The `newloc` argument may take one of the following values:

1. A handle returned by a call to `newlocale` or `duplocale`. The thread current locale is set to the specified locale.
2. The special locale object handle `LC_GLOBAL_LOCALE`. The thread current locale is set to the global locale.
3. `(locale_t)0`. The thread current locale is left unchanged/

42 See the `uselocale(3)` manual page

III.3.3 Duplicating a locale object: duplocale

The `duplocale`⁴³ function is used to duplicate a locale object, it is defined as follows.

```
#include <locale.h>

locale_t duplocale(locale_t locobj);

Returns: a handle for the new locale object, (locale_t)0 on error and sets
errno
```

The `toupper.c` program converts a given string to uppercase. It uses `duplocale` to create a duplicate of the current locale, which is then passed to `toupper_l`⁴⁴ for locale-aware character conversion. This approach is necessary because `uselocale` may return the special value `LC_GLOBAL_LOCALE`. That value cannot be safely used as an argument to `toupper_l`. By duplicating the locale with `duplocale`, the program ensures it has a valid, usable locale object for the conversion function. The program is taken from the `duplocale(3)` manual page.

```
1 #define _XOPEN_SOURCE 700
2 #include <ctype.h>
3 #include <locale.h>
4 #include <stdio.h>
5 #include <stdlib.h>

6 #define errExit(msg) do { perror(msg); exit(EXIT_FAILURE); } while (0)

7 int main(int argc, char *argv[])
8 {
9     locale_t loc, nloc;

10    if (argc != 2) {
11        fprintf(stderr, "Usage: %s string\n", argv[0]);
12        exit(EXIT_FAILURE);
13    }

14    loc = uselocale((locale_t) 0);
15    if (loc == (locale_t) 0)
16        errExit("uselocale");

17    nloc = duplocale(loc);
18    if (nloc == (locale_t) 0)
19        errExit("duplocale");

20    for (char *p = argv[1]; *p; p++)
21        putchar(toupper_l(*p, nloc));
22    printf("\n");
23    freelocale(nloc);
```

⁴³ See the `duplocale(3)` manual page

⁴⁴ See the `toupper(3)` manual page

```
24     exit(EXIT_SUCCESS);  
25 }
```

toup.c

- Line 6. Defines a macro `errExit(msg)` that prints an error message using `perror` and immediately terminates the program with `exit(EXIT_FAILURE)`*
- Line 9. Declares two `locale_t` variables used to store the current locale and its duplicate*
- Line 14. Retrieves the current thread-specific locale using `uselocale((locale_t)0)` without modifying it, allowing the program to access the active locale safely*
- Line 17. Creates a duplicate of the current locale using `duplocale`, which allows the program to use a separate locale object without affecting the global or thread locale*
- Line 20–21. Iterates through each character of the input string and converts it to uppercase using `toupper_l`, which performs locale-aware character conversion based on the duplicated locale. The result is printed character by character*
- Line 23. Frees the memory allocated for the duplicated locale object using `freelocale`*

We execute the program.

```
[linux@users] ./toup hello
```

```
HELLO
```

The program receives the input string "hello", processes each character using `toupper_l` with the current locale, and converts all lowercase letters to uppercase. The resulting transformed string is then printed to the standard output.

III.4 The User Database: `/etc/passwd`

III.4.1 User and group files

UNIX was designed to be a multi-user operating system, this fact has affected all the subsequent systems based on it, including Linux. On Linux the users belong to one or more groups. If we create a new user, a new group with the same name of the user will be created in a way that the user will belong to that group. This because on Linux a user must belong to at least one group, it is not possible for a user to not belong to any group. When a user is created it is assigned to it an user ID (UID), a group ID (GID) and optionally the supplementary groups.

The system's groups are stored in the file `/etc/group`, which follows a specific syntax⁴⁵:

```
group_name:password:GID:user_list
```

`user_list` is the list of all the users that belong to that group.

The field `password` has the placeholder 'x' if the password is not set. E.g.

```
newuser:x:1000:
sudo:x:27:newuser
root:x:0:
```

If a group has a password set, it is stored in the `/etc/gshadow` file.

The list of system users is stored in the `/etc/passwd` file, which follows a defined syntax. This file is also referred to by POSIX as the *user database*.

```
login_name:encrypted_password:UID:GID:user_name:home_dir:interpreter
```

The `encrypted_password` field is optional because the actual password is typically stored in the `/etc/shadow` file; in `/etc/passwd`, this field usually contains the placeholder 'x'. The `interpreter` field specifies the shell assigned to the user when the user logs in. The `user_name` field is also known as the *comment* field. For example:

```
newuser:x:1000:1000:newuser,,,:/home/newuser:/usr/bin/zsh
```

If the password field is empty, it means that no password is set for the user, and the user can log in without entering a password. If the password field starts with an exclamation mark (!), it means that the password is locked and password-based authentication is disabled for that account.

The *comment* field is used by programs such as `finger`⁴⁶ to retrieve descriptive information about a user, including on remote systems.

The `/etc/shadow` file contains the password information for the accounts and optional aging information. The fields of the file are listed in the following table.

Login name	Account name that must exist on the system
Encrypted password	May be empty, in which case no password are required to authenticate

⁴⁵ See the `group(5)` manual page

⁴⁶ See the `finger(1)` manual page

Date of the last password change	The date of the last password change expressed as the number of days since the Epoch
Minimum password age	Number of days the user will have to wait before he can change the password again
Maximum password age	Number of days after which the user will have to change his password
Password warning period	The number of days before a password is going to expire during which the user should be warned
Password inactivity period	Number of days after a password has expired during which the password should still be accepted
Account expiration date	The date of expiration of the account, expressed as the number of days since the Epoch
Reserved field	Reserved for future use

Tab 9. Fields of the `shadow` file

Concerning the groups, the file `/etc/gshadow` contains the shadowed information. The fields of are listed in the following table.

Group name	Name of the group
Encrypted password	Group password, is used when a user who is not a member of the group wants to gain the permission of this group
Administrators	Comma-separated list of administrators of the group
Members	Comma-separated list of user names

Tab 10. Fields of the `gshadow` file

The following table provides a summary of the files and commands used to manage users and groups.

File	Commands	Notes
<code>/etc/passwd</code>	<code>useradd</code> , <code>usermod</code> , <code>userdel</code>	User account information
<code>/etc/shadow</code>	<code>passwd</code> , <code>chage</code>	Hashed passwords and aging
<code>/etc/group</code>	<code>groupadd</code> , <code>groupmod</code> , <code>groupdel</code> , <code>gpasswd</code>	Group information
<code>/etc/gshadow</code>	<code>gpasswd</code>	Group passwords and administrators

Tab 11. Files and commands for user and groups

III.4.2 The /etc/passwd file

To retrieve an entry from the users database (/etc/passwd), we can use the following functions⁴⁷

```
#include <sys/types.h>
#include <pwd.h>

struct passwd *getpwnam(const char *name);
struct passwd *getpwuid(uid_t uid);

int getpwnam_r(const char *restrict name, struct passwd *restrict pwd,
               char buf[restrict .buflen], size_t buflen, struct passwd
               **restrict result);

int getpwuid_r(uid_t uid, struct passwd *restrict pwd, char buf[restrict
               .buflen], size_t buflen, struct passwd **restrict
               result);

Return: a pointer to a passwd structure, NULL if there is no matching entry, on
        error errno is set,
        0 on success and set *result to pwd, 0 if there is no matching entry and NULL
        in *result
```

The `getpwnam` function retrieves the record corresponding to the specified user name and returns it as a structure of type `passwd`.

The `getpwuid` function retrieves the record corresponding to the user ID specified in the `uid` parameter and also returns it as a `passwd` structure.

The functions `getpwnam_r` and `getpwuid_r` retrieve the same information using a reentrant interface. Instead of returning a pointer to a static structure, they store the result in the structure pointed to by the `pwd` parameter. The string fields referenced by the members of the `passwd` structure are stored in the buffer `buf`, whose size is specified by `buflen`.

The `result` argument is set to point to the filled structure on success, or to `NULL` if no matching entry is found.

The `passwd` structure is defined in the `pwd.h` header file as follows.

```
struct passwd {
    char *pw_name;           /* username */
    char *pw_passwd;         /* user password */
    uid_t pw_uid;           /* user ID */
    gid_t pw_gid;           /* group ID */
    char *pw_gecos;          /* user information */
    char *pw_dir;            /* home directory */
    char *pw_shell;          /* shell program */
};
```

When we want to get a record from the user database with `getpwnam_l` and `getpwuid_l`, we need to allocate the memory for the buffer. Considering that we don't know the size of this buffer we can use the following call to obtain an initial suggested size:

⁴⁷ See the `getpwnam(3)` manual page

```
sysconf(_SC_GETPW_R_SIZE_MAX)
```

The `getrecord.c` program is a modified version of the example provided in the `getpwnam(3)` manual page and demonstrates how to retrieve the record corresponding to the user name passed on the command line.

```
1 #include <errno.h>
2 #include <pwd.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <unistd.h>

7 int main(int argc, char *argv[])
8 {
9     struct passwd pwd;
10    struct passwd *result;
11    char *buf;
12    long bufsize;
13    int s;

14    if (argc != 2) {
15        fprintf(stderr, "Usage: %s username\n", argv[0]);
16        exit(EXIT_FAILURE);
17    }

18    bufsize = sysconf(_SC_GETPW_R_SIZE_MAX);
19    if (bufsize == -1)
20        bufsize = 16384;

21    buf = malloc(bufsize);
22    if (buf == NULL) {
23        perror("malloc");
24        exit(EXIT_FAILURE);
25    }

26    s = getpwnam_r(argv[1], &pwd, buf, bufsize, &result);
27    if (result == NULL) {
28        if (s == 0)
29            printf("Not found\n");
30        else {
31            errno = s;
32            perror("getpwnam_r");
33        }
34        exit(EXIT_FAILURE);
35    }

36    printf("User: %s \n", pwd.pw_name);
37    printf("Password: %s \n", pwd.pw_passwd);
```

```
38  printf("UID: %d \n", (intmax_t) pwd.pw_uid);
39  printf("GID: %d \n", (intmax_t) pwd.pw_gid);
40  printf("Comment: %s \n", pwd.pw_gecos);
41  printf("Home: %s \n", pwd.pw_dir);
42  printf("Shell: %s \n", pwd.pw_shell);
43  exit(EXIT_SUCCESS);
44 }
```

getrecord.c

- Line 18. Retrieves the recommended initial size for the buffer `buf` using `sysconf(_SC_GETPW_R_SIZE_MAX)`. This value specifies the suggested buffer size for reentrant password database functions. If the system does not provide a limit, a default size of 16384 bytes is used*
- Line 21. Allocates dynamic memory for the buffer `buf`, which will store the string fields associated with the `passwd` structure*
- Line 26. Calls `getpwnam_r` to retrieve the record associated with the specified user name. The function receives the user name, the `pwd` structure where the result will be stored, the buffer `buf`, its size `bufsize`, and the pointer `result`, which indicates whether the operation succeeded*
- Line 27–35. Checks whether the record was successfully retrieved. If no matching user is found, the program prints "Not found"; otherwise, if an error occurred, it reports the corresponding error message*
- Line 36–42. Prints the information contained in the retrieved `passwd` structure, including user name, encrypted password placeholder, user ID (UID), group ID (GID), comment field, home directory, and login shell*

We execute the program.

```
[linux@users] sudo useradd -m newuser
[linux@users] sudo passwd newuser

[linux@users] New password:
Retype new password:
passwd: password updated successfully

[linux@users] grep newuser /etc/passwd
newuser:x:1002:1002::/home/newuser:/bin/sh

[linux@users] ./getrecord newuser
User: newuser
Password: x
UID: 1002
GID: 1002
Comment:
Home: /home/newuser
Shell: /bin/sh

[linux@users] ./getrecord root
User: root
Password: x
```

```
UID: 0
GID: 0
Comment: root
Home: /root
Shell: /usr/bin/zsh
```

The execution shows that after creating the user `newuser`, its record is correctly added to the user database (`/etc/passwd`). The `grep` command confirms the presence of the entry, including the UID, GID, home directory, and login shell.

Running `getrecord` for both `newuser` and `root` demonstrates that `getpwnam_r` successfully retrieves user information from the user database and stores it in the `passwd` structure. The output fields printed by the program correspond to the values stored in `/etc/passwd`.

The password field contains the placeholder `x`, indicating that the encrypted password is stored separately in the shadow password database (`/etc/shadow`). The execution also highlights differences between regular users and privileged accounts, such as different home directories, shells, and user IDs (UID `0` for `root`).

III.4.3 Iterating the user database : getpwent and setpwent

We can also work with `/etc/passwd` by iterating over the file using the following functions⁴⁸

```
#include <sys/types.h>
#include <pwd.h>

struct passwd *getpwent(void);
void setpwent(void);
void endpwent(void);

Returns: a pointer to a passwd structure, NULL if there is no matching entry,
sets errno on error
```

The `getpwent` function returns a pointer to a structure containing the parsed fields of an entry from the user database (`/etc/passwd`). The first time `getpwent` is called, it returns the first entry in the database; subsequent calls return the following entries sequentially.

The `setpwent` function resets the position in the password database to the beginning, allowing iteration to start again from the first entry.

The `endpwent` function closes the password database after processing is complete and releases any associated resources.

The `iteratepwd.c` program shows an example of how to use `setpwent`, `getpwent`, and `endpwent` to iterate through the user database.

```
1 #include <stdio.h>
2 #include <pwd.h>
3 #include <stdlib.h>

4 int main() {
5     struct passwd *pw;
6     printf("=== Reading /etc/passwd entries ===\n");
7     setpwent();
8     while ((pw = getpwent()) != NULL) {
9         printf("Username: %s\n", pw->pw_name);
10        printf("UID: %d\n", pw->pw_uid);
11        printf("GID: %d\n", pw->pw_gid);
12        printf("Home Directory: %s\n", pw->pw_dir);
13        printf("Shell: %s\n", pw->pw_shell);
14        printf("-----\n");
15    }
16    endpwent();
17    printf("=== Done ===\n");
18    return 0;
19 }
```

iteratepwd.c

48 See the `getpwent(3)` manual page

Line 7. Calls `setpwent` to reset the reading position to the beginning of the password database

Line 8. Uses `getpwent` inside a loop to sequentially retrieve each entry from the database until no more entries are available

Line 9–14. Prints selected information from each `passwd` structure, including user name, user ID (UID), group ID (GID), home directory, and login shell

Line 16. Calls `endpwent` to close the password database stream and release any associated resources

We execute the program.

```
[linux@users] ./iteratepwd
=== Reading /etc/passwd entries ===
Username: root
UID: 0
GID: 0
Home Directory: /root
Shell: /usr/bin/zsh
-----
Username: daemon
UID: 1
GID: 1
Home Directory: /usr/sbin
Shell: /usr/sbin/nologin
-----
...output...
```

The program sequentially reads all entries from the user database (`/etc/passwd`) using `getpwent`. For each entry, it prints selected information contained in the corresponding `passwd` structure, such as the user name, UID, GID, home directory, and login shell.

The output shows both regular users and system accounts (for example `root` and `daemon`), demonstrating how the password database stores information for every account configured on the system. The iteration continues until all entries in the database have been processed.

III.4.4 Working with records: `fgetpwent` and `putpwent`

The `fgetpwent`⁴⁹ function is used to retrieve records from a file formatted like `/etc/passwd`, it is defined as follows.

```
#include <stdio.h>
#include <sys/types.h>
#include <pwd.h>

struct passwd *fgetpwent(FILE *stream);

Returns: a pointer to a passwd structure, NULL if no matching entries, sets
        errno on error
```

The function accepts a pointer to a `FILE` object as argument and returns a `passwd` structure.

The `fiterate.c` program shows an example of how to use `fgetpwent` to iterate through a password-format file provided as input.

```
1 #include <stdio.h>
2 #include <pwd.h>
3 #include <stdlib.h>

4 int main(int argc, char *argv[]) {
5     if (argc != 2) {
6         printf("\nUsage: %s <file_pwd>", argv[0]);
7         exit(EXIT_FAILURE);
8     }

9     struct passwd *pw;
10    FILE *passwd_file;

11    passwd_file = fopen(argv[1], "r");
12    if (passwd_file == NULL) {
13        perror("fopen");
14        return EXIT_FAILURE;
15    }
16    printf("=== Reading %s using fgetpwent() ===\n", argv[1]);

17    while ((pw = fgetpwent(passwd_file)) != NULL) {
18        printf("Username: %s\n", pw->pw_name);
19        printf("UID: %d\n", pw->pw_uid);
20        printf("GID: %d\n", pw->pw_gid);
21        printf("Home Directory: %s\n", pw->pw_dir);
22        printf("Shell: %s\n", pw->pw_shell);
23        printf("-----\n");
24    }
```

49 See the `fgetpwent(3)` manual page

```
25     fclose(passwd_file);
26     printf("=== Done ===\n");
27     return 0;
28 }
```

fiterate.c

Line 11. Opens the file passed as a command-line argument in read mode using `fopen`

Line 16. Iterates through the file using `fgetpwent`, which reads each entry formatted like `/etc/passwd` and returns it as a `passwd` structure. For each record, the program prints the user information stored in the structure

Line 25. Closes the file stream using `fclose` after all entries have been processed, releasing system resources

We execute the program.

```
[linux@users] ./fiterate /etc/passwd
=== Reading /etc/passwd using fgetpwent() ===
Username: root
UID: 0
GID: 0
Home Directory: /root
Shell: /usr/bin/zsh
-----
Username: daemon
UID: 1
GID: 1
Home Directory: /usr/sbin
Shell: /usr/sbin/nologin
-----
...output...
```

When the program is executed with `/etc/passwd`, `fgetpwent` parses each line and returns `passwd` structures. As a result, all user entries are displayed, confirming that the file format matches the expected password database syntax.

```
[linux@users] ./fiterate /etc/networks
=== Reading /etc/networks using fgetpwent() ===
=== Done ===
```

When a file with a different format such as `/etc/networks` is provided, `fgetpwent` fails to interpret the file because it does not follow the `/etc/passwd` structure. No records are printed, and the program terminates after reaching the end of the file.

We execute the program with `sudo` passing the `/etc/shadow` file.

```
[linux@users] sudo ./fiterate /etc/shadow
=== Reading /etc/shadow using fgetpwent() ===
Username: root
```

```
UID: 20174
GID: 0
Home Directory: 7
Shell: ::
```

```
-----
Username: daemon
UID: 20154
GID: 0
Home Directory: 7
Shell: ::
```

```
-----
...output...
```

In this execution, `fgetpwent` is able to read the file because elevated privileges allow access to the *shadow database*. However, since `/etc/shadow` does not follow the same format as `/etc/passwd`, the fields are misinterpreted and incorrectly mapped into the `passwd` structure. As a result, the output is corrupted or inconsistent.

`fgetpwent` is strictly designed to work with files that follow the password database format. If the structure differs, the function silently fails without necessarily producing an error message.

When we create a new user in the system (using `useradd`⁵⁰ or `adduser`⁵¹), a new entry is added to the `/etc/passwd` file. This operation can also be performed programmatically using the `putpwent`⁵² function, which defined as follows.

```
#include <stdio.h>
#include <sys/types.h>
#include <pwd.h>

int putpwent(const struct passwd *restrict p, FILE *restrict stream);

Returns: 0 on success, on error -1 and errno
```

This function writes a *user entry* from the structure `p` into the specified file stream.

The program `newpentry.c` shows an example. Instead of modifying `/etc/passwd`, we will use a separate file to store the generated entries.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pwd.h>
4 #include <string.h>

5 int main(int argc, char *argv[]) {
6     if (argc != 9) {
7         fprintf(stderr, "Usage: %s <output_file> user passwd uid gid
            gecost home shell\n", argv[0]);
```

⁵⁰ See the `useradd(8)` manual page

⁵¹ See the `adduser(8)` manual page

⁵² See the `putpwent(3)` manual page

```

8         return EXIT_FAILURE;
9     }

10    FILE *outfile = fopen(argv[1], "w");
11    if (outfile == NULL) {
12        perror("fopen");
13        return EXIT_FAILURE;
14    }

15    struct passwd pw;
16    pw.pw_name = argv[2];
17    pw.pw_passwd = argv[3];
18    pw.pw_uid = (uid_t) atoi(argv[4]);
19    pw.pw_gid = (gid_t) atoi(argv[5]);
20    pw.pw_gecos = argv[6];
21    pw.pw_dir = argv[7];
22    pw.pw_shell = argv[8];

23    if (putpwent(&pw, outfile) != 0) {
24        perror("putpwent");
25        fclose(outfile);
26        return EXIT_FAILURE;
27    }
28    fclose(outfile);
29    printf("User entry written to %s\n", argv[1]);
30    return EXIT_SUCCESS;
31 }

```

newpentry.c

Line 10. Opens the output file provided as the first argument in write mode using `fopen`

Line 15–22. Declares a `passwd` structure and fills its fields using the values passed on the command line (username, password, UID, GID, GECOS/comment, home directory, and shell)

Line 23. Writes the user entry into the file stream using `putpwent`, which stores the structure in the standard `/etc/passwd`-like format

We execute the program

```

[linux@users] ./newpentry ./mypasswd admin x 1005 1005 adminuser /home/admin /bin/zsh
User entry written to ./mypasswd

[linux@users] cat ./mypasswd
admin:x:1005:1005:adminuser:/home/admin:/bin/zsh

```

The program successfully creates a new file and writes a user entry in `/etc/passwd` format using `putpwent`. The output confirms that all fields provided on the command line are correctly stored in the record, and the entry is formatted with colon-separated values.

When attempting to write directly to `/etc/passwd`, the program fails with `fopen: Permission denied` because this file is a critical system database protected by strict permissions. Only privileged users (typically `root`) are allowed to modify it.

```
[linux@users] ./newpentry /etc/passwd admin x 1005 1005 adminuser /home/admin /bin/zsh  
fopen: Permission denied
```

If we execute the program with `sudo` and `pass /etc/passwd` as the output file, the original content of the file will be overwritten and replaced by the new entry only. This happens because the file is opened using the "w" mode, which truncates the file before writing.

For this reason, when working with critical system files such as `/etc/passwd`, it is essential to use caution and keep a backup copy.

We conclude this section by noting that adding an entry to `/etc/passwd` does not automatically mean that a new user has been fully created.

To properly create a new user, a password must be set, and the user must also be assigned to a group. Therefore, the corresponding group must already exist or be created as part of the setup process.

III.5 The Group Database: /etc/group

The group database /etc/group maintains a list of all existing groups on the system. The fields that make up each record are defined in the `grp.h` header file.

III.5.1 Reading the group database: `getgrnam` and `getgrgid`

To retrieve an entry from the group file we can use the following functions⁵³.

```
#include <sys/types.h>
#include <grp.h>

struct group *getgrnam(const char *name);
struct group *getgrgid(gid_t gid);

int getgrnam_r(const char *restrict name, struct group *restrict grp,
               char buf[restrict .buflen], size_t buflen, struct group
               **restrict result);

int getgrgid_r(gid_t gid, struct group *restrict grp, char
               buf[restrict .buflen], size_t buflen, struct group
               **restrict result);

Return: a pointer to a group structure, NULL if there is no matching entry,
        errno is set
```

The `getgrnam` function retrieve the entry in the group database corresponding to the group name and returns a pointer to a structure containing the parsed fields.

The `getgrgid` function searches the group database for the entry matching the group ID `gid`.

The functions `getgrnam_r` and `getgrgid_r` are the reentrant versions of these functions.

The group structure and the `gid_t` type are defined in the `grp.h` header file as follows.

```
typedef __gid_t gid_t;

struct group {
    char *gr_name;      /* group name */
    char *gr_passwd;    /* group password */
    gid_t gr_gid;       /* group ID */
    char **gr_mem;      /* Array of pointers to names of group members */
};
```

Since entries in the group database can have different sizes, we must allocate an appropriately sized buffer `buf`. An initial size can be obtained using:

```
sysconf(_SC_GETGR_R_SIZE_MAX);
```

53 See the `getgrnam(3)` manual page

III.5.2 Working with the database: `getgrent` and `setgrent`

The `setgrent` and `endgrent` functions⁵⁴ are used to iterate through the entries in the `/etc/group` file.

```
#include <sys/types.h>
#include <grp.h>

struct group *getgrent(void);

void setgrent(void);
void endgrent(void);
```

They work in the same way as the functions used for the user database. The same applies to the `getgrent_r` and `fgetgrent_r`⁵⁵ functions as well.

```
#include <stdio.h>
#include <sys/types.h>
#include <grp.h>

struct group *fgetgrent(FILE *stream);

#include <grp.h>

int getgrent_r(struct group *restrict gbuf, char buf[restrict].buflen,
               size_t buflen, struct group **restrict gbup);
int fgetgrent_r(FILE *restrict stream, struct group *restrict gbuf, char
                buf[restrict].buflen, size_t buflen, struct group
                **restrict gbup);

Return: on success 0 and *gbup is a pointer to the struct group, on error the
error value and *gbup is NULL
```

The `putgrent`⁵⁶ function is used to add an entry in the group file and is defined as follows.

```
#define _GNU_SOURCE
#include <grp.h>

int putgrent(const struct group *restrict grp, FILE *restrict stream);

Returns: 0 on success, on error nonzero
```

⁵⁴ See the `getgrent(3)` manual page

⁵⁵ See the `fgetgrent(3)` and `getgrent_r(3)` manual pages

⁵⁶ See the `putgrent(3)` manual page

The `getgroups.c` program shows an example of how to iterate over the group database using `getgrent_r`.

```
1 #define _GNU_SOURCE
2 #include <grp.h>
3 #include <stdint.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #define BUFLen 4096

7 int main(void)
8 {
9     struct group grp;
10    struct group *grpp;
11    char buf[BUFLen];
12    int i;

13    setgrent();
14    while (1) {
15        i = getgrent_r(&grp, buf, sizeof(buf), &grpp);
16        if (i)
17            break;
18        printf("%s (%jd):", grpp->gr_name, (intmax_t) grpp->gr_gid);
19        for (size_t j = 0; ; j++) {
20            if (grpp->gr_mem[j] == NULL)
21                break;
22            printf(" %s", grpp->gr_mem[j]);
23        }
24        printf("\n");
25    }
26    endgrent();
27    exit(EXIT_SUCCESS);
28 }
```

getgroups.c

Line 13. Calls `setgrent` to reset the position to the beginning of the group database

Line 14. Starts an infinite loop used to read all entries from the database

Line 15. Retrieves the next group entry using `getgrent_r`, storing the result in `grp` and returning a pointer in `grpp`

Line 16–17. Checks the return value of `getgrent_r`. If an error occurs or there are no more entries, the loop is terminated

Line 18. Prints the group name and group ID (gid) for the current entry

Line 19. Starts iterating over the list of group members stored in `gr_mem`

Line 20–21. Checks for the end of the member list (NULL terminator) and exits the loop when no more members are present

Line 22. Prints each group member name

Line 26. Closes the group database using `endgrent` and releases associated resources

We execute the program

```
[linux@users] ./getgroups  
root (0):  
daemon (1):  
bin (2):  
sys (3):  
adm (4): users  
...output...
```

The program iterates through the group database (`/etc/group`) and prints the group name, group ID (GID), and the list of users belonging to that group.

Some groups include members, while others may have an empty member list.

III.5.3 Retrieving the group list: `getgrouplist`

The `getgrouplist`⁵⁷ function retrieves the list of groups to which a user belong, it is defined as follows.

```
#include <grp.h>

int getgrouplist(const char *user, gid_t group, gid_t *groups, int
                 *ngroups);

Returns: If the number of groups of which user is a member is less than or
equal to *ngroups then the value *ngroups is returned, otherwise -1
```

The function scans the group database and returns up to `ngroups` to which the user `user` belong in the `groups` buffer. The `group` argument is usually the group ID of the passed user. If the groups to which the user belong are more than `ngroups` the value of this last one can be used to resize the buffer `group` passed to a further call.

The `getusergroups.c` program is taken from the `getgrouplist(3)` manual page.

```
1 #include <errno.h>
2 #include <grp.h>
3 #include <pwd.h>
4 #include <stdio.h>
5 #include <stdlib.h>

6 int main(int argc, char *argv[])
7 {
8     int ngroups;
9     gid_t *groups;
10    struct group *gr;
11    struct passwd *pw;

12    if (argc != 3) {
13        fprintf(stderr, "Usage: %s <user> <ngroups>\n", argv[0]);
14        exit(EXIT_FAILURE);
15    }

16    ngroups = atoi(argv[2]);
17    groups = malloc(sizeof(*groups) * ngroups);
18    if (groups == NULL) {
19        perror("malloc");
20        exit(EXIT_FAILURE);
21    }

22    errno = 0;
23    pw = getpwnam(argv[1]);
24    if (pw == NULL) {
```

⁵⁷ See the `getgrouplist(3)` manual page

```

25         if (errno)
26             perror("getpwnam");
27         else
28             fprintf(stderr, "no such user\n");
29         exit(EXIT_FAILURE);
30     }

31     if (getgrouplist(argv[1], pw->pw_gid, groups, &ngroups) == -1) {
32         fprintf(stderr, "getgrouplist() returned -1; ngroups = %d\n",
33                 ngroups);
34         exit(EXIT_FAILURE);
35     }

36     fprintf(stderr, "ngroups = %d\n", ngroups);
37     for (int j = 0; j < ngroups; j++) {
38         printf("%d", groups[j]);
39         gr = getgrgid(groups[j]);
40         if (gr != NULL)
41             printf(" (%s)", gr->gr_name);
42     }
43     exit(EXIT_SUCCESS);
44 }

```

getusergroups.c

- Line 12–16. Reads the username and the initial number of groups (`ngroups`) from the command-line arguments and converts the value from string to integer*
- Line 17. Allocates memory for the `groups` array, which will store the group IDs (`gid_t`) returned by `getgrouplist`*
- Line 23. Retrieves the user entry from the user database (`/etc/passwd`) using `getpwnam` in order to obtain the user's primary group ID*
- Line 31. Calls `getgrouplist` to obtain all supplementary groups associated with the user. If the call fails, the required number of groups is reported*
- Line 35–42. Iterates over the resulting list of group IDs and prints each one. For each group ID, `getgrgid` is used to retrieve the corresponding group name from the group database*

We execute the program.

```

[linux@users] ./getusergroups users 5
getgrouplist() returned -1; ngroups = 18

[linux@users] ./getusergroups users 18
ngroups = 18
1000 (users)
4 (adm)
20 (dialout)
24 (cdrom)
...output...

```

In the first execution, the program fails because the initial value passed for the number of groups is too small to hold all supplementary groups associated with the user. As a result, `getgrouplist` returns `-1` and updates `ngroups` with the required size (18 in this case).

In the second execution, the corrected value (18) is used, allowing the function to successfully retrieve the complete list of groups. The program then prints all group IDs along with their corresponding group names obtained via `getgrgid`.

This demonstrates that `getgrouplist` may require a preliminary call or an initial guess for buffer size, and that the correct size must be used to successfully retrieve all group memberships.

This process could be automated replacing the following lines

```
31     if (getgrouplist(argv[1], pw->pw_gid, groups, &ngroups) == -1) {
32         fprintf(stderr, "getgrouplist() returned -1; ngroups = %d\n",
                    ngroups);
33         exit(EXIT_FAILURE);
34     }
```

with

```
if (getgrouplist(argv[1], pw->pw_gid, groups, &ngroups) == -1) {
    getgrouplist(argv[1], pw->pw_gid, groups, &ngroups);
}
```

We execute the program.

```
[linux@users] ./getusergroups users 5
ngroups = 18
1000 (users)
4 (adm)
20 (dialout)
24 (cdrom)
...output...
```

After modifying the code, the program automatically handles the case where the initial buffer size for group IDs is too small. If `getgrouplist` returns `-1`, the function is called again using the updated value of `ngroups`, allowing it to succeed in a single execution.

As a result, the program correctly retrieves and prints all groups for the user without requiring a second manual run.

III.5.4 Supplementary groups: `getgroups`, `setgroups` and `initgroups`

In the file header `linux/limits.h` is defined the `NGROUPS_MAX` constant as follows

```
#define NGROUPS_MAX    65536
```

This constant specifies the maximum number of *supplementary groups* to which a user can belong.

Supplementary groups are the groups a user can join after his primary group. The user can belong to other groups in a way he can have additional permissions without changing his primary group. The same value of this constant is also present in the file `/proc/sys/kernel/ngroups_max`.

To retrieve and set the supplementary groups we can use the following functions⁵⁸.

```
#include <unistd.h>

int getgroups(int size, gid_t list[]);

#include <grp.h>

int setgroups(size_t size, const gid_t *_Nullable list);

    Return: the number supplementary group IDs, on error -1 and errno
            on success 0, on error -1 and errno
```

The `getgroups` function returns the supplementary group IDs of the calling process in `list`. The `size` argument is the maximum number of items that can be stored in the buffer `list`. The `setgroups` function sets the supplementary group IDs for the calling process. To drop all the supplementary groups a process can call

```
setgroups(0, NULL)
```

A process can have up to `NGROUPS_MAX` supplementary group IDs in addition to the effective group ID. The set of supplementary group IDs is inherited from the parent process, and preserved across an `execve`.

To retrieve the maximum number of supplementary group IDs at run time we can use:

```
long ngroups_max;
ngroups_max = sysconf(_SC_NGROUPS_MAX);
```

The `initgroups`⁵⁹ function initializes the supplementary group access list for the calling process by adding all the groups of which the specified user is a member, based on the system group database.

```
#include <sys/types.h>
#include <grp.h>

int initgroups(const char *user, gid_t group);
```

⁵⁸ See the `getgroups(2)` manual page

⁵⁹ See the `initgroups(3)` manual page

Returns: 0 on success, on error -1 and errno

The `getsetgroups.c` program retrieves the groups to which the passed user belongs using the `getgroup` function and replace the retrieved list with the first group. This technique can be used to limit the privileges of the process excluding the groups it does not need. The function `initgroups` is used to set the groups as the process's supplementary groups based on the passed user.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <pwd.h>
6 #include <grp.h>
7 #include <errno.h>
8 #include <limits.h>

9 int main(int argc, char *argv[]) {
10     if (argc != 2) {
11         fprintf(stderr, "Usage: %s <username>\n", argv[0]);
12         exit(EXIT_FAILURE);
13     }

14     const char *username = argv[1];
15     struct passwd *pw = getpwnam(username);
16     if (!pw) {
17         perror("getpwnam");
18         exit(EXIT_FAILURE);
19     }

20     printf("Initializing group list for user: %s\n", username);
21     if (initgroups(username, pw->pw_gid) != 0) {
22         perror("initgroups");
23         exit(EXIT_FAILURE);
24     }

25     gid_t groups[NGROUPS_MAX];
26     int ngroups = getgroups(NGROUPS_MAX, groups);
27     if (ngroups == -1) {
28         perror("getgroups");
29         exit(EXIT_FAILURE);
30     }

31     printf("Supplementary groups (%d):\n", ngroups);
32     for (int i = 0; i < ngroups; i++) {
33         printf(" [%d] GID: %d - %s\n", i, groups[i],
34             getgrgid(groups[i])->gr_name);
35     }
36     if (ngroups >= 1) {
```

```

36     printf("Trying to reduce group list to first one only...\n");
37     if (setgroups(1, &groups[0]) == -1) {
38         perror("setgroups (requires root)");
39     } else {
40         printf("Successfully set reduced group list.\n");
41         gid_t new_groups[NGROUPS_MAX];
42         int new_ngroups = getgroups(NGROUPS_MAX, new_groups);
43         if (new_ngroups == -1) {
44             perror("getgroups (after setgroups)");
45         } else {
46             printf("New supplementary group list (%d):\n",
47                    new_ngroups);
48             for (int i = 0; i < new_ngroups; i++) {
49                 printf(" [%d] GID: %d - %s\n", i,
50                        new_groups[i],
51                        getgrgid(new_groups[i])->gr_name);
52             }
53         }
54     }
55     exit(EXIT_SUCCESS);
56 }

```

getsetgroups.c

Line 15. Retrieves the user entry from the user database using `getpwnam`, in order to obtain the user's primary group ID (`gid`)

Line 21. Initializes the supplementary group access list for the specified user using `initgroups`, which loads all groups the user belongs to

Line 26. Retrieves the current supplementary group list of the process using `getgroups`

Line 32–34. Iterates over the group list and prints each group ID along with its corresponding group name obtained via `getgrgid`

Line 37. Replaces the current supplementary group list with a reduced one containing only the first group using `setgroups` (this operation typically requires root privileges)

Line 42. Retrieves the updated group list after modification

Line 47–49. Prints the new (reduced) list of supplementary groups to confirm the effect of `setgroups`

We execute the program using `sudo`

```

[linux@users] sudo ./getsetgroups users
Initializing group list for user: users
Supplementary groups (18):
[0] GID: 4 - adm
[1] GID: 20 - dialout
[2] GID: 24 - cdrom
[3] GID: 25 - floppy
[4] GID: 27 - sudo
[5] GID: 29 - audio
[6] GID: 30 - dip

```

```
[7] GID: 44 - video
[8] GID: 46 - plugdev
[9] GID: 100 - users
[10] GID: 101 - netdev
[11] GID: 118 - bluetooth
[12] GID: 120 - vboxsf
[13] GID: 124 - lpadmin
[14] GID: 133 - scanner
[15] GID: 138 - kaboxer
[16] GID: 139 - wireshark
[17] GID: 1000 - users
```

Trying to reduce group list to the first one only...

Successfully set reduced group list.

New supplementary group list (1):

```
[0] GID: 4 - adm
```

When the program is executed with `sudo`, it successfully initializes the supplementary group list for the user using `initgroups` and retrieves all groups associated with that user. The full list of groups is printed correctly, showing all memberships defined in the system group database.

The program then attempts to reduce the supplementary group list using `setgroups`, which succeeds only because it is executed with root privileges. After the modification, the process retains only the first group (`adm`), and the updated group list confirms the change.

If the program is executed without root privileges, the call to `setgroups` fails with the error `EPERM` (Operation not permitted), since modifying the process group list requires elevated permissions.

The instruction

```
if (initgroups(username, pw->pw_gid) != 0) {
```

initializes the process supplementary groups list based on the passed user. After this line, the supplementary groups of the process will be those of the passed user. If we comment this instruction we will get only one group in the list, the group `root`, because we have executed the program as `root` (with `sudo`).

Setting the supplementary group list to a single group, as shown in this example, can be useful for privilege restriction and process isolation. By reducing the number of groups associated with a process, it is possible to limit access to system resources and files that are controlled through group permissions. This technique is often used in security-sensitive contexts where a program needs to drop unnecessary privileges after initialization.

III.5.5 Real and effective group ID: `getgid`, `getegid`, `setgid`, `setegid`

To retrieve the group ID and the effective group ID of a process we can use the following functions⁶⁰

```
#include <unistd.h>

gid_t getgid(void);
gid_t getegid(void);
```

Return: the real group ID, the effective group ID

These functions return respectively the real group ID and the effective group ID of the calling process.

The `getgid.c` program shows an example

```
1 #include <stdio.h>
1 #include <stdlib.h>
1 #include <grp.h>
1 #include <unistd.h>

1 int main() {
1     gid_t rgid, egid;

1     printf("\nReal group ID: %d", getgid());
1     printf("\nEffective group ID: %d", getegid());
1     exit(EXIT_SUCCESS);
1 }
```

getgid.c

We execute the program

```
[linux@users] ./getgid
Real group ID: 1000
Effective group ID: 1000

[linux@users] sudo ./getgid
Real group ID: 0
Effective group ID: 0
```

As we can see from the output, when we execute the program with `sudo` we get the value 0 for both GID and EGID. To change the effective group ID of a process means to modify the privileges under which that process operates.

To set a new GID we can use the following function⁶¹.

⁶⁰ See the `getgid(2)` manual page

⁶¹ See the `setgid(2)` manual page

```
#include <unistd.h>

int setgid(gid_t gid);
```

Returns: 0 on success, on error -1 and errno

The `setgid` function sets the effective group ID of the calling process. If the calling process is privileged (has the `CAP_SETGID` capability in its *user namespace*), the real GID and *saved set-group-ID* are also set.

To understand the difference between the real, effective and saved GID we can look at the following table.

Real GID	Who the user is
Effective GID	What permission the user has
Saved GID	To temporarily drop and regain privileges

Tab 12. Real, effective and saved GID

To change only the effective group ID we can use the `setegid`⁶² function.

```
#include <unistd.h>

int setegid(gid_t egid);
```

Returns: 0 on success, on error -1 and errno

The `setegid` function sets the effective group ID of the calling process. Unprivileged processes may only set the effective group ID to the real group ID, the effective group ID or the saved set-group-ID.

The `setregid`⁶³ function is used to change the real and effective group ID of the calling process and is defined as follows.

```
#include <unistd.h>

int setregid(gid_t rgid, gid_t egid);
```

Returns: 0 on success, on error -1 and errno

`setregid` sets real and effective group IDs of the calling process. If we pass a value of `-1` for either the real or effective group ID the group ID is leaved unchanged. Unprivileged processes may only set the effective group ID to the real group ID, the effective group ID, or the saved set-group-ID. Unprivileged users may only set the real group ID to the real group ID or the effective group ID. If the real group ID is set (i.e., `rgid` is not `-1`) or the effective group ID is set to a value not

⁶² See the `seteuid(2)` manual page

⁶³ See the `setreuid(2)` manual page

equal to the previous real group ID, the saved set-group-ID will be set to the new effective group ID.

The `setresgid`⁶⁴ function sets the real, effective and *saved set-group-ID*.

```
#define _GNU_SOURCE
#include <unistd.h>

int setresgid(gid_t rgid, gid_t egid, gid_t sgid);

Returns: 0 on success, on error -1 and errno
```

The following table shows the difference between these functions.

<code>setgid</code>	Effective GID, if process is privileged also real and saved set-group-ID
<code>setegid</code>	Only effective GID
<code>setregid</code>	Real and effective GID
<code>setresgid</code>	Real, effective, saved set-group-ID

Tab 13. `setgid`, `setegid`, `setregid`, `setresgid`

The `setegid.c` program demonstrates how to change the effective group ID (EGID) of a process at runtime by iterating through the supplementary groups of a user.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <grp.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 #include <linux/limits.h>
7 #include <pwd.h>

8 int main(int argc, char *argv[]) {

9     if (argc != 2) {
10         printf("\nUsage: %s <username>", argv[0]);
11         exit(EXIT_FAILURE);
12     }

13     char *username = argv[1];
14     gid_t new_gid;
15     struct group *new_group;
16     gid_t orig_egid = getegid();
17     gid_t groups[NGROUPS_MAX];
18     int i;

19     struct passwd *pw = getpwnam(username);
```

64 See the `setresuid(2)` manual page

```

20  if (initgroups(username, pw->pw_gid) != 0) {
21      perror("initgroups");
22      exit(EXIT_FAILURE);
23  }
24  int ngroups = getgroups(NGROUPS_MAX, groups);

25  printf("Original group: %4u \t %12s \t\t", orig_egid,
         getgrgid(orig_egid)->gr_name);

26  for (i=0; i < ngroups; i++) {
27      new_gid = groups[i];
28      new_group = getgrgid(new_gid);
29      if (setegid(new_gid) != 0)
30          printf("\nCannot switch to group %u", new_gid);
31      else
32          printf("\nSwitched to group: %4u \t %12s \t\tOK",
                 new_gid, new_group->gr_name);
33  }

34  printf("\n\nRestoring EGID to original (%u)", orig_egid);
35  if (setegid(orig_egid) != 0) {
36      perror("setegid (restore)");
37      return EXIT_FAILURE;
38  } else {
39      printf("\nExit...");
40  }
41  exit(EXIT_SUCCESS);
42 }

```

setegid.c

Line 16. Retrieves the current effective group ID (EGID) of the process using `getegid`

Line 19. Retrieves the user entry from the user database (`/etc/passwd`) for the specified username

Line 20. Initializes the supplementary group list for the user using `initgroups`

Line 24. Retrieves the list of supplementary groups associated with the user using `getgroups`

Line 25. Prints the original EGID along with its corresponding group name retrieved from the group database

Line 26. Iterates over all supplementary groups of the user

Line 28. Retrieves the group record corresponding to the current group ID using `getgrgid`

Line 29. Changes the effective group ID of the process to the selected group using `setegid`

Line 35. Restores the original effective group ID after testing all group switches

We execute the program passing different users.

```

[linux@users] sudo ./getegid newuser
Original group:    0          root
Switched to group: 1002       newuser      OK

Restoring EGID to original (0)

```

Exit...

[linux@users] sudo ./getegid root

Original group: 0 root
Switched to group: 0 root OK

Restoring EGID to original (0)

Exit...

[linux@users] sudo ./getegid users

Original group: 0 root
Switched to group: 4 adm OK
Switched to group: 20 dialout OK
Switched to group: 24 cdrom OK
Switched to group: 25 floppy OK
Switched to group: 27 sudo OK
Switched to group: 29 audio OK
Switched to group: 30 dip OK

...output,,

When the program is executed with `sudo`, it starts with root privileges and prints the original effective group ID (EGID). It then iterates over the supplementary groups of the specified user and attempts to switch the process EGID using `setegid`.

For `newuser`, the program switches successfully to the user's primary group. For `root`, the EGID remains unchanged since root belongs to its own group. For `users`, the program successfully cycles through multiple supplementary groups (e.g., `adm`, `dialout`, `cdrom`, etc.), demonstrating how the effective group ID can be changed at runtime based on group membership.

After completing the iteration, the program restores the original EGID, ensuring that the process returns to its initial privilege level.

III.5.6 Saved Set-User-Id and Set-Group-Id

These two bits are set on the executable files to allow program to run with the owner privileges. A program with the *set-user-ID* enabled that belongs to the user `user1` will run with the privileges of the user `user1` even when it will be executed by the user `user2`. This means that the EUID (set-user-ID) or EGID (set-group-ID) will be the same of the user that owns the file or the group of the file.

The *saved set-user-ID* and *set-group-ID* are copied from the value of the effective UID/GID. So if a process has the real, effective and saved set-user-ID, all with the value of 1000 and it execute a program that is owned by the root (UID 0) and that has the *set-user-ID* enabled, the executed program will have the following values during its execution

Real UID	1000
Effective UID	0
Saved set-user-ID	0

The effective UID will be the same as the owner of the executable (`root`). A running process can switch its effective UID/GID to the values of the real UID/GID and *saved set-user/group-id* in a way it can operate with different privileges and be in a state of potentially privileged and privileged. If a program does not need to be privileged it can operate with the real user/group ID and gain the privileged state only when it needs.

The *saved set-user-id* and *set-group-id* are also called *saved user ID* and *saved group ID*.

To retrieve the *saved set-group-id*, the real and effective GID, we can use the following function⁶⁵.

```
#define _GNU_SOURCE
#include <unistd.h>

int getresgid(gid_t *rgid, gid_t *egid, gid_t *sgid);

Returns: 0 on success, on error -1 and errno
```

The `setgroupprog.c` program retrieves and print its real and effective GID.

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>

4 int main() {

5     printf("\nReal GID: %u", getgid());
6     printf("\nEffective GID: %u", getegid());
7     exit(EXIT_SUCCESS);
8 }
```

setgroupprog.c

We execute the program

⁶⁵ See the `getresuid(2)` manual page

```
[linux@users] ./setgrouprog  
Real GID: 1000  
Effective GID: 1000
```

We change the owner and set the set-user-ID and the set-group-ID.

```
[linux@users] sudo chown newuser:newuser setgrouprog  
[linux@users] sudo chmod +s setgrouprog  
[linux@users] ls -l setgrouprog  
-rwsrwsr-x 1 newuser newuser 16112 Jun 25 01:31 setgrouprog
```

We execute the program

```
[linux@users] ./setgrouprog  
Real GID: 1000  
Effective GID: 1002
```

The real GID is that one of the user `users` (who we are) and the effective GID is that one of the user `newuser` (the owner of the executable).

We can modify the program as follows

```
gid_t saved_gid = getegid();           // Store the current EGID  
  
printf("\nReal GID: %u", getgid());  
printf("\nEffective GID: %u", getegid());  
  
setegid(getgid());                     // Set the EGID to the real GID value  
  
printf("\nReal GID: %u", getgid());  
printf("\nEffective GID: %u", getegid());  
  
setegid(saved_gid);                   // Restore the saved EGID  
  
printf("\nReal GID: %u", getgid());  
printf("\nEffective GID: %u", getegid());
```

We execute the program

```
[linux@users] ./setgrouprog  
Real GID: 1000  
Effective GID: 1002  
Real GID: 1000  
Effective GID: 1000  
Real GID: 1000  
Effective GID: 1002
```

In this way, a program can switch between gaining and dropping privileges depending on its effective user and group IDs. If the program is owned by `root` and has the set-user-ID or set-group-ID bits enabled, it will execute with `root` privileges.

The `setgroupprog2.c` program demonstrates this concept by switching between the normal user context and the `root` context, and attempting to access a file owned by `root`.

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5 #include <sys/types.h>

6 int openfile(char *filename) {
7     int fd;
8     fd = open(filename, O_RDONLY);
9     if (fd < 0) {
10         perror("open");
11         return EXIT_FAILURE;
12     } else {
13         close(fd);
14         printf("\nFile opened successfully\n");
15         return EXIT_SUCCESS;
16     }
17 }

18 int main(int argc, char *argv[]) {
19     if (argc != 2) {
20         printf("\nUsage %s <filename>", argv[0]);
21         exit(EXIT_FAILURE);
22     }

23     char *filename = argv[1];
24     gid_t saved_gid = getegid();
25     uid_t saved_uid = geteuid();

26     printf("\nReal GID: %u", getgid());
27     printf("\nEffective GID: %u", getegid());
28     openfile(filename);

29     printf("\nDropping privileges...");
30     setegid(getgid());
31     seteuid(getuid());
32     printf("\nReal GID: %u", getgid());
33     printf("\nEffective GID: %u", getegid());
34     printf("\nReal UID: %u", getuid());
35     printf("\nEffective UID: %u", geteuid());
36     openfile(filename);

37     printf("\n\nRegain privileges...");
```

```

38  setegid(saved_gid);
39  seteuid(saved_uid);
40  printf("\nReal GID: %u", getgid());
41  printf("\nEffective GID: %u", getegid());
42  printf("\nReal UID: %u", getuid());
43  printf("\nEffective UID: %u", geteuid());
44  openfile(filename);

45  exit(EXIT_SUCCESS);
46 }

```

setgroupprog2.c

- Line 6–17. The function `openfile` attempts to open the file passed as argument in read-only mode. If successful, it closes the file descriptor and prints a success message; otherwise, it reports an error*
- Line 24–25. The program stores the current effective group ID (EGID) and effective user ID (EUID) so they can be restored later*
- Line 26–28. Prints the real and effective group IDs and attempts to open the file using the current (possibly privileged) identity*
- Line 29–36. The program drops privileges by setting the effective GID and UID to the real ones. It then prints the updated identity values and attempts again to open the file, which may now fail if access is restricted*
- Line 37–44. The program restores the original saved privileges (typically root if running as SUID/SGID), prints the restored identity values, and tries again to open the file, which should succeed if privileges allow it*

We create a file owned by `root`.

```

[linux@users] echo hello > rootfile.txt
[linux@users] sudo chown root:root rootfile.txt
[linux@users] sudo chmod 640 rootfile.txt

```

We execute the program.

```

[linux@users] sudo chown root:root setgroupprog2
[linux@users] sudo chmod +s setgroupprog2
[linux@users] ls -l setgroupprog2
-rwsrwsr-x 1 root root 16552 Jun 25 03:21 setgroupprog2

[linux@users] ./setgroupprog2 ./rootfile
Real GID: 1000
Effective GID: 0
File opened successfully

Dropping privileges...
Real GID: 1000
Effective GID: 1000
Real UID: 1000
open: Permission denied

```

Effective UID: 1000

Regain privileges...

Real GID: 1000

Effective GID: 0

Real UID: 1000

Effective UID: 0

File opened successfully

The program demonstrates how file access depends on both the effective user ID (EUID) and effective group ID (EGID), especially when running a set-user-ID/set-group-ID executable.

Initially, the program runs with elevated privileges (EUID=0, EGID=0), allowing it to successfully open the `root`-owned file. After dropping privileges using `setegid(getgid())` and `seteuid(getuid())`, both identifiers are reduced to the calling user's values, and access to the `root`-owned file is denied (Permission denied). When privileges are restored using the saved IDs, access is granted again.

This behavior highlights that both EUID and EGID jointly determine access rights: reducing only the group ID is not sufficient to restrict access if the process still retains `root` EUID.

If `seteuid(getuid())` is removed, then even after dropping the group privileges, the process still keeps EUID = 0 due to the set-user-ID mechanism, and the file remains accessible.

III.5.7 Checking whether a process is in a group: `group_member`

We can know if one of the supplementary groups of a process matches a specified GID using the `group_member`⁶⁶ function, which is defined as follows.

```
#include <unistd.h>

int group_member(gid_t gid);

Returns: nonzero if a supplementary group matches gid, zero if not
```

The `grpmember.c` program demonstrates the use of `group_member` to check whether a given group ID (GID) is part of the supplementary groups of the current process.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <grp.h>

6 int main(int argc, char *argv[]) {
7     if (argc != 2) {
8         fprintf(stderr, "Usage: %s <gid>\n", argv[0]);
9         exit(EXIT_FAILURE);
10    }

11    gid_t gid = atoi(argv[1]);
12    int match;
13    match = group_member(gid);

14    if (match != 0)
15        printf("\nGID %u group found: %s", gid, getgrgid(gid)->gr_name);
16    else
17        printf("\nNo matching group found");

18    exit(EXIT_SUCCESS);
19 }
```

grpmember.c

Line 7–10. Checks that exactly one command-line argument (the GID) is provided; otherwise, it prints usage information and exits

Line 11. Converts the input argument from string to integer and stores it as a `gid_t`

Line 13. Calls `group_member(gid)`, which returns a non-zero value if the specified GID is found in the process's supplementary group list

Line 14–17. If the group is found, the program prints the group name obtained via `getgrgid`. Otherwise, it reports that no matching group was found

⁶⁶ See the `group_member(3)` manual page

The program is executed.

```
[linux@users] grep users /etc/group
adm:x:4:users
dialout:x:20:users
cdrom:x:24:users
floppy:x:25:users
sudo:x:27:users
audio:x:29:pulse,users
dip:x:30:users
video:x:44:users
plugdev:x:46:users
users:x:100:users
```

```
[linux@users] ./grpmember 27
GID 27 group found: sudo
```

```
[linux@users] ./grpmember 29
GID 29 group found: audio
```

```
[linux@users] ./grpmember 0
No matching group found
```

The output shows that GIDs such as 27 (`sudo`) and 29 (`audio`) are found because they belong to the user's supplementary group list as defined in `/etc/group`.

When a GID that is not part of the process's supplementary groups (such as 0, `root`) is tested, the function correctly reports that no matching group is found. This demonstrates how `group_member` can be used to verify group membership at runtime based on the process's current group context.

III. 6 User functions

For the group-related functions discussed in the previous section, there are equivalent functions for user management. These are summarized in the following table.

Get group/user ID	getgid	getuid
Get effective group/user ID	getegid	geteuid
Set group/user ID	setgid	setuid
Set effective user/group ID	setegid	seteuid
Set real and effective group/user ID	setregid	setreuid
Set real, effective, saved group/user ID	setresgid	setresuid

Tab 14. Get and set user/group ID functions

Each of these functions has a dedicated manual page in the section 2: `getuid(2)`, `setuid(2)`, etc.. The prototypes of these system calls are listed below.

```
#include <unistd.h>

uid_t getuid(void);
uid_t geteuid(void);
int setuid(uid_t uid);
int seteuid(uid_t euid);
int setreuid(uid_t ruid, uid_t euid);

#define _GNU_SOURCE
#include <unistd.h>

int setresuid(uid_t ruid, uid_t euid, uid_t suid);
```

The return values and the arguments are the same as the groups functions, with the difference that the user related functions work with the `uid_t` data type instead of `gid_t`.

III.6.1 Setting user ID: `getuid`, `geteuid`, `setuid` and `seteuid`

The `writeuserfile.c` program demonstrates how changing the effective user ID (EUID) and effective group ID (EGID) allows a process to access and modify a file owned by another user.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <pwd.h>
6 #include <fcntl.h>
```

```

7 #include <string.h>

8 int main (int argc, char *argv[]) {
9     if (argc != 4) {
10         printf("\nUsage: %s <filename> <username> <string>", argv[0]);
11         exit(EXIT_FAILURE);
12     }

13     uid_t newuid;
14     int fd;
15     size_t bw;
16     char *buf = argv[3];
17     struct passwd *pw;

18     pw = getpwnam(argv[2]);
19     if (!pw) {
20         perror("getpwnam");
21         exit(EXIT_FAILURE);
22     }

23     printf("\nOriginal EUID: %u EGID: %u", geteuid(), getegid());
24     if (setegid(pw->pw_gid) != 0) {
25         perror("setegid");
26         exit(EXIT_FAILURE);
27     }
28     if (seteuid(pw->pw_uid) != 0) {
29         perror("seteuid");
30         exit(EXIT_FAILURE);
31     }
32     printf("\nNew EUID: %u EGID: %u", geteuid(), getegid());

33     fd = open(argv[1], O_RDWR | O_APPEND);
34     if (fd == -1) {
35         perror("open");
36         exit(EXIT_FAILURE);
37     } else {
38         printf("\nFile opened: %d", fd);
39     }
40     bw = write(fd, buf, strlen(buf));
41     if (bw == -1) {
42         perror("write");
43         exit(EXIT_FAILURE);
44     }
45     printf("\n%d bytes written into the file", bw);
46     exit(EXIT_SUCCESS);
47 }

```

Line 18. Retrieves the user record from the user database (/etc/passwd) using `getpwnam` to obtain the target user's UID and GID

Line 24. Sets the effective group ID (EGID) of the process to the group ID of the specified user using `setegid`

Line 28. Sets the effective user ID (EUID) of the process to the user ID of the specified user using `seteuid`

We create the file `newuserfile.txt` in the `/tmp` directory.

```
[linux@users] echo hello > /tmp/newuserfile.txt
[linux@users] sudo chown newuser:newuser /tmp/newuserfile.txt
[linux@users] sudo chmod 640 /tmp/newuserfile.txt
[linux@users] ls -l /tmp/newuserfile.txt
-rw-r----- 1 newuser newuser 6 Jun 25 17:38 /tmp/newuserfile.txt

[linux@users] cat /tmp/newuserfile.txt
cat: /tmp/newuserfile.txt: Permission denied

[linux@users] sudo cat /tmp/newuserfile.txt
hello
```

The `newuserfile.txt` file is owned by the user `newuser` and the group has the read permission set. If we try to read the file we obtain a “Permission denied” error because the user `users` is not in the group `newuser`. If we try to read the file with `sudo`, the execution succeeds.

We write some data into the file.

```
[linux@users] echo world >> /tmp/newuserfile.txt
zsh: permission denied: /tmp/newuserfile.txt

[linux@users] sudo echo world >> /tmp/newuserfile.txt
zsh: permission denied: /tmp/newuserfile.txt
```

We obtain a “permission denied” error even when using `sudo` because we cannot modify a file owned by another user if write permissions are not granted to others (or if appropriate privileges are not applied to the operation).

Hence, if we have `sudo` access and want to modify this file, we must obtain the appropriate permissions of the file owner (or operate with sufficient elevated privileges that effectively bypass ownership restrictions).

We execute the program using `sudo`

```
[linux@users] sudo ./writeuserfile /tmp/newuserfile.txt users one
Original EUID: 1000 EGID: 1000
open: Permission denied
New EUID: 1000 EGID: 1000

[linux@users] sudo ./writeuserfile /tmp/newuserfile.txt newuser two
Original EUID: 0 EGID: 0
New EUID: 1005 EGID: 1005
```

File opened: 3

3 bytes written into the file

[[linux@users](#)] sudo cat /tmp/newuserfile.txt

hello

two

[[linux@users](#)] sudo ./writeuserfile /tmp/newuserfile.txt **root** three

Original EUID: 0 EGID: 0

New EUID: 0 EGID: 0

File opened: 3

5 bytes written into the file

As shown in the execution results, the program behaves differently depending on the target user used to set the effective UID and GID.

When attempting to use a regular user (e.g., `users`), the program fails to open the file due to insufficient permissions, even though it is executed with `sudo`, because the privilege change is not sufficient to match the file's ownership and access rights.

When switching to `newuser`, the program successfully updates the effective UID and GID, allowing the file to be opened and modified. The same occurs when switching to `root`, which has full access and can freely write to the file.

As a result, only users with appropriate privileges (or `root`) can successfully modify the file. The program itself must be executed with `sudo`, otherwise it cannot change the process's effective UID and GID and therefore cannot perform privileged operations.

III.6.2 getresgid and getresuid

The `switchuid.c` program is a modified version of `setgroupprog2.c` that uses `getresgid` and `getresuid` to retrieve real, effective, and saved user and group IDs, allowing more precise control over privilege switching.

```
1 #define _GNU_SOURCE
2 #include <stdlib.h>
3 #include <stdio.h>
4 #include <unistd.h>
5 #include <fcntl.h>
6 #include <sys/types.h>

7 int openfile(char *filename) {
8     int fd;
9     fd = open(filename, O_RDONLY);
10    if (fd < 0) {
11        perror("open");
12        return EXIT_FAILURE;
13    } else {
14        close(fd);
15        printf("\nFile opened successfully\n");
16        return EXIT_SUCCESS;
17    }
18 }

19 int main(int argc, char *argv[]) {
20     if (argc != 2) {
21         printf("\nUsage %s <filename>", argv[0]);
22         exit(EXIT_FAILURE);
23     }

24     char *filename = argv[1];
25     gid_t rgid, egid, sgid;
26     uid_t ruid, euid, suid;

27     if (getresgid(&rgid, &egid, &sgid) != 0) {
28         perror("getresgid");
29         exit(EXIT_FAILURE);
30     }
31     if (getresuid(&ruid, &euid, &suid) != 0) {
32         perror("getresuid");
33         exit(EXIT_FAILURE);
34     }

35     printf("\nReal GID: %u", rgid);
36     printf("\nEffective GID: %u", egid);
37     openfile(filename);

38     printf("\nDropping privileges...");
```

```

39  setegid(rgid);
40  seteuid(ruid);
41  printf("\nReal GID: %u", getgid());
42  printf("\nEffective GID: %u", getegid());
43  printf("\nReal UID: %u", getuid());
44  printf("\nEffective UID: %u", geteuid());
45  openfile(filename);

46  printf("\n\nRegain privileges...");
47  setegid(sgid);
48  seteuid(suid);
49  printf("\nReal GID: %u", getgid());
50  printf("\nEffective GID: %u", getegid());
51  printf("\nReal UID: %u", getuid());
52  printf("\nEffective UID: %u", geteuid());
53  openfile(filename);

54  exit(EXIT_SUCCESS);
55 }

```

switcheid.c

Line 27. Retrieves the real, effective, and saved group IDs using `getresgid`

Line 31. Retrieves the real, effective, and saved user IDs using `getresuid`

Line 35–37. Attempts to open the file using the current effective privileges

Line 38–45. Drops privileges by setting the effective UID and GID to the real ones, then attempts again to open the file with reduced permissions

Line 46–53. Restores the saved privileges (saved UID and GID) and retries opening the file with elevated permissions

We execute the program

```

[linux@users] ls -l rootfile.txt
-rw-r----- 1 root root 6 Jun 25 03:04 rootfile.txt

[linux@users] sudo chown root:root switcheid
[linux@users] sudo chmod +s switcheid

[linux@users] ./switcheid ./rootfile.txt
Real GID: 1000
Effective GID: 0
File opened successfully

Dropping privileges...
Real GID: 1000
Effective GID: 1000
Real UID: 1000
open: Permission denied
Effective UID: 1000

Regain privileges...

```

```
Real GID: 1000
Effective GID: 0
Real UID: 1000
Effective UID: 0
File opened successfully
```

The program is executed on a `root`-owned file with restricted permissions (`rw-r-----`), meaning only the owner (`root`) can access it.

Initially, the program runs with elevated privileges (`EUID=0`, `EGID=0` due to the `setuid/setgid` bits), allowing the file to be opened successfully. After dropping privileges by setting the effective UID and GID to the real ones, access is denied (Permission denied) because the process is now running as a normal user.

When privileges are restored using the saved IDs, the process regains `root`-level access (`EUID=0`, `EGID=0`), and the file is successfully opened again.

This demonstrates how `getresuid` and `getresgid()` allow a program to correctly track and restore real, effective, and saved privileges, enabling controlled privilege dropping and re-acquisition during execution.

III.7 The Password Database: `/etc/shadow`

The `/etc/shadow`⁶⁷ file contains the encrypted passwords of the users and the aging data. The functions⁶⁸ to work with the database are analogous to those related to the user database.

```
#include <shadow.h>

struct spwd *getspnam(const char *name);
struct spwd *getspent(void);

void setspent(void);
void endspent(void);

struct spwd *fgetspent(FILE *stream);
struct spwd *sgetspent(const char *s);

int putspent(const struct spwd *p, FILE *stream);

int lckpwdf(void);
int ulckpwdf(void);

/* GNU extension */
#include <shadow.h>

int getspent_r(struct spwd *spbuf, char buf[.buflen], size_t buflen,
               struct spwd **spbufp);
int getspnam_r(const char *name, struct spwd *spbuf, char buf[.buflen],
               size_t buflen, struct spwd **spbufp);

int fgetspent_r(FILE *stream, struct spwd *spbuf, char buf[.buflen],
                size_t buflen, struct spwd **spbufp);
int sgetspent_r(const char *s, struct spwd *spbuf, char buf[.buflen],
                size_t buflen, struct spwd **spbufp);

Return: a pointer to a spwd struct, NULL if no match or if error
        0 on success, on error -1 and errno
```

The `getspnam` function returns a pointer to a structure containing the broken-out fields of the record in the shadow password database that matches the username `name`.

The `getspent` function returns a pointer to the next entry in the shadow password database. The position in the input stream is initialized by `setspent`. When the reading is finished, the program may call `endspent` to deallocate the resources.

The `fgetspent` function is similar to `getspent` but uses a stream instead of the one implicitly opened by `setspent`.

The `sgetspent` function parses the string `s` into a `struct spwd`.

The `putspent` function writes the contents of the supplied `struct spwd *p` as a text line in the shadow password file format to the file `stream`. String entries with value `NULL` and numerical entries with value `-1` are written as an empty string.

⁶⁷ See the `shadow(5)` manual page

⁶⁸ See the `getspnam(3)` manual page

The `lockpwordf` function is intended to protect against multiple simultaneous accesses of the shadow password database. It tries to acquire a lock and returns 0 on success or -1 on failure (lock not obtained within 15 seconds).

The `unlockpwordf` function releases the lock again. There is no protection against direct access of the shadow password file and only programs that use `lockpwordf` will notice the lock.

The `getspnam_r` function is like `getspnam` but stores the retrieved shadow password structure in the space pointed to by `spbuf`. This shadow password structure contains pointers to strings, and these strings are stored in the buffer `buf` of size `buflen`. A pointer to the result (in case of success) or NULL (in case no entry was found or an error occurred) is stored in `*spbufp`.

The functions `getspent_r`, `fgetspent_r`, and `sgetspent_r` are similarly analogous to their nonreentrant counterparts.

The `spwd` structure (*shadow password*) is defined in the `shadow.h` header file as follows.

```
struct spwd {
    char *sp_namp;           /* Login name */
    char *sp_pwdp;           /* Encrypted password */
    long sp_lstchg;          /* Date of last change (measured in days since Epoch) */
    long sp_min;             /* Min # of days between changes */
    long sp_max;             /* Max # of days between changes */
    long sp_warn; /* # of days before password expires to warn user to change it */
    long sp_inact; /* # of days after password expires until account is disabled */
    long sp_expire; /* Date when account expires (measured in days since Epoch) */
    unsigned long sp_flag; /* Reserved */
};
```

In the `paths.h` header file, the constant `_PATH_SHADOW` is defined and it specifies the pathname of the system shadow password file.

The `getshadow.c` program retrieves the shadow password entry corresponding to the specified user from the system shadow database.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <shadow.h>

4 int main(int argc, char *argv[]) {
5     if (argc != 2) {
6         fprintf(stderr, "Usage: %s <username>\n", argv[0]);
7         exit(EXIT_FAILURE);
8     }

9     const char *username = argv[1];
10    struct spwd *shadow_entry;

11    shadow_entry = getspnam(username);
12    if (!shadow_entry) {
13        printf("\nUser not found or permission error");
14        exit(EXIT_FAILURE);
15    }
```

```
16  printf("Shadow entry for user: %s\n", username);
17  printf(" Encrypted password: %s\n", shadow_entry->sp_pwdp);
18  printf(" Last password change (days since epoch): %ld\n",
        shadow_entry->sp_lstchg);
19  printf(" Min days between changes: %ld\n", shadow_entry->sp_min);
20  printf(" Max days between changes: %ld\n", shadow_entry->sp_max);
21  printf(" Warning period (days): %ld\n", shadow_entry->sp_warn);
22  printf(" Inactive period (days): %ld\n", shadow_entry->sp_inact);
23  printf(" Account expiration (days since epoch): %ld\n",
        shadow_entry->sp_expire);
24  printf(" Reserved field: %lu\n", shadow_entry->sp_flag);
25  exit(EXIT_SUCCESS);
26 }
```

getshadow.c

Line 10. Declares a pointer to a `spwd` structure, which will hold the retrieved shadow entry

Line 11. Retrieves the shadow password record for the given username using `getspnam`

Line 12–15. Checks whether the retrieval was successful. If the user does not exist or permission is denied (since `/etc/shadow` is restricted), the program prints an error message and exits

Line 16–24. Prints the fields of the `spwd` structure, including password aging information, expiration data, and the encrypted password field

We execute the program using `sudo`

```
[linux@users] sudo ./getshadow users
Shadow entry for user: users
Encrypted password: $y$j9...
Last password change (days since epoch): 20257
Min days between changes: 0
Max days between changes: 99999
Warning period (days): 7
Inactive period (days): -1
Account expiration (days since epoch): -1
Reserved field: 18446744073709551615
```

When the program is executed with `sudo`, it successfully accesses the shadow password database and retrieves the record for the specified user. The output shows the encrypted password hash (stored in `/etc/shadow`), along with password aging and account policy information such as last password change, minimum and maximum validity periods, warning time, and expiration values. As observed, passwords in `/etc/shadow` are not stored in plain text but as cryptographic hashes. During login, the system hashes the password entered by the user and compares it with the stored hash to verify authenticity, rather than decrypting the stored value.

III.7.1 Creating an encrypted passphrase: `crypt`

As we have seen, the password database stores encrypted passwords. The encryption mechanism used by the system is provided through the `crypt`⁶⁹ function, which encapsulates the underlying hashing algorithm and is defined as follows.

```
#include <crypt.h>

char *crypt(const char *phrase, const char *setting);

char *crypt_r(const char *phrase, const char *setting, struct crypt_data
              *data);

char *crypt_rn(const char *phrase, const char *setting, struct
               crypt_data *data, int size);

char *crypt_ra(const char *phrase, const char *setting, void **data, int
               *size);

    Return: a pointer to a string which encodes the hashed passphrase and the
            settings used to encode it, on error an invalid hash or a NULL pointer
            (crypt_rn, crypt_ra) and errno
```

These functions create an *hash* from the passed passphrase `phrase`. This *hash* can then be stored in the password database `/etc/shadow`. Basically an *hash* is a string containing the algorithm (*hashing method*) used to encrypt the passed string and the resulting encrypted string. The parameter `settings` represents the *hashing method* to use and also the parameters to pass to the *hashing method*. The most important parameter is the *salt*, which represents a string used to ensure that no two stored hashes are the same even if the strings `phrase` are the same.

The `crypt_data` structure is defined as follows in the `crypt.h` header file.

```
struct crypt_data {
    char output[CRYPT_OUTPUT_SIZE];
    char setting[CRYPT_OUTPUT_SIZE];
    char input[CRYPT_MAX_PASSPHRASE_SIZE];
    char initialized;
};
```

The `output` field will contain the hashed passphrase after a call to `crypt_r`. The input string is stored in the field `input` and the field `setting` contains the passed setting. The `initialized` field must be set to zero before the first time the structure is first used in a call to `crypt_r`.

The `data` argument to `crypt_rn` should also point to a `struct crypt_data` object, and the parameter `size` is the size of that object, cast to `int`. When used with `crypt_rn`, the entire data object (except for the `input` and `setting` fields) must be zeroed before its first use

On the first call to `crypt_ra`, `data` should be the address of a `void *` variable set to `NULL`, and `size` should be the address of an `int` variable set to zero. `crypt_ra` will allocate and

⁶⁹ See the `crypt(3)` manual page

initialize a struct `crypt_data` object, using `malloc`, and write its address and size into the variables pointed to by `data` and `size`. These can be reused in subsequent calls. After the application is done hashing passphrases, it should deallocate the struct `crypt_data` object using `free`.

The `crypt` function places its result in a static storage area, which will be overwritten by subsequent calls to `crypt`, so it is not safe to call `crypt` from multiple threads simultaneously. The functions `crypt_r`, `crypt_rn`, and `crypt_ra` place their result in the output field of their `data` argument, so it is safe to call them from multiple threads simultaneously, as long as a separate data object is used for each thread.

On error the functions `crypt_r`, `crypt_rn`, and `crypt_ra` write an invalid hash to the output field of their `data` argument, and `crypt` writes an invalid hash to its static storage area. This string will be shorter than 13 characters, will begin with a `*`, and will not compare equal to setting.

The functions `crypt_rn` and `crypt_ra` return a NULL pointer. `crypt_r` and `crypt` may also return a NULL pointer, or they may return a pointer to the invalid hash, depending on how `libcrypt`⁷⁰ was configured.

The `cryptpass.c` program generates a *hashed password* (passphrase) using the provided *salt* and plaintext password.

```
1 #define _XOPEN_SOURCE
2 #include <crypt.h>
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>

6 int main(int argc, char *argv[]) {
7     if (argc != 3) {
8         printf("\nUsage %s <salt> <password>", argv[0]);
9         exit(EXIT_FAILURE);
10    }

11    const char *password = argv[2];
12    const char *salt = argv[1];
13    char *hashed = crypt(password, salt);
14    if (hashed == NULL) {
15        perror("crypt");
16        return 1;
17    }
18    printf("Password: %s\n", password);
19    printf("Salt: %s\n", salt);
20    printf("Hashed: %s\n", hashed);
21    return 0;
22 }
```

cryptpass.c

70 The library containing these functions, on Kali Linux the packages `libcrypt1` and `libcrypt-dev`

Line 7–10. Ensures that exactly two arguments are provided: the *salt* and the *password*. If not, it prints usage information and exits

Line 11–12. Stores the *password* and *salt* values passed from the command line

Line 13. Calls the *crypt* function to compute the hashed password using the given *salt*

Line 14–17. Checks whether *crypt* succeeded. If it returns *NULL*, an error is reported and the program exits

Line 18–20. Prints the original password, the salt used, and the resulting hashed password

We compile the program linking to the `libcrypt` library.

```
[linux@users] gcc -o cryptpass cryptpass.c -lcrypt
```

We execute the program.

```
[linux@users] ./cryptpass '$6$rounds=5000$randsalt' hellocrypt
Password: hellocrypt
Salt: $6$rounds=5000$randsalt
Hashed:
$6$rounds=5000$randsalt$FRyfopyD51r/VM658UTz0j/WZzwMhBXewslQsAzUFP25PHPxyWz
EBNOLbx7Z.CgptJxtcN69PjcahUpEwq/JJ0
```

It is better to put the *salt* between the `''` as it contains special characters. As we can see from the output, the *hashed passphrase* contains the *salt* and the *encrypted passphrase*.

The *salt* is necessary so that the same passphrase may produce many possible hashes. The produced *hashed passphrase* is composed by the following fields

Field	Value	Description
Prefix	<code>\$6\$</code>	The <i>hashing method</i> used to encrypt the passphrase (<code>sha512crypt</code>)
Options	<code>rounds=5000</code>	The number of how many iterations of the <i>hashing algorithm</i> are applied to the <i>password</i> + <i>salt</i> combination
Salt	<code>randsalt</code>	The string used as <i>salt</i>
Hash	<code>FRyfopyD51r/ VM658UTz0j/ WZzwMhBXewslQsAzUFP25P HPxyWzEBNOLbx7Z.CgptJx tcN69PjcahUpEwq/JJ0</code>	The encrypted password

Tab 15. The *hashed passphrase* fields

The following table lists the hashing methods supported by `crypt`, which can be used to generate hashed passphrases. The available algorithms are documented in the `crypt(5)` manual page.

Method	Prefix
yescrypt	\$y\$
gost-yescrypt	\$gy\$
scrypt	\$7\$
bcrypt	\$2b\$
sha512crypt	\$6\$
sha256crypt	\$5\$
shalcrypt	\$sha1
SunMD5	\$md5
md5crypt	\$1\$
bsdicrypt	—
descrypt	"" (empty string)
bigcrypt	"" (empty string)
NT	\$3\$

Tab 16. Hashing methods supported by `crypt`

The *hashed passphrase* is composed by: *prefix*, *options*, *salt*, *hash*. The *prefix* and *options* field could be empty based on the *hashing method* used. The character `$` separates the fields.

On Linux systems we can generate a new *password hash* on the command line with the `mkpasswd`⁷¹ and `openssl`⁷² tools.

```
[linux@users] mkpasswd -m sha-512
Password:
$6$k7na4AiyfEIItDeB$NzJfUCWTAX8iF0I1ToFD.gcARO1/
yOWIFkKQJ4RYEQ9VGSDYG.Ns7vxxI1wFkRT8bXc9bvCN6p64oQD2kzs.T/

[linux@users] openssl passwd -6
Password:
Verifying - Password:
$6$FmMFyYTOt5/mZ2Uj$P7oQ3FYcZ/aH.5214T6WXRmUH/
Dfl8buyN50v9BxUVHdYlEJmHJlRL0Ewexpi1GwW/qpnDnL6Bd8hAxIAEK/
```

The option `-m` of `mkpasswd` defines the *hashing method* to use, as we can see from the output the prefix is `$6$` (`sha512crypt`).

The option `passwd` of `openssl` tells `openssl` to generate a new *password hash*, the option `-6` defines the *hashing method*.

The `checkpwd.c` program verifies user input by comparing it with a previously stored hash.

To prepare a valid test case, first generate a password hash using `mkpasswd`, including a salt.

⁷¹ `mkpasswd` is a frontend for `crypt`, see the `mkpasswd(1)` manual page

⁷² See the `openssl(1)` manual page

```
[linux@users] mkpasswd -m sha-512 --salt=newsaltstr
```

Password:

```
$6$newsaltstr$I05pcgWEWXk1jrSaBBs6hZLISQTtealfWT6VdeD94GYykJ010I8cKg8C08FW099g/YC/GRBL.t./Y.tus/EM0
```

I have chosen the password `mars123` . Now we can use the generated hash in our program.

```
1 #define _XOPEN_SOURCE
2 #include <crypt.h>
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdlib.h>

6 int main() {
7     const char *stored_hash =
8         "$6$newsaltstr$I05pcgWEWXk1jrSaBBs6hZLISQTtealfWT6VdeD94GYykJ010I8cKg8C08FW099g/YC/GRBL.t./Y.tus/EM0";

9     char input[100];
10    printf("Enter password: ");
11    if (fgets(input, sizeof(input), stdin) == NULL) {
12        perror("fgets");
13        return 1;
14    }

15    input[strcspn(input, "\n")] = '\0';

16    char salt[100];
17    const char *p = stored_hash;
18    int dollar_count = 0;
19    size_t i = 0;

20    while (*p && dollar_count < 3 && i < sizeof(salt) - 1) {
21        salt[i++] = *p;
22        if (*p == '$') {
23            dollar_count++;
24        }
25        p++;
26    }
27    salt[i] = '\0';

28    char *hashed = crypt(input, salt);
29    if (hashed == NULL) {
30        perror("crypt");
31        return 1;
32    }

33    if (strcmp(hashed, stored_hash) == 0) {
34        printf(" Password correct!\n");
35    } else {
```

```
35         printf(" Invalid password.\n");
36     }
37     return 0;
38 }
```

checkpwd.c Check the password

Line 7. Stores the precomputed hashed password in the constant `stored_hash`

Line 9–13. Prompts the user for a password and reads it from standard input using `fgets`, storing it in `input`

Line 14. Removes the trailing newline character from the input string

Line 15–26. Extracts the salt from the stored hash by scanning the string up to the third `$` delimiter, reconstructing the correct salt format required by `crypt`

Line 27. Calls `crypt` with the user input and extracted salt, producing a new hash stored in `hashed`

Line 32–36. Compares the computed hash with the stored hash using `strcmp`. If they match, the password is correct; otherwise, it is rejected

The program is executed.

```
[linux@users] ./checkpwd
Enter password: mars123
Password correct!
```

```
[linux@users] ./checkpwd
Enter password: mars12
Invalid password.
```

The program correctly validates the user input by hashing the entered password with the same salt used in the stored hash and comparing the result.

When the correct password (`mars123`) is entered, the computed hash matches the stored hash and the program confirms successful authentication. When an incorrect password (`mars12`) is entered, the hashes differ and the program correctly rejects the input.

III.7.2 Generating a salt: `crypt_gensalt`

Before hashing a password with `crypt`, it is necessary to generate a salt. A salt is a random value added to the password before hashing in order to produce unique hashes even when two users choose the same password. This mechanism helps protect against precomputed attacks such as rainbow tables.

The `crypt_gensalt`⁷³ function can be used to automatically generate salts compatible with the hashing methods supported by `crypt`.

```
#include <crypt.h>

char *crypt_gensalt(const char *prefix, unsigned long count, const char
                    *rbytes, int nrbytes);

char *crypt_gensalt_rn(const char *prefix, unsigned long count, const
                       char *rbytes, int nrbytes, char *output, int
                       output_size);

char *crypt_gensalt_ra(const char *prefix, unsigned long count, const
                       char *rbytes, int nrbytes);

Return: pointer to an encoded setting string, on error a NULL pointer and errno
```

These functions generate a string that can be used as the settings argument for `crypt`, `crypt_r`, `crypt_rn`, and `crypt_ra`. The `prefix` argument specifies the hashing method, `count` controls the computational cost of the hash, and `rbytes` points to `nrbytes` cryptographically secure random bytes used to generate the salt.

The `crypt_gensalt_rn` function stores the generated result in the buffer `output`, which has `output_size` bytes available. The value of `output_size` should be greater than or equal to the constant `CRYPT_GENSALT_OUTPUT_SIZE`, defined in the `crypt.h` header file.

The `gensalt.c` program generates a salt using `crypt_gensalt` and then uses it to hash a password with `crypt`.

```
1 #include <stdio.h>
2 #include <crypt.h>
3 #include <stdlib.h>

4 int main(int argc, char *argv[]) {
5     if (argc != 2) {
6         printf("\nUsage: %s <password>", argv[0]);
7         exit(EXIT_FAILURE);
8     }

9     const char *password = argv[1];
10    char *salt = crypt_gensalt("$6$", 5000, NULL, 0);
11    if (salt == NULL) {
12        perror("crypt_gensalt");
```

73 See the `crypt_gensalt(3)` manual page

```

13         exit(EXIT_FAILURE);
14     }

15     printf("Generated salt: %s\n", salt);
16     char *hash = crypt(password, salt);
17     if (hash == NULL) {
18         perror("crypt");
19         exit(EXIT_FAILURE);
20     }
21     printf("Password: %s\n", password);
22     printf("Hash: %s\n", hash);
23     exit(EXIT_SUCCESS);
24 }

```

gensalt.c

- Line 10. Generates a salt using the SHA-512 hashing method (\$6\$) with a processing cost of 5000 rounds*
- Line 11–14. Checks whether the salt generation succeeded. If `crypt_gensalt` returns `NULL`, the program reports the error and exits*
- Line 15. Prints the generated salt*
- Line 16. Hashes the password passed on the command line using the generated salt and stores the result in `hash`*
- Line 17–20. Checks whether the hashing operation succeeded. If `crypt` fails, the program prints an error message and terminates*
- Line 21–22. Prints the original password and the resulting hash*

We compile and execute the program.

```

[linux@users] gcc -o gensalt gensalt.c -lcrypt

[linux@users] ./gensalt mypwd123
Generated salt: $6$2wiE2QdAYJA1y1T
Password: mypwd123
Hash:
$6$2wiE2QdAYJA1y1T$P0COfmjnMMjjWliY7653huNPHsmbR9vUzP2Dyiw7/ROtnLJ6oM0p
2D8GvOnOnrscqzcMURPV1uZMTFein9xIh0

```

The `gensalt.c` program can be modified to generate the salt using cryptographically secure random bytes obtained from the system.

```

1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <crypt.h>
4 #include <stdlib.h>
5 #include <sys/random.h>
6 #define SALT_BYTES 16

7 int main(int argc, char *argv[]) {
8     if (argc != 2) {

```

```

9         printf("\nUsage: %s <password>", argv[0]);
10        exit(EXIT_FAILURE);
11    }

12    const char *password = argv[1];
13    unsigned char random_bytes[SALT_BYTES];
14    if (getrandom(random_bytes, SALT_BYTES, GRND_NONBLOCK) < 0) {
15        perror("urandom");
16        return 1;
17    }

18    char *salt = crypt_gensalt("$6$", 10000, (const char
                                *)random_bytes, SALT_BYTES);
19    if (salt == NULL) {
20        perror("crypt_gensalt");
21        exit(EXIT_FAILURE);
22    }

23    printf("Generated salt: %s\n", salt);
24    char *hash = crypt(password, salt);
25    if (hash == NULL) {
26        perror("crypt");
27        exit(EXIT_FAILURE);
28    }

29    printf("Password: %s\n", password);
30    printf("Hash: %s\n", hash);
31    exit(EXIT_SUCCESS);
32 }

```

gensalt.c

Line 13. Declares a buffer used to store the random bytes that will be used for salt generation

Line 14. Calls `getrandom` to retrieve 16 cryptographically secure random bytes from the kernel random number generator

Line 18. Generates a SHA-512 salt using `crypt_gensalt`, passing the random bytes as entropy for the salt generation process

Line 19–22. Checks whether the salt generation succeeded. If `crypt_gensalt` fails, the program reports the error and exits

Line 23. Prints the generated salt

Line 24. Hashes the provided password using the generated salt

Line 25–27. Checks whether the hashing operation succeeded and reports any error returned by `crypt`

Line 29–30. Prints the plaintext password and the resulting hash

`getrandom` does not directly read from `/dev/random`; it accesses the kernel random number generator through a system call interface.

We execute the program

```
[linux@users] ./gensalt mypwd123
Generated salt: $6$rounds=10000$KKGNh2f.6mp4TTc.
Password: mypwd123
Hash: $6$rounds=10000$KKGNh2f.6mp4TTc.
$TliJPu.QGV1U7QNLm3CcBWDNQAZ8gqnImVo9GBSIWZq9jBuoD8AIWOeAjWULvmd6hK
lwJDem8/auka/jqsvw5/
```

The program successfully generates a cryptographically secure salt using random bytes obtained from the system random generator and then uses it to hash the provided password with the SHA-512 algorithm (\$6\$).

The generated hash includes:

- the hashing method identifier (\$6\$ for SHA-512)
- the number of rounds (rounds=10000)
- the number of rounds (rounds=10000)
- the generated salt
- and the final hashed password

Even when the same password is used multiple times, different random salts produce different hashes, improving resistance against precomputed attacks such as rainbow tables.

III.7.3 Hiding the input with BSD `readpassphrase`

When entering a password on the terminal, it is often useful to hide the typed characters for security reasons. To achieve this, we can use the `readpassphrase`⁷⁴ function, provided by the `libbsd-dev`⁷⁵ package, which is defined as follows.

```
#include <readpassphrase.h>

char *readpassphrase(const char *prompt, char *buf, size_t bufsiz, int
                    flags);

Returns: pointer to the passphrase, on error a NULL pointer
```

To install the BSD Development library we can use `apt`

```
[linux@users] sudo apt install libbsd-dev
...output...
```

The `libbsd(7)` manual page contains a list of the utility functions from the BSD systems. The `readpassphrase` function displays a prompt and reads in a passphrase from the terminal (`/dev/tty`). The input is stored in the `buf` pointer and the function reads up `bufsiz - 1` characters. The `flags` argument can be the bitwise OR of zero or more of the following values.

<code>RPP_ECHO_OFF</code>	Turn off echo (default)
<code>RPP_ECHO_ON</code>	Leave echo on
<code>RPP_REQUIRE_TTY</code>	Fail if there is no tty
<code>RPP_FORCELOWER</code>	Force input to lower case
<code>RPP_FORCEUPPER</code>	Force input to upper case
<code>RPP_SEVENBIT</code>	Strip the high bit from input
<code>RPP_STDIN</code>	Read passphrase from <code>stdin</code> , ignore the prompt

Tab 17. Values for the `flags` argument

We can modify the `checkpwd.c` program in the following way

```
#include <bsd/readpassphrase.h>

/*  printf("Enter password: ");
    if (fgets(input, sizeof(input), stdin) == NULL) {
        perror("fgets");
        return 1;
    }
*/
```

⁷⁴ See the `readpassphrase(3bsd)` manual page

⁷⁵ Berkeley Software Distribution Development Library

```
if(readpassphrase("Password:", input, sizeof(input), RPP_REQUIRE_TTY) == NULL)
    perror("readpassphrase");
```

The program is compiled by linking both the `crypt` and `bsd` libraries, since `readpassphrase` is provided by `libbsd`⁷⁶.

```
[linux@users] gcc -o checkpwd checkpwd.c -lcrypt -lbsd
```

```
[linux@users] ./checkpwd
```

```
Password:
```

```
Password correct!
```

During execution, the password prompt does not display the typed characters on the terminal, allowing the user to enter sensitive information securely. After the password is entered, the program hashes the input and compares it with the stored hash to verify correctness.

⁷⁶ See the `libbsd(7)` manual page

III.8 Login

On Linux systems, three files are maintained to keep track of the logins.

/var/run/utmp	User accounting database, who is currently logged in
/var/log/wtmp	User accounting log files, tracks all logins and logouts
/var/log/btmp	Tracks failed logins attempts

Tab 18. utmp, wtmp and btmp files

These files are not mandatory on a system, some distro use them while others don't. On Debian-based systems these files are not used and the management of the logins is performed by the `systemd` service.

When a user log in the `login`⁷⁷ program writes a record in the `utmp` file. To store this information the `utmp`⁷⁸ structure is used, it is defined as follows in the `utmp.h` header file.

```
struct utmp {
    short ut_type;           /* Type of record */
    pid_t ut_pid;            /* PID of login process */
    char ut_line[UT_LINESIZE]; /* Device name of tty - "/dev/" */
    char ut_id[4];           /* Terminal name suffix, or inittab(5) ID */

    char ut_user[UT_NAMESIZE]; /* Username */
    char ut_host[UT_HOSTSIZE]; /* Hostname for remote login, or kernel version for run-level messages */

    struct exit_status ut_exit; /* Exit status of a process marked as DEAD_PROCESS; not used by Linux init(1) */

#ifdef __WORDSIZE == 64 && defined __WORDSIZE_COMPAT32
    int32_t ut_session;        /* Session ID (getsid(2)) */
    struct {
        int32_t tv_sec;        /* Seconds */
        int32_t tv_usec;      /* Microseconds */
    } ut_tv;                  /* Time entry was made */
#else
    long ut_session;          /* Session ID */
    struct timeval ut_tv;     /* Time entry was made */
#endif

    int32_t ut_addr_v6[4];    /* Internet address of remote host; IPv4 address uses just ut_addr_v6[0] */
    char __unused[20];        /* Reserved for future use */
};
```

The `ut_type` field may take one of the following values.

⁷⁷ See the `login(1)` manual page

⁷⁸ See the `utmp(5)` manual page

Constant	Value	Description
EMPTY	0	Unused record
RUN_LVL	1	Runlevel change
BOOT_TIME	2	System boot time
NEW_TIME	3	Time after clock change
OLD_TIME	4	Time before clock change
INIT_PROCESS	5	Process spawned by <code>init</code>
LOGIN_PROCESS	6	Login process
USER_PROCESS	7	Normal user process (e.g. shell)
DEAD_PROCESS	8	Terminated process
ACCOUNTING	9	Not used in most Linux systems

Tab 19. `ut_type` values

The `exit_status` structure is defined as follows.

```
struct exit_status {
    short e_termination;    /* Process termination status */
    short e_exit;           /* Process exit status */
};
```

III.8.1 Working with the utmp file: getutent, getutid, setutent

To work with the `utmp` file, which stores information about current logins and system sessions, we can use the following functions⁷⁹.

```
#include <utmp.h>

struct utmp *getutent(void);
struct utmp *getutid(const struct utmp *ut);
struct utmp *getutline(const struct utmp *ut);

struct utmp *pututline(const struct utmp *ut);

void setutent(void);
void endutent(void);

int utmpname(const char *file);

Return: getutent, getutid, getutline: a pointer to a struct utmp on succes, NULL
on error, pututline: returns ut, NULL on error utmpname: returns 0 on success,
-1 on error and errno
```

The `utmpname` function sets the name of the `utmp`-format file for the other `utmp` functions to access. If `utmpname` is not used to set the filename before the other functions are used, they assume `_PATH_UTMP`, as defined in the file header `paths.h`.

The function `setutent` rewinds the file pointer to the beginning of the `utmp` file while `endutent` closes the `utmp` file.

The `getutent` function reads a line from the current file position in the `utmp` file and returns a pointer to a structure containing the fields of the line.

The function `getutid` searches forward from the current file position in the `utmp` file based upon `ut`. If `ut->ut_type` is one of `RUN_LVL`, `BOOT_TIME`, `NEW_TIME`, or `OLD_TIME`, `getutid` will find the first entry whose `ut_type` field matches `ut->ut_type`. If `ut->ut_type` is one of `INIT_PROCESS`, `LOGIN_PROCESS`, `USER_PROCESS`, or `DEAD_PROCESS`, `getutid` will find the first entry whose `ut_id` field matches `u->ut_id`.

The `getutline` function searches forward from the current file position in the `utmp` file. It scans entries whose `ut_type` is `USER_PROCESS` or `LOGIN_PROCESS` and returns the first one whose `ut_line` field matches `ut->ut_line`.

The `pututline` function writes the `utmp` structure `ut` into the `utmp` file. It uses `getutid` to search for the proper place in the file to insert the new entry. If it cannot find an appropriate slot for `ut`, `pututline` will append the new entry to the end of the file.

The `readutmp.c` program reads and displays entries from the `utmp` file related to logged-in user sessions.

```
1 #include <stdio.h>
2 #include <utmp.h>
3 #include <stdlib.h>
```

⁷⁹ See the `getutent(3)` manual page

```

4 int main(void) {
5     struct utmp *ut;

6     setutent();
7     while ((ut = getutent()) != NULL) {
8         if (ut->ut_type == USER_PROCESS) {
9             printf("User: %s Line: %s Host: %s Time: %ld\n",
                    ut->ut_user, ut->ut_line, ut->ut_host, ut->ut_tv.tv_sec);
10        }
11    }
12    endutent();
13    exit(EXIT_SUCCESS);
14 }

```

readutmp.c

Line 6. Calls `setutent` to reset the file position to the beginning of the `utmp` database

Line 7. Reads entries one by one from the `utmp` file using `getutent` and stores the result in `ut`

Line 8–9. Checks whether the current entry type is `USER_PROCESS`, meaning it represents an active user session, and prints information such as the username, terminal line, host, and login time

Line 12. Calls `endutent` to close the `utmp` database

We execute the program

```

[linux@users] ./readutmp
User: users Line: pts/0 Host: 192.168.1.100 Time: 1720001234

```

The program reads the `utmp` database and prints information about active user sessions. In the example output:

- User is the name of the logged-in user (`users`)
- Line is the terminal device associated with the session (`pts/0`, meaning pseudo-terminal 0)
- Host indicates the remote host from which the login originated (`192.168.1.100`). This field may be empty for local logins
- Time represents the login timestamp stored as UNIX epoch seconds (`1720001234`)

The `addremoveutmp.c` program demonstrates how to manipulate the `utmp` database by adding and removing login session entries. It prints the list of currently logged-in users (using `who`), inserts a new `USER_PROCESS` entry, displays the updated list, then marks the entry as `DEAD_PROCESS` to simulate logout, and finally prints the list again. The program is taken from the `getutent(3)` manual page.

```

1 #include <stdio.h>
2 #include <pwd.h>
3 #include <stdlib.h>
4 #include <string.h>

```

```

5 #include <time.h>
6 #include <unistd.h>
7 #include <utmp.h>

8 int main(void)
9 {
10     struct utmp entry;

11     system("echo before adding entry;;who");
12     entry.ut_type = USER_PROCESS;
13     entry.ut_pid = getpid();
14     strcpy(entry.ut_line, ttyname(STDIN_FILENO) + strlen("/dev/"));
15     strcpy(entry.ut_id, ttyname(STDIN_FILENO) + strlen("/dev/tty"));
16     entry.ut_tv.tv_sec = time(NULL);
17     entry.ut_tv.tv_usec = 0 ;
18     strcpy(entry.ut_user, getpwuid(getuid())->pw_name);
19     memset(entry.ut_host, 0, UT_HOSTSIZE);
20     entry.ut_addr_v6[0] = 0;
21     setutent();
22     if (pututline(&entry) == NULL) {
23         perror("pututline");
24         exit(EXIT_FAILURE);
25     }

26     system("echo after adding entry;;who");
27     entry.ut_type = DEAD_PROCESS;
28     memset(entry.ut_line, 0, UT_LINESIZE);
29     entry.ut_tv.tv_sec = time(NULL);
30     entry.ut_tv.tv_usec = 0 ;
31     memset(entry.ut_user, 0, UT_NAMESIZE);
32     setutent();
33     pututline(&entry);

34     system("echo after removing entry;;who");
35     endutent();
36     exit(EXIT_SUCCESS);
37 }

```

addremoveutmp.c

- Line 10. Declares an `utmp` structure that will be used to create and modify login entries*
- Line 11. Executes `who` to display the current list of logged-in users before modification*
- Line 12. Sets the entry type to `USER_PROCESS`, indicating an active login session*
- Line 13. Stores the current process ID in the `ut_pid` field*
- Line 14. Retrieves the terminal line name and stores it in `ut_line`*
- Line 15. Retrieves the terminal identifier and stores it in `ut_id`*
- Line 16–17. Sets the timestamp of the login entry using the current system time*
- Line 18. Stores the username associated with the current process*
- Line 19–20. Clears network-related fields (`ut_host` and `ut_addr_v6`) since this is a local session*

Line 21. Positions the `utmp` database pointer at the beginning using `setutent`
Line 22–25. Writes the new login entry into the `utmp` database using `pututline`
Line 26. Executes `who` again to show the newly added login session
Line 27. Changes the entry type to `DEAD_PROCESS`, marking the session as terminated
Line 28–31. Clears fields related to the session information
Line 32. Resets the `utmp` database pointer
Line 33. Writes the updated (logout) entry back into the database
Line 34. Executes `who` again to confirm that the entry has been removed
Line 35. Closes the `utmp` database with `endutent`

Since `utmp` can introduce security concerns if it is directly manipulated, Kali Linux typically disables or avoids relying on it, delegating session management to `systemd` services instead. For this reason, the program was tested on a Debian system where `utmp` is still available and used by compatibility tools.

We execute the program on a Debian system.

```
[apli@debian] sudo ./addremoveutmp ^
before adding entry:
apli  tty2      2025-07-05 01:27 (tty2)
apli  pts/1     2025-07-05 01:37
after adding entry:
apli  tty2      2025-07-05 01:27 (tty2)
apli  pts/1     2025-07-05 01:37
root  pts/1     2025-07-05 01:37
after removing entry:
apli  tty2      2025-07-05 01:27 (tty2)
apli  pts/1     2025-07-05 01:37
```

As shown in the output, after the `pututline` call, an additional entry appears in the `utmp` database (associated with `root` on `pts/1`). This entry is then removed when the program marks it as `DEAD_PROCESS`.

The `who`⁸⁰ command reads the `utmp` database to display the list of currently logged-in users. Another useful tool available on Debian systems is `utmpdump`⁸¹, which reads the binary `utmp` file and prints its contents in a human-readable format.

It is important to note that `utmp` is not a text file but a binary database, so it must be parsed before it can be interpreted. Both `who` and `utmpdump` perform this parsing and convert the raw records into readable output for the user.

On Kali Linux and other modern Debian-based systems, `utmp` is still present and supported, but it has become partially deprecated in practice and is no longer the main mechanism used for tracking user sessions. The traditional files still exist and continue to be maintained, including `/var/run/utmp` for current login sessions, `/var/log/wtmp` for login history, and `/var/log/btmp` for failed login attempts. These files are still used by standard tools such as `who`, `w`, and `last` to display user session information.

However, even though `utmp` is still available and functional, modern Debian-based systems increasingly rely on `systemd` components (such as `systemd-logind`) for session management. As a result, `utmp` should be considered a legacy interface that remains for compatibility rather than the primary source of truth for active sessions.

80 See the `who(1)` manual page

81 See the `utmpdump(1)` manual page

III.8.2 The login: login and logout

When a user logs into the system, a corresponding record is written to the `utmp` file, which tracks currently active sessions. When the user logs out, a related entry is recorded in the `wtmp` file, which maintains the historical log of login and logout events.

These operations are handled automatically by the following two functions⁸².

```
#include <utmp.h>

void login(const struct utmp *ut);
int logout(const char *ut_line);

Returns: logout, 1 if the entry was written in the db, 0 on error
```

The `login` function takes a `utmp` structure as an argument and records it in both the `utmp` and `wtmp` files, thereby registering the session as an active login and storing it in the login history. The `logout` function removes or updates the corresponding entry in the `wtmp` file to reflect the end of the session.

The `logincustom.c` program implements a simplified user login process on a Linux system. It authenticates the user using the shadow password database and, if the authentication succeeds, it updates system login records and starts a shell under the user's privileges.⁸³

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <pwd.h>
5 #include <shadow.h>
6 #include <string.h>
7 #include <stdlib.h>
8 #include <crypt.h>
9 #include <utmp.h>
10 #include <time.h>
11 #include <sys/types.h>
12 #include <sys/wait.h>
13 #include <errno.h>
14 #include <signal.h>
15 #include <grp.h>

16 void do_login_record(const char *username) {
17     struct utmp ut;
18     char *tty_full = ttyname(STDIN_FILENO);
19     if (!tty_full) return;
20     char *tty = tty_full + strlen("/dev/");
21     memset(&ut, 0, sizeof(ut));
22     ut.ut_type = USER_PROCESS;
```

⁸² See the `login(3)` manual page

⁸³ Modern Linux systems use PAM (Pluggable Authentication Modules) to handle login and security-related operations

```

23  ut.ut_pid = getpid();
24  strncpy(ut.ut_line, tty, sizeof(ut.ut_line));
25  strncpy(ut.ut_user, username, sizeof(ut.ut_user));
26  ut.ut_tv.tv_sec = time(NULL);
27  ut.ut_tv.tv_usec = 0;
28  login(&ut);
29 }

30 void do_logout_record(void) {
31     char *tty_full = ttyname(STDIN_FILENO);
32     if (tty_full) logout(tty_full + strlen("/dev/"));
33     endutent();
34 }

35 int main(void) {
36     char username[64];
37     char *password;
38     struct passwd *pw;
39     struct spwd *sp;
40     char *crypted, *input_crypted;

41     printf("Login: ");
42     fflush(stdout);
43     if (!fgets(username, sizeof(username), stdin)) return 1;
44     username[strcspn(username, "\n")] = 0;
45     password = getpass("Password: ");

46     pw = getpwnam(username);
47     if (!pw) {
48         fprintf(stderr, "Login incorrect.\n");
49         return 1;
50     }

51     sp = getspnam(username);
52     if (!sp) {
53         fprintf(stderr, "Cannot read shadow entry.\n");
54         return 1;
55     }

56     crypted = sp->sp_pwdp;
57     input_crypted = crypt(password, crypted);

58     if (strcmp(input_crypted, crypted) != 0) {
59         fprintf(stderr, "Login incorrect.\n");
60         return 1;
61     }
62     printf("Welcome %s!\n", username);

63     do_login_record(username);
64     if (initgroups(username, pw->pw_gid) < 0 || setgid(pw->pw_gid) < 0
        || setuid(pw->pw_uid) < 0) {

```

```

65     perror("Failed to drop privileges");
66     return 1;
67 }

68     char *shell = pw->pw_shell && pw->pw_shell[0] ? pw->pw_shell :
        "/bin/sh";
69     execlp(shell, shell, (char *)NULL);
70     perror("Failed to exec shell");
71     do_logout_record();
72     return 1;
73 }

```

logincustom.c

- Line 16. Defines a helper function that records a successful login event by adding an entry to the `utmp` database*
- Line 17–27. Prepares the `utmp` structure by filling in fields such as the process ID, terminal line, username, and login timestamp*
- Line 28. Calls `login` to write the entry into both `utmp` and `wtmp`, registering the session as an active login*
- Line 30–33. Defines a logout function that retrieves the current terminal and calls `logout` to remove or update the corresponding session entry, then closes the `utmp` database*
- Line 42–45. Handles user input: the program flushes output, reads the username from standard input, removes the newline character, and securely prompts for a password using `getpass`*
- Line 46–50. Retrieves the user account information from the system database (`/etc/passwd`). If the user does not exist, the login attempt is rejected*
- Line 51–55. Retrieves the corresponding shadow password entry from `/etc/shadow`. If access fails, the program terminates*
- Line 56–58. Extracts the stored encrypted password and hashes the user-provided password using `crypt`, then compares both values to verify authentication*
- Line 59–61. If authentication fails, the login attempt is rejected*
- Line 62. If authentication succeeds, the user is welcomed*
- Line 63–67. The program records the login in `utmp`, initializes the supplementary groups with `initgroups`, and drops privileges by setting the group ID and user ID to those of the authenticated user*
- Line 68–69. Determines the user's shell (or defaults to `/bin/sh`) and replaces the current process with it using `execlp`*
- Line 70–72. These lines are only executed if `execlp` fails, since a successful call replaces the current process image*

The line 45 uses the `getpass`⁸⁴ function that is now deprecated. This function displays a prompt and turns the echo off. It is defined as follows.

```

#include <unistd.h>

[[deprecated]] char *getpass(const char *prompt);

Returns: pointer to the password without the trailing newline

```

⁸⁴ See the `getpass(3)` manual page

The line 57 calls `crypt` passing the password typed by the user on `stdin` and the retrieved shadowed password `crypted`. `crypt` extracts the *salt* and the *algorithm* from the encrypted password, re-hashes the password using the same *method* and *salt* and returns the new generated encrypted string. This value can be compared with the crypted password. If they match the login is allowed.

The line 69 executes a shell therefore it never returns. `execvp` returns only if there is an error.

We execute the program on the Debian system. First, we create a new user using `useradd`

```
[apli@debian] sudo useradd newuser
[apli@debian] sudo passwd newuser
New password:
Retype new password:
passwd: password updated successfully

[apli@debian] sudo ./logincustom
Login: newuser
Password:
Welcome newuser!

$ whoami
newuser
$
```

On another terminal we run the `who` command to display the list of users currently logged into the system.

Shell 2

```
[apli@debian] who
apli    tty2    2025-07-05 06:44 (tty2)
newuser pts/1    2025-07-05 17:06
```

On the first terminal we perform the logout.

Shell 1

```
$ exit
[apli@debian]
```

In the second shell we re-execute `who`

Shell 2

```
[apli@debian] who
apli    tty2    2025-07-05 06:44 (tty2)
```

If a Linux system does not use `utmp` to manage user login records, it is still possible to work with a custom `utmp` file. In such cases, we can create our own `utmp` file and manipulate it using the same functions introduced earlier.

III.9 The Host, network, protocol and service databases

The resolution process for system databases such as users, groups, and hosts is controlled by the `nsswitch.conf` configuration file. This file determines the order and sources that the system uses when looking up information like usernames, group names, and network names.

For example, when a program queries user or group information (through functions like `getpwnam` or `getgrnam`), the system does not access only `/etc/passwd` or `/etc/group`. Instead, it follows the rules defined in `nsswitch.conf`, which may include local files, network services such as LDAP, or other name service providers. This allows Linux systems to be flexible and support multiple identity sources transparently.

III.9.1 The host database: `/etc/hosts`

The `/etc/hosts` file is a local host database⁸⁵ that maps IP addresses to hostnames. It provides a simple, static name resolution mechanism that is checked before querying DNS.

If the system's `nsswitch.conf` is configured as:

```
hosts:    files dns
```

this means that the Name Service Switch (NSS) will first attempt to resolve hostnames using local files (such as `/etc/hosts`). Only if the name is not found there will it proceed to query external DNS servers.

For each host a single line should be present with the following information:

```
IP_address canonical_hostname [aliases...]
```

An example of the host database is the following⁸⁶

```
# The following lines are desirable for IPv4 capable hosts
127.0.0.1          localhost

# 127.0.1.1 is often used for the FQDN of the machine
127.0.1.1          thishost.example.org    thishost
192.168.1.10       foo.example.org        foo
192.168.1.13       bar.example.org        bar
146.82.138.7       master.debian.org      master
209.237.226.90     www.opensource.org

# The following lines are desirable for IPv6 capable hosts
::1               localhost ip6-localhost ip6-loopback
ff02::1           ip6-allnodes
ff02::2           ip6-allrouters
```

⁸⁵ See the `hosts(5)` manual page

⁸⁶ From the `hosts(5)` manual page

The first column is the IP address and the second is the canonical hostname. Both the IPv4 and IPv6 addresses are settable in the file. To query the host database we can use the `gethostent` function⁸⁷, which is defined in the `netdb.h` header file as follows.

```
#include <netdb.h>

void sethostent(int stayopen);
void endhostent(void);
struct hostent *gethostent(void);

int gethostent_r(struct hostent *restrict ret, char
                 buf[restrict], rsize_t buflen, struct hostent
                 **restrict result, int *restrict h_errnop);

                Return: an hostent structure, NULL on failure
                0 on success, nonzero on failure
```

The `hostent` structure is defined as follows.

```
struct hostent {
    char *h_name;           /* official name of host */
    char **h_aliases;       /* alias list */
    int h_addrtype;         /* host address type */
    int h_length;           /* length of address */
    char **h_addr_list;     /* list of addresses */
}

#define h_addr h_addr_list[0] /* for backward compatibility */
```

The `gethostent` function returns the next entry in the host database. The `gethostent_r` function is the reentrant version of `gethostent` that supplies a `hostent` structure `ret` which will be filled in on success, and a temporary work buffer `buf` of size `buflen`. After the call, `result` will point to the result on success. In case of an error or if no entry is found `result` will be `NULL`. The functions return 0 on success and a nonzero error number on failure.

The `sethostent` function specifies, if `stayopen` is true (1), that the file must be kept open between calls, and that if `stayopen` is false (0), the file will be closed after each call.

The `endhostent` function closes the file.

The `gethost.c` program demonstrates how to access and read the host database using the system's network name service functions.

```
1 #include <stdio.h>
2 #include <netdb.h>
3 #include <arpa/inet.h>
4 #include <string.h>

5 int main(void) {
6     struct hostent *host;
7     sethostent(0);
```

⁸⁷ See the `gethostbyname(3)` manual page

```

8   while ((host = gethostent()) != NULL) {
9       printf("Name: %s\n", host->h_name);
10      for (char **alias = host->h_aliases; *alias != NULL; alias++)
11          {
12              printf("  Alias: %s\n", *alias);
13          }
14      for (int i = 0; host->h_addr_list[i] != NULL; i++) {
15          char ip[INET_ADDRSTRLEN];
16          inet_ntop(AF_INET, host->h_addr_list[i], ip, sizeof(ip));
17          printf("  Address: %s\n", ip);
18      }
19      printf("\n");
20  }
21  endhostent();
22  return 0;
23 }

```

gethost.c

Line 6. Declares a pointer to a `hostent` structure, which will store each host entry retrieved from the database

Line 7. Calls `sethostent(0)` to open the host database and configure it to not keep the file open between reads

Line 8. Iterates through all entries in the host database using `gethostent` until no more records are available

Line 9. Prints the canonical hostname of the current entry

Line 10–13. Iterates through the list of aliases associated with the hostname and prints each one

Line 14–18. Iterates through all IP addresses associated with the host. Each binary address is converted into a human-readable string using `inet_ntop` before being printed

Line 21. Closes the host database using `endhostent`

Since a hostname can have multiple addresses, `gethostent` returns a list of structures containing these addresses. The line 14 iterates over the list of IP addresses associated with the given host.

The `INET_ADDRSTRLEN` constant is defined with a value of 16 to hold an IP address in the file header `netinet/in.h` that is imported by `netdb.h` and `arpa/inet.h`.

The function `inet_ntop`⁸⁸ of the line 16 converts IPv4 and IPv6 addresses from binary to text form and is explained in the *Theory and insights* section.

We execute the program

```

[linux@users] ./gethost
Name: localhost
  Address: 127.0.0.1

Name: kali
  Address: 127.0.1.1

Name: localhost

```

⁸⁸ See the `inet_ntop(3)` manual page

Alias: ip6-localhost Alias: ip6-loopback Address: 127.0.0.1

The output shows the entries stored in the host database as resolved by the system. For each host, the program prints its canonical name, followed by any aliases, and then the associated IP addresses. For example, `localhost` is mapped to the loopback address `127.0.0.1`, while the system hostname `kali` is mapped to `127.0.1.1` as defined in the local configuration.

III.9.2 The network database: `/etc/networks`

The `/etc/networks` file⁸⁹ contains the list of the networks of the system. Each line represents a network and has the following structure

```
name number aliases ...
```

The *name* field represents the symbolic name of the network, the *number* field corresponds to its network address, and the *aliases* field contains any optional alternative names that the network may have.

An example of this file is the following

```
default      0.0.0.0
loopback     127.0.0.0
link-local   169.254.0.0
```

To work with the network database, we can use the following functions⁹⁰.

```
#include <netdb.h>

struct netent *getnetent(void);

struct netent *getnetbyname(const char *name);
struct netent *getnetbyaddr(uint32_t net, int type);

void setnetent(int stayopen);
void endnetent(void);

Return: a pointer to a netent struct, NULL on error or EOF
```

While the `-ent` functions are the analogous of the same functions we have seen for the hosts database, the `getnetbyname` and `getnetbyaddr` functions retrieve a specific entry based on the passed parameters. `getnetbyname` returns the entry that matches the network name `name`, `getnetbyaddr` returns the entry that matches the network number `net` of type `type`.

The `netent` structure is defined in the `netdb.h` header file as follows.

```
struct netent {
    char *n_name;           /* official network name */
    char **n_aliases;       /* alias list */
    int n_addrtype;         /* net address type */
    uint32_t n_net;         /* network number */
}
```

The `n_addrtype` field has always the value `AF_INET`.

⁸⁹ See the `networks(5)` manual page

⁹⁰ See the `getnetent(3)` manual page

The `getnetbyname.c` program retrieves a network entry from the system networks database based on the name provided as a command-line argument.

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <netdb.h>
4 #include <arpa/inet.h>

5 int main(int argc, char *argv[]) {

6     if (argc != 2) {
7         printf("\nUsage %s <net_name>", argv[0]);
8         exit(EXIT_FAILURE);
9     }

10    struct netent *net = getnetbyname(argv[1]);
11    if (net == NULL) {
12        printf("\nNetwork %s not found", argv[1]);
13        exit(EXIT_FAILURE);
14    } else {
15        printf("\n Network %s found \n\n", net->n_name);
16        printf(" Official Name: %s\n", net->n_name);
17        printf(" Address : %s\n", inet_ntoa((struct in_addr)
            {htonl(net->n_net)}));
18        for (char **alias = net->n_aliases; *alias != NULL; alias++) {
19            printf(" Alias : %s\n", *alias);
20        }
21    }
22    exit(EXIT_SUCCESS);
23 }
```

getnetbyname.c

Line 10. Calls `getnetbyname` to search the network database for an entry matching the given network name

Line 11–14. Checks whether the lookup succeeded. If no matching network is found, the program prints an error message and exits

Line 15–16. Prints the official name of the network as stored in the database

Line 17. Converts the network number into a human-readable IP-style format and prints it

Line 18–20. Iterates through the list of aliases associated with the network and prints each one

The line 27 converts the `net->n_net` value from the host byte order to the network byte order with `htonl`⁹¹, then it converts the internet address (given in network byte order) to a string in IPv4 dotted-decimal notation with `inet_ntoa`⁹². Before this last conversion the value is casted to a structure of type `in_addr`. This structure represents an *Internet Address* and is defined as follows in the `netinet/in.h` header file.

⁹¹ See the `byteorder(3)` manual page

⁹² See the `inet(3)` manual page

```

/* Internet address. */
typedef uint32_t in_addr_t;
struct in_addr
{
    in_addr_t s_addr;
};

```

The `in_addr_t` type is a 32 bytes unsigned integer that stores the value of the IP address.

We execute the program

```

[linux@users] ./getnetbyname loopback
Network loopback found

Official Name: loopback
Address      : 127.0.0.0

[linux@users] ./getnetbyname link-local
Network link-local found

Official Name: link-local
Address      : 169.254.0.0

```

The output shows the result of querying the networks database using the network names provided as input. For each network, the program prints the official name and its corresponding network address in human-readable form. For example, the *loopback* network corresponds to `127.0.0.0`, while the *link-local* network corresponds to `169.254.0.0`.

To perform a lookup based on the network address instead of the network name, we can replace

```

10    struct netent *net = getnetbyname(argv[1]);

```

with the following

```

struct in_addr addr;
if (!inet_aton(argv[1], &addr)) {
    fprintf(stderr, "Invalid IP address: %s\n", argv[1]);
    return 1;
}

uint32_t net_num = ntohl(addr.s_addr);
struct netent *net = getnetbyaddr(net_num, AF_INET);

```

The `inet_aton`⁹³ function converts the passed Internet host address from the IPv4 numbers-and-dots notation into binary form (in network byte order) and stores it in the structure passed as second argument.

93 See the `inet(3)` manual page

III.9.3 The protocol database: `/etc/protocols`

The file `/etc/protocols` is the system's protocols database⁹⁴. It contains a list of the various DARPA Internet protocols supported by the TCP/IP subsystem, mapping protocol names to their corresponding numerical identifiers.

This file allows applications and system libraries to translate between human-readable protocol names (such as `tcp`, `udp`, or `icmp`) and their numeric protocol values used internally by the network stack.

Each line is of the following format:

```
protocol number aliases ...
```

The `protocol` field is the native name of the protocol, while `number` is the official number of the protocol and `aliases` defines the optional aliases the protocol can have.

The file has a fixed content and any changes to it would result in incorrect IP packages. The protocol numbers and names are specified by the IANA (Internet Assigned Numbers Authority).

The first 4 lines of the file are as follows:

```
ip          0      IP      # internet protocol, pseudo protocol number
hopopt      0      HOPOPT  # IPv6 Hop-by-Hop Option [RFC1883]
icmp 1      ICMP     # internet control message protocol
igmp 2      IGMP     # Internet Group Management
```

To work with the protocol database the following functions⁹⁵ are available.

```
#include <netdb.h>

struct protoent *getprotoent(void);

struct protoent *getprotobyname(const char *name);
struct protoent *getprotobynumber(int proto);

void setprotoent(int stayopen);
void endprotoent(void);

Return: a pointer to a protoent struct, NULL on error or EOF
```

The `protoent` structure is defined as follows.

```
struct protoent {
    char *p_name;           /* official protocol name */
    char **p_aliases;       /* alias list */
    int p_proto;            /* protocol number */
}
```

⁹⁴ See the `protocols(5)` manual page

⁹⁵ See the `getprotoent(3)` manual page

The `getprotonum.c` program retrieves information about a network protocol from the system protocols database based on the numeric protocol identifier provided as input.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <netdb.h>

4 int main(int argc, char *argv[]) {
5     if (argc != 2) {
6         fprintf(stderr, "Usage: %s <protocol_number>\n", argv[0]);
7         exit(EXIT_FAILURE);
8     }

9     int proto_num = atoi(argv[1]);
10    struct protoent *proto = getprotobynumber(proto_num);
11    if (proto == NULL) {
12        printf(" Protocol number %d not found.\n", proto_num);
13        exit(EXIT_FAILURE);
14    }
15    printf(" Protocol number %d:\n", proto_num);
16    printf(" Name : %s\n", proto->p_name);
17    for (char **alias = proto->p_aliases; *alias != NULL; alias++) {
18        printf(" Alias : %s\n", *alias);
19    }
20    exit(EXIT_SUCCESS);
21 }
```

getprotonum.c

Line 9. Converts the command-line argument from a string to an integer representing the protocol number

Line 10. Calls `getprotobynumber` to search the protocols database (`/etc/protocols`) for a matching protocol entry

We execute the program

```
[linux@users] ./getprotonum 0
Protocol number 0:
Name : ip
Alias : IP

[linux@users] ./getprotonum 1
Protocol number 1:
Name : icmp
Alias : ICMP
```

The output shows the protocol information retrieved from the system protocols database (`/etc/protocols`). Each protocol number is mapped to its corresponding name and optional aliases. For example, protocol 0 corresponds to `ip`, while protocol 1 corresponds to `icmp`.

III.9.4 The service database: `/etc/services`

The `/etc/services` file is the service database⁹⁶. It provides the mapping between internet services and their assigned port numbers and protocol types. Port numbers are assigned by the IANA, and the current policy is to assign both TCP and UDP protocols when assigning a port number. The port numbers below 1024 (so-called "low numbered" ports) can be bound to only by the `root` user. In the file header `netdb.h` is defined the `_PATH_SERVICES` constant that specifies the path of the services file.

Each line of the file has the following format:

```
service-name    port/protocol    [aliases ...]
```

A sample service file look as follows:

```
netstat          15/tcp
gotd              17/tcp          quote
msp              18/tcp          # message send protocol
msp              18/udp          # message send protocol
chargen          19/tcp          ttytst source
chargen          19/udp          ttytst source
ftp              21/tcp
# 22 - unassigned
telnet           23/tcp
```

To work with the service database the following functions⁹⁷ are available.

```
#include <netdb.h>

struct servent *getservent(void);

struct servent *getservbyname(const char *name, const char *proto);
struct servent *getservbyport(int port, const char *proto);

void setservent(int stayopen);
void endservent(void);
```

The `servent` structure is defined as follows.

```
struct servent {
    char *s_name;           /* official service name */
    char **s_aliases;       /* alias list */
    int s_port;             /* port number */
    char *s_proto;          /* protocol to use */
}
```

⁹⁶ See the `services(5)` manual page

⁹⁷ See the `getservent(3)` manual page

The `getservice.c` program retrieves information about a network service from the system services database based on the service name and transport protocol provided by the user.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <netdb.h>
5 #include <arpa/inet.h>

6 int main(int argc, char *argv[]) {
7     if (argc != 3) {
8         fprintf(stderr, "Usage: %s <service_name> <tcp|udp>\n",
9             argv[0]);
10        return EXIT_FAILURE;
11    }

12    const char *service = argv[1];
13    const char *protocol = argv[2];

14    if (strcmp(protocol, "tcp") != 0 && strcmp(protocol, "udp") != 0) {
15        fprintf(stderr, " Protocol must be 'tcp' or 'udp'\n");
16        return EXIT_FAILURE;
17    }

18    struct servent *svc = getservbyname(service, protocol);
19    if (svc == NULL) {
20        printf(" No service found for '%s/%s'\n", service, protocol);
21        return EXIT_FAILURE;
22    }

23    printf(" Service: %s\n", svc->s_name);
24    printf(" Port : %d\n", ntohs(svc->s_port));
25    printf(" Proto : %s\n", svc->s_proto);

26    for (char **alias = svc->s_aliases; *alias != NULL; alias++) {
27        printf(" Alias : %s\n", *alias);
28    }
29    return EXIT_SUCCESS;
30 }
```

getservice.c

Line 17. Calls `getservbyname` with the service name and protocol (TCP or UDP) to search the services database (typically `/etc/services`) and obtain the corresponding service entry

We execute the program.

<pre>[linux@users] ./getservice ftp tcp Service: ftp</pre>
--

```
Port : 21  
Proto : tcp
```

```
[linux@users] ./getservice ssh tcp  
Service: ssh  
Port : 22  
Proto : tcp
```

```
[linux@users] ./getservice sip udp  
Service: sip  
Port : 5060  
Proto : udp
```

The output shows the result of querying the system services database (typically `/etc/services`) using the service name and protocol provided as input. Each service is mapped to its corresponding well-known port number and transport protocol. For example, `ftp` uses port 21 over **TCP**, `ssh` uses port 22 over **TCP**, and `sip` uses port 5060 over **UDP**.

III.10 System identification

III.10.1 Getting system information: `uname`

On Linux systems we can get information about the identity of the system through the `uname`⁹⁸ command, which is implemented with the `uname`⁹⁹ system call, defined as follows.

```
#include <sys/utsname.h>

int uname(struct utsname *name);

Returns: a non-negative value on success, on error -1 and errno
```

The `utsname` structure is defined as follows in the `sys/utsname.h` header file.

```
#define __NEW_UTS_LEN 64

struct new_utsname {
    char sysname[__NEW_UTS_LEN + 1];
    char nodename[__NEW_UTS_LEN + 1];
    char release[__NEW_UTS_LEN + 1];
    char version[__NEW_UTS_LEN + 1];
    char machine[__NEW_UTS_LEN + 1];
    char domainname[__NEW_UTS_LEN + 1];
};
```

The fields of the structure are listed in the following table.

<code>sysname</code>	System name
<code>nodename</code>	Host name
<code>release</code>	OS Release
<code>version</code>	OS version
<code>machine</code>	Hardware identifier
<code>domainname</code>	Network domain name

Tab 20. Fields of the `utsname` structure

The `systemid.c` program demonstrates how to retrieve basic system identification information using the `uname` system call.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <sys/utsname.h>

4 int main(void) {
```

⁹⁸ See the `uname(1)` manual page

⁹⁹ See the `uname(3POSIX)` manual page

```

5     struct utsname sysinfo;
6     if (uname(&sysinfo) == -1) {
7         perror("uname");
8         return 1;
9     }

10    printf("System Information:\n");
11    printf("  Sysname  : %s\n", sysinfo.sysname);
12    printf("  Nodename: %s\n", sysinfo.nodename);
13    printf("  Release  : %s\n", sysinfo.release);
14    printf("  Version  : %s\n", sysinfo.version);
15    printf("  Machine  : %s\n", sysinfo.machine);
16    printf("  Domainname: %s\n", sysinfo.domainname);
17    return 0;
18 }

```

systemid.c

Line 5. Declares a `utsname` structure that will hold system information such as the kernel and machine details

Line 6–9. Calls `uname` to fill the structure with information about the current system. If the call fails, an error message is printed and the program exits

Line 10–16. Prints various system identification fields, including operating system name, node name (hostname), kernel release, kernel version, machine architecture, domain name

We execute the program

```

[linux@users] ./systemid
System Information:
  Sysname : Linux
  Nodename: kali
  Release : 6.12.33+kali-amd64
  Version : #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25)
  Machine : x86_64
  Domainname: (none)

```

The output displays key information about the running system obtained through the `uname` system call. It includes the operating system name (`Linux`), the hostname (`kali`), the kernel release and version, the machine architecture (`x86_64`), and the domain name (which is not set in this case, shown as `(none)`).

The `domainname` field is not a POSIX feature but a GNU extension, therefore to retrieve this value we need to define the feature test macro `_GNU_SOURCE` or compile as follows

```
gcc -D_GNU_SOURCE -o systemid systemid.c
```

On the command line, we can use the `uname` utility to retrieve system information. When it is executed with the `-a` option, it prints a complete overview of the system, including the kernel name, hostname, kernel version, build information, machine architecture, and operating system type.

```
[linux@users] uname -a
Linux kali 6.12.33+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25)
x86_64 GNU/Linux
```

Under the `/proc` filesystem there are the following files containing the same information we can obtain with `uname`.

<code>/proc/sys/kernel/ostype</code>	Sysname	Linux
<code>/proc/sys/kernel/osrelease</code>	Release	6.12.33+kali-amd64
<code>/proc/sys/kernel/version</code>	OS version	#1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25)
<code>/proc/version</code>	OS name, release, machine and version	Linux version 6.12.33+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.0, GNU ld (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25)

Tab 21. Files related to `uname`

III.10.2 Getting and setting hostname: `gethostname` and `sethostent`

The following functions¹⁰⁰ can be used to retrieve and set the hostname and the NIS domain name.

```
#include <unistd.h>

int gethostname(char *name, size_t len);
int sethostname(const char *name, size_t len);

int getdomainname(char *name, size_t len);
int setdomainname(const char *name, size_t len);

Return: 0 on success, on error -1 and errno
```

The `gethostname` function returns the name of the host in the `name` argument. The `getdomainname` function returns the domain name in the `name` argument.

The `gethostdom.c` program demonstrates how to retrieve and display the system's hostname and domain name using standard system calls.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <limits.h>
5 #include <string.h>
6 #include <errno.h>

7 int main(void) {
8     char hostname[HOST_NAME_MAX + 1];
9     char domainname[HOST_NAME_MAX + 1];

10    if (gethostname(hostname, sizeof(hostname)) == -1) {
11        perror("gethostname");
12        return 1;
13    }
14    if (getdomainname(domainname, sizeof(domainname)) == -1) {
15        perror("getdomainname");
16        return 1;
17    }

18    printf("Hostname : %s\n", hostname);
19    printf("Domain name : %s\n", domainname);
20    return 0;
21 }
```

gethostdom.c

¹⁰⁰ See the `gethostname(2)` and `getdomainname(2)` manual pages

Line 8–9. Declares buffers to store the hostname and domain name, each with a size defined by `HOST_NAME_MAX + 1` to ensure sufficient space

Line 10–13. Calls `gethostname` to obtain the system's hostname. If the call fails, an error is printed and the program exits

Line 14–17. Calls `getdomainname` to retrieve the system's domain name. Similarly, if it fails, an error message is displayed and the program terminates

Line 18–19. Prints the retrieved hostname and domain name

We execute the program.

```
[linux@users] ./gethostdom
Hostname   : kali
Domain name : (none)
```

The two corresponding command for these system calls are `hostname` and `domainname`

```
[linux@users] hostname
kali

[linux@users] domainname
(none)
```

We can add the following code after the line 19 in order to change the hostname

```
if (sethostname("linux", 5) == -1) {
    perror("sethostname");
    return 1;
}
if (gethostname(hostname, sizeof(hostname)) == -1) {
    perror("gethostname");
    return 1;
}
printf("Hostname : %s\n", hostname);
return 0;
```

We execute the program using `sudo`

```
[linux@users] sudo ./gethostdom
Hostname   : kali
Domain name : (none)
Hostname   : linux

[linux@users] hostname
linux
```

If we run the `hostname` command after executing the program, we can observe the updated system hostname reflected in the output.

The `/etc/hostname` file contains the system's configured hostname, which is applied during boot. When the system starts, this value is read and used to set the machine's hostname.

If the system is rebooted, the hostname defined in `/etc/hostname` becomes the active hostname again, overriding any temporary changes made during the previous session.

```
[linux@users] cat /etc/hostname  
kali
```

The `sethostname` function does not change the content of the `/etc/hostname` file, instead it changes the `/proc/sys/kernel/hostname` value. This will be the value until the next reboot or the next change.

```
[linux@users] cat /proc/sys/kernel/hostname  
linux
```

The same value can be retrieve from the system configuration using the `sysctl`¹⁰¹ command.

```
[linux@users] sudo sysctl kernel.hostname  
kernel.hostname = linux
```

The table below summarizes the different places where the hostname is stored and how it can be modified.

File / System setting	Description	Change with
<code>/etc/hostname</code>	Global configuration read at the boot	Edit the file to change the value
<code>/proc/sys/kernel/hostname</code>	Kernel memory value	<code>sethostname</code> , <code>hostname</code>
<code>kernel.hostname</code>	Kernel memory value	<code>sethostname</code> , <code>hostname</code>

Tab 22. File and settings related to the hostname

101 See the `sysctl(8)` manual page

III.11 Pluggable Authentication Modules

The login process on Linux is managed by the PAM¹⁰². Linux-PAM is a system of libraries designed to handle the authentication of the applications. Also the `login` program uses PAM to authenticate the user. The authentication process of PAM is configurable in a way that the system administrator can choose how an application will authenticate the users. The main configuration file is `/etc/pam.conf` and in the `/etc/pam.d` folder there are the files related to the applications authentication configuration. If this directory exists, PAM will ignore the `pam.conf` file. The configuration files in `/etc/pam.d` define the connection between the applications (services) and the pluggable authentication modules (PAMs) that perform the actual authentication tasks.

The authentication tasks are separated in four groups:

Account management	Account verification types of service: has the user's password expired?, is this user permitted access to the requested service?
Authentication management	Authenticate a user and set up user credentials
Password management	Update authentication mechanism
Session management	Things that should be done prior to a service being given and after it is withdrawn, e.g. mounting of the user home directory

Tab 23. PAM authentication tasks

¹⁰² See the `pam(7)` manual page

III.11.1 PAM Authentication: `pam_authenticate`

To integrate PAM into the `logincustom.c` program, we modify it so that authentication and session handling are delegated to PAM modules instead of manually checking `/etc/shadow` and managing login logic ourselves. To support this change, we also create a PAM configuration file for the new `loginpam.c` program.

```
[linux@users] sudo vi /etc/pam.d/loginpam
# /etc/pam.d/loginpam
auth      required pam_unix.so
account   required pam_unix.so
```

We have defined in the file `loginpam`, two groups: `auth` and `account`. The first column is the authentication task to perform, the second column tells PAM that the module of the third column is required, the third column is the library that will be used to manage the authentication task.

We modify the original `logincustom.c` program so that it uses PAM (Pluggable Authentication Modules) instead of directly accessing `/etc/shadow` via `getspnam` and performing password verification with `crypt`. This results in a new version of the program called `loginpam.c`.

In this updated version, authentication is delegated to PAM, which provides a standardized and secure framework for handling user login requests. As a consequence, the manual retrieval of shadow entries and password hashing is removed from the code. However, the functions `do_login_record` and `do_logout_record` remain unchanged, since they are responsible for updating the `utmp/wtmp` records and are independent of the authentication mechanism used.

This separation of concerns allows PAM to handle security and authentication, while the program continues to manage session accounting as before.

```
1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6 #include <pwd.h>
7 #include <grp.h>
8 #include <errno.h>
9 #include <utmp.h>
10 #include <time.h>
11 #include <sys/types.h>
12 #include <security/pam_appl.h>
13 #include <security/pam_misc.h>

14 static struct pam_conv conv = {
15     misc_conv,
16     NULL
17 };

18 int main(void) {
19     char username[64];
20     struct passwd *pw;
21     pam_handle_t *pamh = NULL;
```

```

22  int pam_result;

23  printf("Login: ");
24  fflush(stdout);
25  if (!fgets(username, sizeof(username), stdin)) return 1;
26  username[strcspn(username, "\n")] = 0;

27  pam_result = pam_start("loginpam", username, &conv, &pamh);
28  if (pam_result != PAM_SUCCESS) {
29      fprintf(stderr, "pam_start failed: %s\n", pam_strerror(pamh,
30          pam_result));
31      return 1;
32  }

32  pam_result = pam_authenticate(pamh, 0);
33  if (pam_result != PAM_SUCCESS) {
34      fprintf(stderr, "Authentication failed: %s\n",
35          pam_strerror(pamh, pam_result));
36      pam_end(pamh, pam_result);
37      return 1;
38  }

38  pam_result = pam_acct_mgmt(pamh, 0);
39  if (pam_result != PAM_SUCCESS) {
40      fprintf(stderr, "Account not permitted: %s\n",
41          pam_strerror(pamh, pam_result));
42      pam_end(pamh, pam_result);
43      return 1;
44  }

44  printf("Welcome %s!\n", username);

45  pw = getpwnam(username);
46  if (!pw) {
47      fprintf(stderr, "getpwnam failed.\n");
48      pam_end(pamh, PAM_SUCCESS);
49      return 1;
50  }

51  do_login_record(username);

52  if (initgroups(username, pw->pw_gid) < 0 || setgid(pw->pw_gid) < 0
53      || setuid(pw->pw_uid) < 0) {
54      perror("Failed to drop privileges");
55      pam_end(pamh, PAM_SUCCESS);
56      return 1;
57  }

57  char *shell = pw->pw_shell && pw->pw_shell[0] ? pw->pw_shell :
58      "/bin/sh";
59  execlp(shell, shell, (char *)NULL);

```

```

59  perror("Failed to exec shell");
60  do_logout_record();
61  pam_end(pamh, PAM_SUCCESS);
62  return 1;
63 }

```

loginpam.c Login using PAM

- Line 14–17. Defines the PAM conversation structure, which uses `misc_conv` to interact with the user (for example, to prompt for a password)*
- Line 23–26. Prompts the user for a username and removes the trailing newline from the input*
- Line 27–31. Initializes a PAM transaction using `pam_start` with the service name "loginpam". If initialization fails, the program terminates with an error*
- Line 32–37. Performs authentication using `pam_authenticate`. If authentication fails, the PAM session is properly closed with `pam_end` and the program exits*
- Line 38–43. Performs account management checks using `pam_acct_mgmt`, which verifies whether the account is permitted to log in (e.g., not expired or disabled)*
- Line 44. Prints a welcome message if authentication succeeds*
- Line 45–50. Retrieves the user's system entry from `/etc/passwd` using `getpwnam`. This is needed to obtain UID, GID, and shell information*
- Line 51. Calls `do_login_record` to record the login event in `utmp/wtmp`*
- Line 52–56. Sets group membership (`initgroups`) and drops privileges by switching to the user's UID and GID*
- Line 57–58. Launches the user's login shell using `execlp`, replacing the current process image*
- Line 59–62. These lines execute only if `execlp` fails. In that case, the program prints an error, records logout, and ends the PAM session*

To access the manual pages and development resources for PAM-related functions, it is necessary to install the `libpam0g-dev` package. This package provides the headers, documentation, and development files required to work with PAM APIs such as `pam_start`, `pam_authenticate`, and `pam_acct_mgmt`, as well as their corresponding manual pages.

[\[linux@users\]](#) `sudo apt install libpam0g-dev`

The prototypes of the used functions are as follows.

```

#include <security/pam_appl.h>

int pam_start(const char *service_name, const char *user, const struct
              pam_conv *pam_conversation, pam_handle_t **pamh);

int pam_start_confdir(const char *service_name, const char *user, const
                      struct pam_conv *pam_conversation, const char
                      *confdir, pam_handle_t **pamh);

Return: PAM_SUCCESS | PAM_ABORT | PAM_BUF_ERR | PAM_SYSTEM_ERR

```

The `pam_start`¹⁰³ function creates the PAM context and initiates the PAM transaction. The `service_name` argument is the name of the application for which the context will be created and it will be stored as `PAM_SERVICE`, the argument `user` specifies the target user and it will be stored as `PAM_USER` in the context, the `pam_conversation` argument defines the conversation function to use (it is needed for the direct communication between a loaded module and the application), the `pamh` argument is the handle that contains the PAM context after a successful call (`PAM_SUCCESS`).

The `pam_start_confdir` works like `pam_start` but allows to specify a configuration directory to use at the place of `/etc/pam.d` (default).

The `pam_authenticate`¹⁰⁴ function performs the account authentication and is defined as follows.

```
#include <security/pam_appl.h>

int pam_authenticate(pam_handle_t *pamh, int flags);

Returns: PAM_SUCCESS | PAM_ABORT | PAM_AUTH_ERR | PAM_CRED_INSUFFICIENT |
         PAM_AUTHINFO_UNAVAIL | PAM_MAXTRIES | PAM_USER_UNKNOWN
```

The `pamh` argument is the handle returned by a previous call to `pam_start`, and the `flags` argument can be zero or more of the following values:

<code>PAM_SILENT</code>	Do not emit any messages
<code>PAM_DISALLOW_NULL_AUTHTOK</code>	The PAM module service should return <code>PAM_AUTH_ERR</code> if the user does not have a registered authentication token

Tab 24. Values of the `flags` argument

`pam_authenticate` authenticates the user with the name passed before to `pam_start`.

The `pam_acct_mgmt`¹⁰⁵ function validates the account and is defined as follows.

```
#include <security/pam_appl.h>

int pam_acct_mgmt(pam_handle_t *pamh, int flags);

Returns: PAM_SUCCESS | PAM_ACCT_EXPIRED | PAM_AUTH_ERR | PAM_NEW_AUTHTOK_REQD |
         PAM_PERM_DENIED | PAM_USER_UNKNOWN
```

The function checks if the user's account is valid. It checks: the authentication token (password, finger print, etc..), the account expiration and access restriction.

The `pamh` parameter is the handle, and `flags` may take the values listed in the Table 22.

The `pam_end`¹⁰⁶ function terminates the PAM transaction. It is the last function an application should call in the PAM context and is defined as follows.

¹⁰³ See the `pam_start(3)` manual page

¹⁰⁴ See the `pam_authenticate(3)` manual page

¹⁰⁵ See the `pam_acct_mgmt(3)` manual page

¹⁰⁶ See the `pam_end(3)` manual page

```
#include <security/pam_appl.h>
```

```
int pam_end(pam_handle_t *pamh, int pam_status);
```

Returns: PAM_SUCCESS | PAM_SYSTEM_ERR

The `pam_status` argument should be set to the value returned to the application by the last PAM library call and it will be passed to the module callback function in a way the module can know if the last PAM call has terminated successfully or if it has failed. This function free all the memory for the items associated with the handle.

The `pam_strerror` function returns a pointer to a string describing the PAM error code.

```
#include <security/pam_appl.h>
```

```
const char *pam_strerror(pam_handle_t *pamh, int errnum);
```

Returns: a pointer to string describing the error

We compile the program by linking it against the PAM-related libraries (`libpam`, `libpam_misc`) and the utility library (`libutil`), since the program relies on PAM functions and helper utilities for authentication and session handling. To execute the program correctly, it must be run with elevated privileges because it performs authentication and changes the user's identity.

```
[linux@users] gcc -o loginpam loginpam.c -lpam -lpam_misc -lutil
```

```
[linux@users] sudo ./loginpam
```

```
Login: newuser
```

```
Password:
```

```
Welcome newuser!
```

```
$ whoami
```

```
newuser
```

The output shows that after successful PAM authentication, the program starts a shell under the authenticated user's identity. This is confirmed by the `whoami` command, which returns `newuser`, indicating that the privilege switch and session setup were correctly applied.

The lines 14-17 define a struct of type `pam_conv` to specify how the application will handle the authentication prompts. The structure `pam_conv` is defined in the file header `security/_pam_types.h` as follows

```
struct pam_conv {  
    int (*conv)(int num_msg, const struct pam_message **msg, struct  
                pam_response **resp, void *appdata_ptr);  
    void *appdata_ptr;  
};
```

`conv` is a pointer to a callback function that PAM uses to ask questions (like password prompts). `appdata_ptr` is user-defined data we can pass to the callback.

`misc_conv` is a standard implementation provided by the `libpam_misc` library that automatically handles PAM prompts (like "Password:") using `getpass` for hidden input and `printf` for the messages.

Internally PAM will execute

```
conv(1, &msg, &resp, NULL);
```

and this will display the prompt. If we want to personalize the prompt we have to use a custom `pam_conv` function.

The structures `pam_message` and `pam_response` are defined as follows in the `security/_pam_types.h` header file.

```
struct pam_message {
    int msg_style;
    const char *msg;
};

struct pam_response {
    char *resp;
    int resp_retcode;
};
```

The `msg_style` field can be one of the following values.

PAM_PROMPT_ECHO_OFF	Turn off the echo
PAM_PROMPT_ECHO_ON	Turn on the echo
PAM_ERROR_MSG	Error message style
PAM_TEXT_INFO	Informational message style

Tab 25. `msg_style` values

III.11.2 PAM Session: `pam_open_session` and `pam_close_session`

If we want our program to behave like a real login and ensure full PAM compatibility, we must add the session management. With the session management we can integrate the logging, the resource limit, the environment and we can ensure the login session is recognized by `systemd`.

We will use the following functions¹⁰⁷.

```
#include <security/pam_appl.h>

int pam_open_session(pam_handle_t *pamh, int flags);

int pam_close_session(pam_handle_t *pamh, int flags);

Return: PAM_SUCCESS | PAM_ABORT | PAM_BUF_ERR | PAM_SESSION_ERR
```

The `pam_open_session` function sets up a user session for a previously authenticated user (create/mount the user home directory, sets the environment, etc..). The `flags` argument can have the value `PAM_SILENT` (do not emit any messages).

The `pam_close_session` ends an authenticated session (unmount the user home directory, etc..).

When we create a session we can also set the environment with the `pam_getenvlist` function¹⁰⁸, which is defined as follows.

```
#include <security/pam_appl.h>

char **pam_getenvlist(pam_handle_t *pamh);

Returns: the PAM environment, NULL on failure
```

The `pam_getenvlist` function returns the PAM environment associated with a given `pamh` PAM context. This environment is provided as an array of strings (`char **`), where each element represents an environment variable in the standard `NAME=value` format.

The structure of the returned array is compatible with the format required by functions such as `execle`, meaning it can be directly passed as the environment argument when executing a new program. This allows a process started under PAM control to inherit or propagate environment variables established during the authentication session.

Except the main function the rest of the program remains unchanged, so the code below contains the updated main of `loginpam.c`.

```
18 int main(void) {
19     char username[64];
20     struct passwd *pw;
21     pam_handle_t *pamh = NULL;
22     int pam_result;

23     printf("Login: ");
```

¹⁰⁷ See the `pam_open_session(3)` and `pam_close_session(3)` manual pages

¹⁰⁸ See the `pam_getenvlist(3)` manual page

```

24     fflush(stdout);
25     if (!fgets(username, sizeof(username), stdin)) return 1;
26     username[strcspn(username, "\n")] = 0;
27     pam_result = pam_start("loginpam", username, &conv, &pamh);
28     if (pam_result != PAM_SUCCESS) {
29         fprintf(stderr, "pam_start failed: %s\n", pam_strerror(pamh,
30             pam_result));
31     }
32     pam_result = pam_authenticate(pamh, 0);
33     if (pam_result != PAM_SUCCESS) {
34         fprintf(stderr, "Authentication failed: %s\n",
35             pam_strerror(pamh, pam_result));
36         pam_end(pamh, pam_result);
37         return 1;
38     }
39     pam_result = pam_acct_mgmt(pamh, 0);
40     if (pam_result != PAM_SUCCESS) {
41         fprintf(stderr, "Account check failed: %s\n",
42             pam_strerror(pamh, pam_result));
43         pam_end(pamh, pam_result);
44         return 1;
45     }
46     pam_result = pam_open_session(pamh, 0);
47     if (pam_result != PAM_SUCCESS) {
48         fprintf(stderr, "Failed to open PAM session: %s\n",
49             pam_strerror(pamh, pam_result));
50         pam_end(pamh, pam_result);
51         return 1;
52     }
53     pw = getpwnam(username);
54     if (!pw) {
55         fprintf(stderr, "getpwnam failed.\n");
56         pam_close_session(pamh, 0);
57         pam_end(pamh, PAM_SUCCESS);
58         return 1;
59     }
60     do_login_record(username);
61     setgid(pw->pw_gid);
62     if (initgroups(username, pw->pw_gid) < 0 || setgid(pw->pw_gid) < 0
63         || setuid(pw->pw_uid) < 0) {
64         perror("Failed to drop privileges");
65         pam_close_session(pamh, 0);
66         pam_end(pamh, PAM_SUCCESS);
67         return 1;
68     }

```

```

65     setuid(pw->pw_uid);
66     setenv("HOME", pw->pw_dir, 1);
67     setenv("USER", pw->pw_name, 1);
68     setenv("LOGNAME", pw->pw_name, 1);
69     setenv("SHELL", pw->pw_shell, 1);

70     char **envlist = pam_getenvlist(pamh);
71     if (envlist) {
72         for (char **env = envlist; *env; ++env) {
73             putenv(*env);
74         }
75         free(envlist);
76     }

77     if (chdir(pw->pw_dir) < 0) {
78         fprintf(stderr, "Warning: Failed to change to home directory
79             %s\n", pw->pw_dir);

80     printf("Welcome %s!\n", username);
81     char *shell = pw->pw_shell && pw->pw_shell[0] ? pw->pw_shell :
82         "/bin/sh";
83     execvp(shell, (char *[]){ shell, NULL });
84     perror("Failed to exec shell");
85     do_logout_record();
86     pam_close_session(pamh, 0);
87     pam_end(pamh, PAM_SUCCESS);
88     return 1;
89 }

```

loginpam.c

- Line 27. Initializes the PAM context for the "loginpam" service and associates it with the provided username*
- Line 32. Authenticates the user using PAM modules configured in the system*
- Line 38. Performs account management checks, such as verifying whether the account is allowed to log in*
- Line 44. Opens a PAM session, allowing session-related modules to execute (e.g., logging, environment setup)*
- Line 50. Retrieves the user's system account entry from the password database*
- Line 57. Records the login event in the system using utmp accounting functions*
- Line 58. Sets the primary group ID for the user*
- Line 59. Initializes supplementary group memberships and switches both group ID and user ID to those of the authenticated user*
- Line 65. Ensures the process runs under the correct user ID after privilege changes*
- Line 66–69. Manually sets key environment variables such as HOME, USER, LOGNAME, and SHELL for the new user session*
- Line 70. Retrieves the environment variables provided by PAM via pam_getenvlist*

Line 71–73. Iterates through the PAM environment list and imports each variable into the current process environment using `putenv`

Line 75. Frees the memory allocated for the PAM environment list

Line 77–79. Changes the current working directory to the user’s home directory, if possible

Line 80. Prints a welcome message after successful authentication and setup

Line 81–82. Determines the user's shell and replaces the current process with it using `execvp`

Line 83–87. These lines are executed only if `execvp` fails; they handle error reporting, close the PAM session, and clean up before exiting

First, we update the `/etc/pam.d/loginpam` file to add the session module.

```
[linux@users] sudo vi /etc/pam.d/loginpam
# /etc/pam.d/loginpam
auth      required  pam_unix.so
account   required  pam_unix.so
session   required  pam_unix.so
```

We compile and execute the program.

```
[linux@users] gcc -o loginpam loginpam.c -lpam -lpam_misc -lutil

[linux@users] sudo ./loginpam
Login: newuser
Password:
Welcome newuser!

[linux@newuser] pwd
/home/newuser

[linux@newuser] whoami
newuser
```

We use the lines

```
66     setenv("HOME", pw->pw_dir, 1);
67     setenv("USER", pw->pw_name, 1);
68     setenv("LOGNAME", pw->pw_name, 1);
69     setenv("SHELL", pw->pw_shell, 1);
```

to set the environment variables in a order to have the correct environment of the user `newuser`. This because the environment is inherited from `root`, therefore we must override these values to get the environment of `newuser`.

The lines

```
77     if (chdir(pw->pw_dir) < 0) {
78         fprintf(stderr, "Warning: Failed to change to home directory
79             %s\n", pw->pw_dir);
80     }
```

changes the CWD (current working directory) to that one defined in the `/etc/passwd` for the user `newuser`. In this way we will be in the `/home/newuser` folder after the login.

Since the program is now integrated with PAM, login sessions are properly recorded by the system's authentication and session management infrastructure. As a result, these login events can be observed in system accounting tools such as `last`¹⁰⁹.

By running the `last` command, we can see the user sessions created through the PAM-based login program, along with their login time, logout time, and originating terminal. This confirms that the session is being correctly registered in the system logs (typically stored in `/var/log/wtmp`).

Shell 1

```
[linux@users] sudo ./loginpam
Login: newuser
Password:
Welcome newuser!

[linux@newuser]
```

Shell 2

```
[linux@users] last
newuser          Wed Jul 9 03:29 - still logged in
users tty2       Wed Jul 9 03:23 - still logged in
Debian-g tty1    Wed Jul 9 03:22 - 03:23 (00:00)
```

We can also observe active and historical login sessions using the `loginctl`¹¹⁰ command, which is part of `systemd`'s session management system.

Shell 2

```
[linux@users] loginctl
SESSION UID USER  SEAT LEADER CLASS  TTY IDLE SINCE
  2 1000 users  seat0 1951 user   tty2 no  -
  3 1000 users  -    1961 manager - no  -
  5 1005 newuser -   4820 manager - no  -
c2 1005 newuser -   4792 background - no  -

4 sessions listed.
```

When executed, `loginctl` displays a list of current sessions along with details such as the user ID, username, associated seat, session leader process, session class, and terminal information. This provides a more structured and `systemd`-integrated view of user logins compared to traditional tools like `who` or `last`.

¹⁰⁹ See the `wtmpdb(8)` manual page

¹¹⁰ See the `loginctl(1)` manual page

III.11.3 Process environment and PAM environment: `pam_getenv`, `pam_putenv`

The PAM library provides additional functions¹¹¹ for working with the PAM environment beyond `pam_getenvlist`. These functions allow programs to directly query and modify environment variables associated with a PAM session.

```
#include <security/pam_appl.h>

const char *pam_getenv(pam_handle_t *pamh, const char *name);

int pam_putenv(pam_handle_t *pamh, const char *name_value);

Return: a pointer to the value of the variable,
        PAM_SUCCESS | PAM_PERM_DENIED | PAM_BAD_ITEM | PAM_ABORT | PAM_BUF_ERR
```

The `pam_getenv` function retrieves the value of the variable `name` from the PAM context `pamh`. The function `pam_putenv` sets the variable `name_value` in the PAM context `pamh`. The `name_value` argument can be passed in one of the following forms:

- `NAME=value`
The environment variable `NAME` is set to the indicated value. If the variable already exists and has a value, it is overwritten, otherwise it is added to the PAM environment
- `NAME=`
Sets the variable to an empty value
- `NAME`
Without the '=', `pam_putenv` will delete the corresponding variable from the PAM environment

The `pam_putenv` function operates on a copy of `name_value`, which means in contrast to `putenv`, the application is responsible for freeing the data.

The PAM environment is not the process environment. In the process environment we get/set the variables with `getenv` and `setenv/putenv`. The following table shows a summary of the functions used to set the environment variables.

Functions	Purpose	Affects
<code>getenv</code> , <code>setenv</code> , <code>putenv</code>	Gets or sets/changes an environment variable in current process	Real process environment (<code>environ</code>)
<code>pam_getenvlist</code> , <code>pam_getenv</code> , <code>pam_putenv</code>	Gets/sets a PAM-managed environment variable	PAM environment (not <code>environ</code>)

Tab 26. Functions of the process environment and PAM environment

PAM maintains an *internal environment*, separate from the process real `environ`, to store variables that PAM modules set during authentication and session setup. The PAM environment is private to the PAM handle/context and exists only during the lifetime of the PAM session.

¹¹¹ See the `pam_getenv(3)` and `pam_putenv(3)` manual pages

The following table lists some PAM modules that can populate environment variables.

Module	Environment variables it may set
pam_env.so	From /etc/environment, /etc/security/pam_env.conf
pam_krb5.so	Kerberos ticket cache variables
pam_systemd.so	XDG_SESSION_ID, XDG_RUNTIME_DIR, etc.
pam_mail.so	MAIL=/var/mail/username
Custom modules	Any app-defined variables

Tab 27. PAM modules and variables

Without the lines (loginpam.c)

```
70     char **envlist = pam_getenvlist(pamh);
71     if (envlist) {
72         for (char **env = envlist; *env; ++env) {
73             putenv(*env);
74         }
75         free(envlist);
76     }
```

the child shell could not have the XDG_ variables (used by desktop services).

If we add the line

```
printf(" %s\n", *env);
```

before the line 73 we can print all the PAM environment variables.

```
[linux@users] sudo ./loginpam
Login: newuser
Password:
XDG_SESSION_ID=c5
XDG_RUNTIME_DIR=/run/user/1005
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1005/bus
XDG_SESSION_TYPE=unspecified
XDG_SESSION_CLASS=background
Welcome newuser!
```

These variables are ultimately sourced from /etc/environment, which is one of the system-wide configuration files used to define global environment variables on Linux systems.

During session initialization, PAM (typically through modules like pam_env) reads /etc/environment and other related configuration files, then loads the defined variables into the PAM environment. These variables can later be accessed through functions such as pam_getenv or pam_getenvlist, or viewed via tools that inspect the PAM session environment.

In this way, /etc/environment serves as a central place for defining system-wide environment variables that are propagated into user sessions at login.

The `pam_getenv` command-line tool provides a shell-level equivalent to the `pam_getenv` function in the PAM library.

It allows users or scripts to query a specific environment variable from the current PAM session, instead of accessing the full environment list returned by `pam_getenvlist`. In this way, it behaves similarly to the function version, but is usable directly from the command line for inspection and debugging of PAM-managed session variables.

```
[linux@users] pam_getenv PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
```

III.11.4 PAM items: `pam_get_item` and `pam_set_item`

PAM items are the named data elements inside the PAM context that we can read and write. They are represented by the following constants defined in the `security/_pam_types.h` header file.

Value	Constant	Purpose
1	PAM_SERVICE	PAM service name
2	PAM_USER	The username
3	PAM_TTY	The terminal
4	PAM_RHOST	Remote hostname
5	PAM_CONV	Conversation structure
6	PAM_AUTHTOK	Authentication token (password)
7	PAM_OLDAUTHTOK	Old authentication token (password)
8	PAM_RUSER	Remote user (e.g. SSH)
9	PAM_USER_PROMPT	Prompt for getting a username
10	PAM_FAIL_DELAY	App supplied function to override failure delays
11	PAM_XDISPLAY	X display name
12	PAM_XAUTHDATA	X server authentication data
13	PAM_AUTHOK_TYPE	Type for <code>pam_get_authtok</code> ¹¹²

Tab 28. PAM items

¹¹² See the `pam_get_authtok(3)` manual page

We can read and write these items with the `pam_get_item`¹¹³ and `pam_set_item`¹¹⁴ functions, which are defined as follows.

```
#include <security/pam_modules.h>

int pam_get_item(const pam_handle_t *pamh, int item_type, const void
                **item);

int pam_set_item(pam_handle_t *pamh, int item_type, const void *item);

Return: PAM_SUCCESS|PAM_BAD_ITEM|PAM_BUF_ERR|PAM_PERM_DENIED|PAM_SYSTEM_ERR
        PAM_SUCCESS | PAM_BAD_ITEM | PAM_BUF_ERR | PAM_SYSTEM_ERR
```

A *PAM service* is a file that defines how authentication, authorization, session setup, and password management should work for a specific application or system component.

The following table lists some of the PAM services we can find on a Linux system.

File	PAM service name	Used by
/etc/pam.d/login	login	Console login (/bin/login)
/etc/pam.d/sshd	sshd	OpenSSH
/etc/pam.d/sudo	sudo	Privilege escalation
/etc/pam.d/gdm-password	gdm-password	Gnome Display Manager
/etc/pam.d/myservice	myservice	Custom service

Tab 29. PAM services example

We can get the values of all the PAM items with the two functions mentioned above except `PAM_AUTHTOK` because this is a sensitive data.

We modify our `loginpam.c` program to retrieve the user and the service item from the context. We add the following lines after line 76

```
const void *user, *service;

ret = pam_get_item(pamh, PAM_USER, &user);
if (ret != PAM_SUCCESS || !user) {
    fprintf(stderr, "pam_get_item failed: %s\n", pam_strerror(pamh, ret));
    pam_end(pamh, ret);
} else {
    printf("\nPAM USER: %s", user);
}

ret = pam_get_item(pamh, PAM_SERVICE, &service);
if (ret != PAM_SUCCESS || !service) {
    fprintf(stderr, "pam_get_item failed: %s\n", pam_strerror(pamh, ret));
```

113 See the `pam_get_item(3)` manual page

114 See the `pam_set_item(3)` manual page

```

    pam_end(pamh, ret);
} else {
    printf("\nPAM SERVICE: %s", service);
}

```

By doing this, the program becomes more tightly integrated with PAM, since it no longer depends solely on local variables, but instead extracts authentication-related information directly from the PAM transaction context.

```

[linux@users] sudo ./loginpam
Login: newuser
Password:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
COMMAND_NOT_FOUND_INSTALL_PROMPT=1
POWERSHELL_UPDATECHECK=Off
POWERSHELL_TELEMETRY_OPTOUT=1
DOTNET_CLI_TELEMETRY_OPTOUT=1
XDG_SESSION_ID=c39
XDG_RUNTIME_DIR=/run/user/1005
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1005/bus
XDG_SESSION_TYPE=unspecified
XDG_SESSION_CLASS=background
PAM USER: newuser
PAM SERVICE: loginpam
Welcome newuser!

```

As we have said before, we cannot retrieve the value of `PAM_AUTHTOK` directly because it is a sensitive data. This means PAM does not save it after the call to `pam_authenticate` because it is intended for internal use. Therefore if we want, for example, to store the password typed by the user, we need to use a custom conversation function.

The following `pam_conversation` function is an example we can use with the `loginpam.c` program.

```

1 char *captured_password = NULL;
2 static int pam_conversation(int num_msg, const struct pam_message
                             **msg, struct pam_response **resp, void
                             *appdata_ptr)
3 {
4     struct pam_response *reply = calloc(num_msg, sizeof(struct
                                         pam_response));
5     if (!reply)
6         return PAM_CONV_ERR;
7     for (int i = 0; i < num_msg; ++i) {
8         switch (msg[i]->msg_style) {
9             case PAM_PROMPT_ECHO_OFF: {
10                 char *input = getpass(msg[i]->msg);
11                 reply[i].resp = input ? strdup(input) : NULL;
12                 reply[i].resp_retcode = 0;
13                 if (captured_password) {
14                     free(captured_password);

```

```

15         }
16         captured_password = strdup(input);
17         break;
18     }
19     case PAM_PROMPT_ECHO_ON: {
20         char buffer[512];
21         printf("%s", msg[i]->msg);
22         if (!fgets(buffer, sizeof(buffer), stdin))
23             return PAM_CONV_ERR;
24         buffer[strcspn(buffer, "\n")] = 0;
25         reply[i].resp = strdup(buffer);
26         reply[i].resp_retcode = 0;
27         break;
28     }
29     case PAM_TEXT_INFO:
30         printf("%s\n", msg[i]->msg);
31         break;
32     case PAM_ERROR_MSG:
33         fprintf(stderr, "%s\n", msg[i]->msg);
34         break;
35     default:
36         free(reply);
37         return PAM_CONV_ERR;
38 }
39 }
40 *resp = reply;
41 return PAM_SUCCESS;
42 }

```

pam_conversation()

Line 4. Allocates memory for an array of `pam_response` structures, one for each message received from PAM

Line 7. Iterates through all messages sent by the PAM module

Line 8. Checks the type of each message to determine how it should be handled

Line 9–18 (PAM_PROMPT_ECHO_OFF). Handles password prompts where input must not be displayed. The function uses `getpass` to securely read the password. The response is duplicated and stored in the `reply` structure. Additionally, the password is saved in the global variable `captured_password`, replacing any previous value

Line 19–28 (PAM_PROMPT_ECHO_ON). Handles prompts where user input should be visible. The program prints the prompt message and reads input using `fgets`. The response is then stored in the `pam_response` structure

Line 29–31 (PAM_TEXT_INFO). Displays informational messages provided by PAM

Line 32–34 (PAM_ERROR_MSG). Displays error messages sent by PAM to standard error

Line 35–38 (default case). Handles unexpected message types by freeing allocated memory and returning an error

Line 40–41. Returns the array of responses back to PAM and signals successful completion of the conversation

We can add this code before the `main` and the following lines before the `chdir` call.

```

if (captured_password) {
    printf("\nPassword captured: %s\n", captured_password);
    free(captured_password);
} else {
    printf("\nPassword not captured.\n");
}

```

The full code is available in the *Theory and Insights* section. We execute the program.

```

[linux@users] sudo ./loginpam
Login: newuser
Password:
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/local/games:/usr/games
COMMAND_NOT_FOUND_INSTALL_PROMPT=1
POWERSHELL_UPDATECHECK=Off
POWERSHELL_TELEMETRY_OPTOUT=1
DOTNET_CLI_TELEMETRY_OPTOUT=1
XDG_SESSION_ID=c2
XDG_RUNTIME_DIR=/run/user/1005
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1005/bus
XDG_SESSION_TYPE=unspecified
XDG_SESSION_CLASS=background

PAM USER: newuser
PAM SERVICE: loginpam
Password captured: newuser

Welcome newuser!

```

With this modification, the program is extended to make use of information that is already available inside the PAM conversation context. Instead of only using PAM for authentication and then discarding the interaction details, the updated version now explicitly retrieves data that has been captured during the PAM exchange.

In particular, the `pam_conversation` function stores the password entered by the user in a global variable (`captured_password`) when handling a `PAM_PROMPT_ECHO_OFF` request. This allows the program to retain a copy of the password that was provided during authentication. Before changing the working directory with `chdir`, the program checks whether this value exists. If the password has been successfully captured, it is printed and then freed from memory; otherwise, a message indicates that no password was stored.

This change highlights how PAM's conversation mechanism can be used not only to interact with the user during authentication, but also to extract and reuse contextual information exchanged during the login process, such as the password entered by the user.

III.11.5 Changing password: `pam_chauthtok`

The `pam_chauthtok`¹¹⁵ function can be used to change the password of the user, it is defined as follows.

```
#include <security/pam_appl.h>

int pam_chauthtok(pam_handle_t *pamh, int flags);

Returns: PAM_SUCCESS|PAM_AUTHOK_ERR|PAM_AUTHOK_RECOVERY_ERR|
PAM_AUTHOK_LOCK_BUSY|PAM_AUTHOK_DISABLE_AGING|PAM_PERM_DENIED|PAM_TRY_AGAIN|
PAM_USER_UNKNOWN
```

The function changes the authentication token for a given user. The argument `flags` can be `PAM_SILENT` or `PAM_CHANGE_EXPIRED_AUTHOK` (the user's authentication token should only be changed if it has expired).

The `pamchangepwd.c` program allows a user to change his password. The function `my_conv` is the same as the function `pam_conversation` of the `loginpam.c` program (except for the lines related to capture the password that have been removed), so it will not listed here.

```
1 int main() {
2     const char *service = "passwd";
```

¹¹⁵ See the `pam_chauthtok(3)` manual page

```

3     char username[64];
4     pam_handle_t *pamh = NULL;
5     struct pam_conv conv = { my_conv, NULL };
6     int ret;

7     printf("Username: ");
8     fflush(stdout);
9     if (!fgets(username, sizeof(username), stdin)) return 1;
10    username[strcspn(username, "\n")] = 0;

11    ret = pam_start(service, username, &conv, &pamh);
12    if (ret != PAM_SUCCESS) {
13        fprintf(stderr, "pam_start failed: %s\n", pam_strerror(pamh,
14            ret));
15        return 1;
16    }

17    ret = pam_authenticate(pamh, 0);
18    if (ret != PAM_SUCCESS) {
19        fprintf(stderr, "Authentication failed: %s\n",
20            pam_strerror(pamh, ret));
21        pam_end(pamh, ret);
22        return 1;
23    }

24    ret = pam_acct_mgmt(pamh, 0);
25    if (ret != PAM_SUCCESS) {
26        fprintf(stderr, "Account check failed: %s\n",
27            pam_strerror(pamh, ret));
28        pam_end(pamh, ret);
29        return 1;
30    }

31    ret = pam_chauthtok(pamh, PAM_SILENT);
32    if (ret != PAM_SUCCESS) {
33        fprintf(stderr, "Password change failed: %s\n",
34            pam_strerror(pamh, ret));
35    } else {
36        printf("Password changed successfully.\n");
37    }
38    pam_end(pamh, PAM_SUCCESS);
39    return 0;
40 }

```

pamchangepwd.c

At the beginning, the program defines the PAM service name (`passwd`), which tells PAM which configuration rules to apply. It then reads the username from standard input and removes the trailing newline so it can be safely used in subsequent PAM calls.

Next, the PAM session is initialized using `pam_start`, which creates a PAM transaction associated with the given user and service. If this step fails, the program terminates immediately. Authentication is then performed using `pam_authenticate`, ensuring that the user is correctly verified before any sensitive operation is allowed. This is followed by `pam_acct_mgmt`, which checks whether the account is valid and permitted to perform the requested action (for example, not locked or expired).

Once authentication and account validation succeed, the program calls `pam_chauthtok`, which is the PAM function responsible for changing authentication tokens, in this case the user's password. The `PAM_SILENT` flag is used to reduce unnecessary output during the process, since PAM modules typically handle their own prompts.

If the password change is successful, a confirmation message is printed; otherwise, an error message is displayed describing the failure.

Finally, `pam_end` is called to close the PAM session and release all associated resources.

We execute the program

```
[linux@users] sudo ./pamchangepwd
Username: newuser
Password:
New password:
Retype new password:
Password changed successfully.
```

This output shows the typical interaction flow of `pam_chauthtok`, which delegates password management to PAM modules. After the user is authenticated, PAM enforces the system's password policy by prompting for the current password and then requesting the new password twice for confirmation.

III.12 Theory and insights

III.12.1 The Process Execution Environment

The *Process Execution Environment* is the context in which a program runs. When an ELF binary is executed via `execve`, the kernel prepares the following context in the memory.

<code>argc</code>	→ command-line arguments count
<code>argv[0] ... argv[n]</code>	→ command-line arguments vector
<code>envp[0] ... envp[n]</code>	→ environment variables vector
<code>auxv[]</code>	→ auxiliary vector (<code>AT_*</code> entries)

Fig. 1 Process Execution Environment

The *auxiliary vector* is a part of the PXE passed by the kernel when the program is executed. It is an array of key-value pairs (type, value) describing low-level information about the process environment. These information are related to how and where the program is run and are useful for system-level or secure programming. Each entry in the *auxiliary vector* has the following form as defined in the `elf.h` file header

```
typedef struct
{
    uint64_t a_type;        /* Entry type */
    union
    {
        uint64_t a_val;    /* Integer value */
    } a_un;
} Elf64_auxv_t;
```

Where the `a_type` field is the entry type and `a_val` represents the value. The following table lists the types available.

AT_BASE	The base address of the program interpreter (usually, the dynamic linker)
AT_BASE_PLATFORM	A pointer to a string (PowerPC and MIPS only)
AT_CLKTCK	The frequency with which the <code>times</code> ¹¹⁶ function counts. This value can also be obtained via <code>sysconf(_SC_CLK_TCK)</code>
AT_DCACHEBSIZE	The data cache block size
AT_EGID	The effective group ID of the thread
AT_ENTRY	The entry address of the executable
AT_EUID	The effective user ID of the thread
AT_EXECFD	File descriptor of program
AT_EXECPN	A pointer to a string containing the pathname used to execute the program
AT_FLAGS	Flags (unused)
AT_FPUCW	Used FPU control word (SuperH architecture only)
AT_GID	The real group ID of the thread
AT_HWCAP	An architecture and ABI dependent bit-mask whose settings indicate detailed processor capabilities ¹¹⁷
AT_HWCAP2	Further machine-dependent hints about processor capabilities
AT_ICACHEBSIZE	The instruction cache block size
AT_L1D_CACHEGEOMETRY	Geometry of the L1 data cache
AT_L1D_CACHESIZE	The L1 data cache size
AT_L1I_CACHEGEOMETRY	Geometry of the L1 instruction cache, encoded as for AT_L1D_CACHEGEOMETRY
AT_L1I_CACHESIZE	The L1 instruction cache size
AT_L2_CACHEGEOMETRY	Geometry of the L2 cache, encoded as for AT_L1D_CACHEGEOMETRY
AT_L2_CACHESIZE	The L2 cache size

¹¹⁶ See the `times(2)` manual page

AT_L3_CACHEGEOMETRY	Geometry of the L3 cache, encoded as for AT_L1D_CACHEGEOMETRY
AT_L3_CACHESIZE	The L3 cache size
AT_PAGESZ	The system page size ¹¹⁸
AT_PHDR	The address of the program headers of the executable
AT_PHERENT	The size of program header entry
AT_PHNUM	The number of program headers
AT_PLATFORM	A pointer to a string that identifies the hardware platform that the program is running on
AT_RANDOM	The address of sixteen bytes containing a random value
AT_SECURE	Has a nonzero value if this executable should be treated securely ¹¹⁹
AT_SYSINFO	The entry point to the system call function in the vDSO ¹²⁰ . Not present on all architectures (e.g., absent on x86-64)
AT_SYSINFO_EHDR	The address of a page containing the vDSO
AT_UCACHEBSIZE	The unified cache block size
AT_UID	The real user ID of the thread

Tab 30. Auxiliary vector types

To retrieve these values we can use the `getauxval`¹²¹ system call, which is defined as follows.

```
#include <sys/auxv.h>

unsigned long getauxval(unsigned long type);

Returns: on success the value corresponding to type, 0 if type is not found
```

The program `auxvec.c` demonstrates how to retrieve information from the auxiliary vector.

```
1 #include <stdio.h>
2 #include <sys/auxv.h>

3 int main() {
4     unsigned long uid = getauxval(AT_UID);
5     unsigned long pagesz = getauxval(AT_PAGESZ);
6     unsigned long secure = getauxval(AT_SECURE);
7     unsigned long entry = getauxval(AT_ENTRY);
8     unsigned long base = getauxval(AT_BASE);
9     const char *execfn = (const char *) getauxval(AT_EXECFN);

10    printf("AT_BASE = 0x%lx\n", base);
11    printf("AT_EXECFN = %s\n", execfn);
12    printf("AT_UID = %lu\n", uid);
13    printf("AT_PAGESZ = %lu bytes\n", pagesz);
14    printf("AT_SECURE = %lu\n", secure);
15    printf("AT_ENTRY = 0x%lx\n", entry);
16    return 0;
17 }
```

auxvec.c Retrieves some values from the *auxiliary vector*

Line 4–8. The function `getauxval` is used to extract different entries from the auxiliary vector:

- `AT_UID` returns the real user ID of the process
- `AT_PAGESZ` returns the system page size
- `AT_SECURE` indicates whether the process is running in a secure-execution mode
- `AT_ENTRY` gives the program's entry point address
- `AT_BASE` provides the base address of the program interpreter or shared libraries

117 The contents of the bit mask are hardware dependent (for example, the source file `arch/x86/include/asm/cpufeature.h` in the kernel source contains details relating to the Intel x86 architecture; the value returned is the first 32-bit word of the array described there). A human-readable version of the same information is available via `/proc/cpuinfo`

118 The same value returned by `sysconf(_SC_PAGESIZE)`

119 Most commonly, a nonzero value indicates that the process is executing a set-user-ID or set-group-ID binary (so that its real and effective UIDs or GIDs differ from one another), or that it gained capabilities by executing a binary file that has capabilities. Alternatively, a nonzero value may be triggered by a Linux Security Module. When this value is nonzero, the dynamic linker disables the use of certain environment variables (see the manual page `ld-linux.so(8)`) and glibc changes other aspects of its behavior. See also the `secure_getenv(3)` manual page

120 The Virtual Dynamic Shared Object is a kernel feature

121 See the `getauxval(3)` manual page

Line 9. The value associated with `AT_EXECFN` is cast to a string (`const char *`) because it contains the pathname of the executable file that was used to launch the program.

Line 10–15. The retrieved values are then printed in a human-readable format, showing both memory-related information (such as addresses and page size) and execution context details (such as the executable path and security status).

We execute the program.

```
[linux@users] ./auxvec
AT_BASE   = 0x7f537a28a000
AT_EXECFN = ./auxvec
AT_UID    = 1000
AT_PAGESZ = 4096 bytes
AT_SECURE = 0
AT_ENTRY  = 0x5613f86e8060
```

The fields have been retrieved and printed on `stdout`.

The *auxiliary vector* is also available via the binary file `/proc/self/auxv`. To read the file we can use the `hexdump`¹²² program.

```
[linux@users] hexdump -C /proc/self/auxv
00000000 21 00 00 00 00 00 00 00 00 10 fd b8 c5 7f 00 00 |!.....|
00000010 33 00 00 00 00 00 00 00 f0 07 00 00 00 00 00 00 |3.....|
00000020 10 00 00 00 00 00 00 00 ff fb eb bf 00 00 00 00 |.....|
...output...
```

The program `auxvec2.c` demonstrates how to manually read the *auxiliary vector* of a running Linux process by accessing the special file `/proc/self/auxv`. Unlike `getauxval`, which provides a high-level API, this approach reads the raw kernel-provided data directly from the `proc` filesystem.

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <elf.h>
4 #include <stdio.h>

5 int main () {
6     int fd = open("/proc/self/auxv", O_RDONLY);
7     Elf64_auxv_t aux;
8     while (read(fd, &aux, sizeof(aux)) == sizeof(aux)) {
9         printf("Type: %lu, Value: %lx\n", aux.a_type, aux.a_un.a_val);
10    }

11    return 0;
12 }
```

auxvec2.c

¹²² Display file contents in hexadecimal, decimal, octal or ascii, see the `hexdump(1)` manual page

Line 6. The program opens `/proc/self/auxv`, which represents the auxiliary vector of the current process (`self`). This file contains a sequence of key–value pairs describing execution parameters passed by the kernel at program startup

Line 7. An `Elf64_auxv_t` structure is declared. This structure represents a single entry in the auxiliary vector, containing a type (`a_type`) and a corresponding value (`a_un.a_val`)

Line 8–10. The program repeatedly reads entries from the file, one structure at a time. For each successfully read entry, it prints:

- the type of the auxiliary vector entry (`a_type`)
- the value associated with that type (`a_un.a_val`)

This continues until all entries have been processed.

We execute the program.

```
[linux@users] ./auxvec2
Type: 33, Value: 7fb1234b5000
Type: 51, Value: 7f0
Type: 16, Value: bfebfbff
Type: 6, Value: 1000
Type: 17, Value: 64
Type: 3, Value: 555c1710d040
Type: 4, Value: 38
Type: 5, Value: e
Type: 7, Value: 7fb1234b7000
Type: 8, Value: 0
Type: 9, Value: 555c1710e070
Type: 11, Value: 3e8
Type: 12, Value: 3e8
Type: 13, Value: 3e8
Type: 14, Value: 3e8
Type: 23, Value: 0
Type: 25, Value: 7ffdd22d4fc9
Type: 26, Value: 2
Type: 31, Value: 7ffdd22d7fee
Type: 15, Value: 7ffdd22d4fd9
Type: 27, Value: 1c
Type: 28, Value: 20
Type: 0, Value: 0
```

The output of the program shows the raw contents of the auxiliary vector for the current process, as read from `/proc/self/auxv`. Each line corresponds to a kernel-provided entry composed of a numeric type identifier and its associated value.

These entries describe low-level execution details such as memory layout, system configuration, and process metadata. For example, some values represent memory addresses (such as program segments or the stack), while others encode system limits, hardware capabilities, or security-related flags.

The final entry:

```
Type: 0, Value: 0
```

marks the end of the auxiliary vector.

Because the output uses numeric identifiers (e.g., 33, 51, 16), it reflects the raw ELF-level representation rather than human-readable labels. These numbers correspond to constants defined in system headers such as `elf.h`, which map each type to a meaningful symbolic name (e.g., `AT_BASE`, `AT_ENTRY`, `AT_UID`, etc.).

Overall, this execution demonstrates how the Linux kernel passes essential runtime information to user-space programs at startup, and how this information can be inspected directly through the `/proc` filesystem.

The first program that uses the *auxiliary vector* is the dynamic linker `ld-linux.so`¹²³.

To see the *auxiliary vector* supplied to a program we can use the `LD_SHOW_AUXV` variable as follows.

```
[linux@users] LD_SHOW_AUXV=1 ls
AT_SYSINFO_EHDR:      0x7feb295eb000
AT_MINSIGSTKSZ:       2032
AT_HWCAP:              0xbfebfbbff
AT_PAGESZ:             4096
AT_CLKTCK:             100
AT_PHDR:               0x55eaf52e2040
AT_PHEENT:              56
AT_PHNUM:              14
AT_BASE:               0x7feb295ed000
AT_FLAGS:              0x0
AT_ENTRY:              0x55eaf52e8760
AT_UID:                1000
AT_EUID:               1000
AT_GID:                1000
AT_EGID:               1000
AT_SECURE:             0
AT_RANDOM:             0x7fffffaa8e199
AT_HWCAP2:             0x2
AT_EXEFCN:             /usr/bin/ls
AT_PLATFORM:          x86_64
AT_RSEQ_FEATURE_SIZE:  28
AT_RSEQ_ALIGN:         32
```

[Desktop](#) [Downloads](#) [Screenshots](#)

If we set the environment variable `LD_SHOW_AUXV` to any value, the *dynamic linker* will print the *auxiliary vector* before executing the target program. Before the normal output of `ls`, the system will display a list of *auxiliary vector entries*.

This happens because the dynamic loader (`ld.so`) checks for the presence of `LD_SHOW_AUXV` and, when it is set, prints the auxiliary vector during program startup for debugging purposes.

After this diagnostic output, the command proceeds normally and the standard `ls` output is displayed.

Each process in Linux has its own auxiliary vector, which is stored by the kernel and can be accessed through the `proc` filesystem at `/proc/<pid>/auxv`.

¹²³ See the `ld.so(8)` manual page

For example, we can inspect the auxiliary vector of a running shell process.

```
[linux@users] ps
  PID TTY          TIME CMD
 607608 pts/1    00:00:07 zsh
 632150 pts/1    00:00:00 ps

[linux@users] hexdump -C /proc/607608/auxv

00000000  21 00 00 00 00 00 00 00 00 70 36 9b da 7f 00 00 |!.....p6.....|
00000010  33 00 00 00 00 00 00 00 f0 07 00 00 00 00 00 00 |3.....|
00000020  10 00 00 00 00 00 00 00 ff fb eb bf 00 00 00 00 00 |.....|
00000030  06 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00 |.....|
00000040  11 00 00 00 00 00 00 00 64 00 00 00 00 00 00 00 |.....d.....|
00000050  03 00 00 00 00 00 00 00 40 30 ac dd d2 55 00 00 |.....@0...U..|
00000060  04 00 00 00 00 00 00 00 38 00 00 00 00 00 00 00 |.....8.....|
00000070  05 00 00 00 00 00 00 00 0e 00 00 00 00 00 00 00 |.....|
00000080  07 00 00 00 00 00 00 00 00 90 36 9b da 7f 00 00 |.....6.....|
00000090  08 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
000000a0  09 00 00 00 00 00 00 00 20 ad ad dd d2 55 00 00 |.....U..|
000000b0  0b 00 00 00 00 00 00 00 e8 03 00 00 00 00 00 00 |.....|
000000c0  0c 00 00 00 00 00 00 00 e8 03 00 00 00 00 00 00 |.....|
000000d0  0d 00 00 00 00 00 00 00 e8 03 00 00 00 00 00 00 |.....|
000000e0  0e 00 00 00 00 00 00 00 e8 03 00 00 00 00 00 00 |.....|
000000f0  17 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000100  19 00 00 00 00 00 00 00 49 39 d9 fb ff 7f 00 00 |.....I9.....|
00000110  1a 00 00 00 00 00 00 00 02 00 00 00 00 00 00 00 |.....|
00000120  1f 00 00 00 00 00 00 00 eb 3f d9 fb ff 7f 00 00 |.....?.....|
00000130  0f 00 00 00 00 00 00 00 59 39 d9 fb ff 7f 00 00 |.....Y9.....|
00000140  1b 00 00 00 00 00 00 00 1c 00 00 00 00 00 00 00 |.....|
00000150  1c 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00 |.....|
00000160  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
```

This output shows the raw auxiliary vector (`auxv`) of the process with PID 607608, read directly from `/proc/607608/auxv`.

The `auxv` file is not text: it is a binary sequence of key–value pairs, where each entry has:

- a type (first 8 bytes)
- a value (next 8 bytes)

On a 64-bit system, each entry is 16 bytes.

To retrieve a human-readable format, we can use `gdb` with the `-p` option to inspect the auxiliary vector of a running process.

```
[linux@users] gdb -q -p 607608
Attaching to process 607608
Reading symbols from /usr/bin/zsh...
(No debugging symbols found in /usr/bin/zsh)
...output...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007fda9b0a4e92 in ?? () from /usr/lib/x86_64-linux-gnu/libc.so.6
(gdb) info auxv
33  AT_SYSINFO_EHDR      System-supplied DSO's ELF header 0x7fda9b367000
51  AT_MINSIGSTKSZ        Minimum stack size for signal delivery 0x7f0
```

16	AT_HWCAP	Machine-dependent CPU capability hints	0xbfebfbbf
6	AT_PAGESZ	System page size	4096
17	AT_CLKTCK	Frequency of times()	100
3	AT_PHDR	Program headers for program	0x55d2ddac3040
4	AT_PHENT	Size of program header entry	56
5	AT_PHNUM	Number of program headers	14
7	AT_BASE	Base address of interpreter	0x7fda9b369000
8	AT_FLAGS	Flags	0x0
9	AT_ENTRY	Entry point of program	0x55d2ddadad20
11	AT_UID	Real user ID	1000
12	AT_EUID	Effective user ID	1000
13	AT_GID	Real group ID	1000
14	AT_EGID	Effective group ID	1000
23	AT_SECURE	Boolean, was exec setuid-like?	0
25	AT_RANDOM	Address of 16 random bytes	0x7ffffbd93949
26	AT_HWCAP2	Extension of AT_HWCAP	0x2
31	AT_EXECFN	File name of executable	0x7ffffbd93feb
"/usr/bin/zsh"			
15	AT_PLATFORM	String identifying platform	0x7ffffbd93959
"x86_64"			
27	AT_RSEQ_FEATURE_SIZE	rseq supported feature size	28
28	AT_RSEQ_ALIGN	rseq allocation alignment	32
0	AT_NULL	End of vector	0x0

(gdb)

The `gdb` command `info auxv` retrieves the entire auxiliary vector of the process currently being debugged and prints it in a readable format directly inside the GDB session.

III.12.2 The `intmax_t` and `uintmax_t` data types

The types `intmax_t` and `uintmax_t` are defined in the header file `stdint.h` and represent the largest standard signed and unsigned integer types available on the platform.

```
#include <stdint.h>

typedef intmax_t;
typedef uintmax_t;

#define INTMAX_WIDTH
#define UINTMAX_WIDTH INTMAX_WIDTH

#define INTMAX_MAX
#define INTMAX_MIN
#define UINTMAX_MAX

#define INTMAX_C(c) c ##
#define UINTMAX_C(c) c ##
```

The following table describe these types¹²⁴.

<code>intmax_t</code>	Signed integer type capable of representing any value of any basic signed integer type supported by the implementation. It is capable of storing values in the range <code>[INTMAX_MIN, INTMAX_MAX]</code>
<code>uintmax_t</code>	Unsigned integer type capable of representing any value of any basic unsigned integer type supported by the implementation. It is capable of storing values in the range <code>[0, UINTMAX_MAX]</code>
<code>[U] INTMAX_WIDTH</code>	Expands to the width in bits of these type
<code>[U] INTMAX_MAX</code>	Expands to the maximum value that these types can hold
<code>INTMAX_MIN</code>	Expands to the minimum value that <code>intmax_t</code> can hold
<code>[U] INTMAX_C()</code>	Expand their argument to an integer constant of type <code>[u] intmax_t</code>

Tab 31. `intmax_t` and `uintmax_t` types

124 From the `intmax_t(3type)` manual page

III.12.3 Using `initgroups` to set up the supplementary groups

The program `test_access.c` use `initgroups` to set the supplementary groups of the caller process with the goal to open the passed file.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pwd.h>
5 #include <grp.h>
6 #include <errno.h>
7 #include <string.h>
8 #include <sys/types.h>
9 #include <fcntl.h>

10 int main(int argc, char *argv[]) {
11     if (argc < 3 || argc > 4) {
12         fprintf(stderr, "Usage: %s <username> <filepath> [--no-initgroups]
13             \n", argv[0]);
14     }

15     const char *username = argv[1];
16     const char *filepath = argv[2];
17     int use_initgroups = 1;
18     if (argc == 4 && strcmp(argv[3], "--no-initgroups") == 0) {
19         use_initgroups = 0;
20     }

21     struct passwd *pw = getpwnam(username);
22     if (!pw) {
23         perror("getpwnam");
24         return EXIT_FAILURE;
25     }

26     printf("Switching to user: %s (UID: %d, GID: %d)\n",
27         username, pw->pw_uid, pw->pw_gid);
28     if (use_initgroups) {
29         printf("Calling initgroups()...\n");
30         if (initgroups(username, pw->pw_gid) != 0) {
31             perror("initgroups");
32             return EXIT_FAILURE;
33         }
34     } else {
35         printf("Skipping initgroups()...\n");
36     }

37     if (setgid(pw->pw_gid) != 0) {
38         perror("setgid");
39         return EXIT_FAILURE;
40     }
41 }
```

```

39     }

40     if (setuid(pw->pw_uid) != 0) {
41         perror("setuid");
42         return EXIT_FAILURE;
43     }

44     printf("Trying to open file: %s\n", filepath);
45     int fd = open(filepath, O_RDONLY);
46     if (fd == -1) {
47         perror("open");
48         printf("Access failed.\n");
49     } else {
50         printf("Access successful! File opened.\n");
51         close(fd);
52     }
53     return 0;
54 }

```

test_access.c

Line 21. The program retrieves the user's account information from the system database using `getpwnam`. This provides the password entry structure containing the user's UID, primary GID, and other related account details

Line 29. The function `initgroups` is used to set the supplementary group list of the process to match those of the specified user. This ensures that the process inherits all additional group memberships defined in `/etc/group`, which are important for file access control

Line 36. The program sets the process's primary group ID using `setgid`, aligning it with the user's primary group as defined in the system account database

Line 40. The process's user ID is changed using `setuid`, effectively dropping privileges and making the process run under the identity of the specified user

Line 44–52. Finally, the program attempts to open the specified file. The success or failure of this operation depends on the permissions associated with the file and the effective user and group identities of the process after the privilege change

We create a temporary file and set the permissions to `root` for the owner and `audio` for the group.

```

[linux@users] sudo touch /tmp/testfile

[linux@users] sudo chown root:audio /tmp/testfile

[linux@users] sudo chmod 640 /tmp/testfile

```

We check whether the user is in the `audio` group.

```

[linux@users] groups users
users : users adm dialout cdrom floppy sudo audio dip video plugdev ...

[linux@users] groups newuser
newuser : newuser

```

The user `users` belong to the group `audio` while the user `newuser` does not.

We execute the program.

```
[linux@users] sudo ./testaccess users /tmp/testfile
Switching to user: users (UID: 1000, GID: 1000)
Calling initgroups(...)
Trying to open file: /tmp/testfile
Access successful! File opened.

[linux@users] sudo ./testaccess newuser /tmp/testfile
Switching to user: newuser (UID: 1001, GID: 1002)
Calling initgroups(...)
Trying to open file: /tmp/testfile
open: Permission denied
Access failed.
```

As we can see, once the supplementary groups of the process are correctly set, the file can be opened only if the specified user belongs to the relevant group (in this case, `audio`). Otherwise, as in the case of `newuser`, access is denied. If `newuser` is added to the `audio` group, the file can also be accessed when running the program with that user.

It is also important to note that the program explicitly switches both the user and the primary group of the process using `setuid` and `setgid`. Without this step, the program would retain elevated privileges from `sudo`, and file access would always succeed regardless of the target user's permissions, defeating the purpose of the experiment.

III.12.4 Command line tools for the groups

To deal with the groups on a Linux system are available several commands. The following table lists some of the most common used.

Command	Description	Example
chgrp ¹²⁵	Change group ownership	chgrp newuser file.txt
groups ¹²⁶	Prints the groups a user belongs	groups
newgrp ¹²⁷	Log in to a new group	newgrp root
groupadd ¹²⁸	Creates a new group	groupadd newjobgroup
groupdel ¹²⁹	Delete a group	groupdel newjobgroup
groupmod ¹³⁰	Modify a group	groupmod -U newuser newjobgroup
addgroup ¹³¹	Add/change users and groups	addgroup newteam
id ¹³²	Print real and effective user and group IDs	id
getent ¹³³	Get entries from the NSS ¹³⁴ databases	getent group newuser

Tab 32. Groups commands

125 See the `chgrp(1)` manual page

126 See the `groups(1)` manual page

127 See the `newgrp(1)` manual page

128 See the `groupadd(8)` manual page

129 See the `groupdel(8)` manual page

130 See the `groupmod(8)` manual page

131 See the `adduser(8)` manual page

132 See the `id(1)` manual page

133 See the `getent(1)` manual page

134 See the `nss(5)` manual page

III.12.5 The Name Service Switch

The *Name Service Switch* (NSS) is an interface used by Linux systems to manage and access system-wide databases such as users, groups, passwords, networks, and related configuration data. Instead of relying on a single source, NSS allows this information to be obtained from multiple backends, including local files, system services, and network-based directory services.

In this model, each system database (for example, users, groups, or hosts) is associated with specific modules that implement the logic for retrieving the corresponding information. When a program calls a function such as `getpwnam` or `getgrnam`, the request is not handled directly by the application. Instead, it is processed by the NSS framework within the GNU C Library, which determines where and how to retrieve the required data.

The behavior of NSS is configured through two main configuration files, which define the lookup order and sources used for each database.

<code>/etc/default/nss</code>	General configuration
<code>/etc/nsswitch.conf</code>	Functionalities configuration

Tab 33. NSS files

The NSS is composed by modules, where each module is responsible of a database. The following table shows the system databases managed by NSS.

<i>passwd</i>	User database	<code>/etc/passwd</code>
<i>group</i>	Group database	<code>/etc/group</code>
<i>shadow</i>	Password database	<code>/etc/shadow</code>
<i>gshadow</i>	Group password database	<code>/etc/gshadow</code>
<i>hosts</i>	Hostname database	<code>/etc/hosts</code>
<i>networks</i>	Network database	<code>/etc/networks</code>
<i>protocols</i>	Protocol database	<code>/etc/protocols</code>
<i>services</i>	Service database	<code>/etc/services</code>
<i>ethers</i>	Ethernet addresses database	<code>/etc/ethers</code>
<i>rpc</i>	Remote procedure call database	<code>/etc/rpc</code>
<i>netgroup</i>	Network groups database	<code>/etc/netgroup</code>

Tab 34. NSS databases

Each of these databases has a dedicated manual page in the section 5: `passwd(5)`, `group(5)`, `host(5)`, `services(5)`, while the main manual page for NSS is `nss(5)`.

The `/etc/nsswitch.conf` file has the following format:

```
passwd:      files systemd winbind
group:       files systemd winbind
shadow:      files systemd
gshadow:     files systemd

hosts:       files myhostname mdns4_minimal [NOTFOUND=return] dns
networks:    files

protocols:   db files
services:    db files
ethers:      db files
rpc:         db files

netgroup:    nis
```

where the first column is the database and the rest of the line specifies how the lookup process works. E.g. for the `passwd` database will be used first the file, then the `systemd`¹³⁵ service and then the `winbind`¹³⁶ service.

¹³⁵ See the `systemd(1)` manual page

¹³⁶ See the `winbindd(8)` manual page

III.12.6 Command line tools for the users

To manage users on a Linux system, several standard commands are available. The table below summarizes some of the most commonly used ones.

<code>adduser</code> ¹³⁷	Add a user to the system	<code>adduser newuser</code>
<code>deluser</code> ¹³⁸	Delete a user	<code>deluser newuser</code>
<code>useradd</code> ¹³⁹	Create a new user	<code>useradd newuser</code>
<code>usermod</code> ¹⁴⁰	Modify a user account	<code>usermod -G newjobgroup newuser</code>
<code>userdel</code> ¹⁴¹	Delete a user	<code>userdel newuser</code>

Tab 35. Users commands

¹³⁷ See the `adduser(1)` manual page

¹³⁸ See the `deluser(1)` manual page

¹³⁹ See the `useradd(1)` manual page

¹⁴⁰ See the `usermod(1)` manual page

¹⁴¹ See the `userdel(1)` manual page

III.12.7 The login process

The following diagram illustrates the login process.

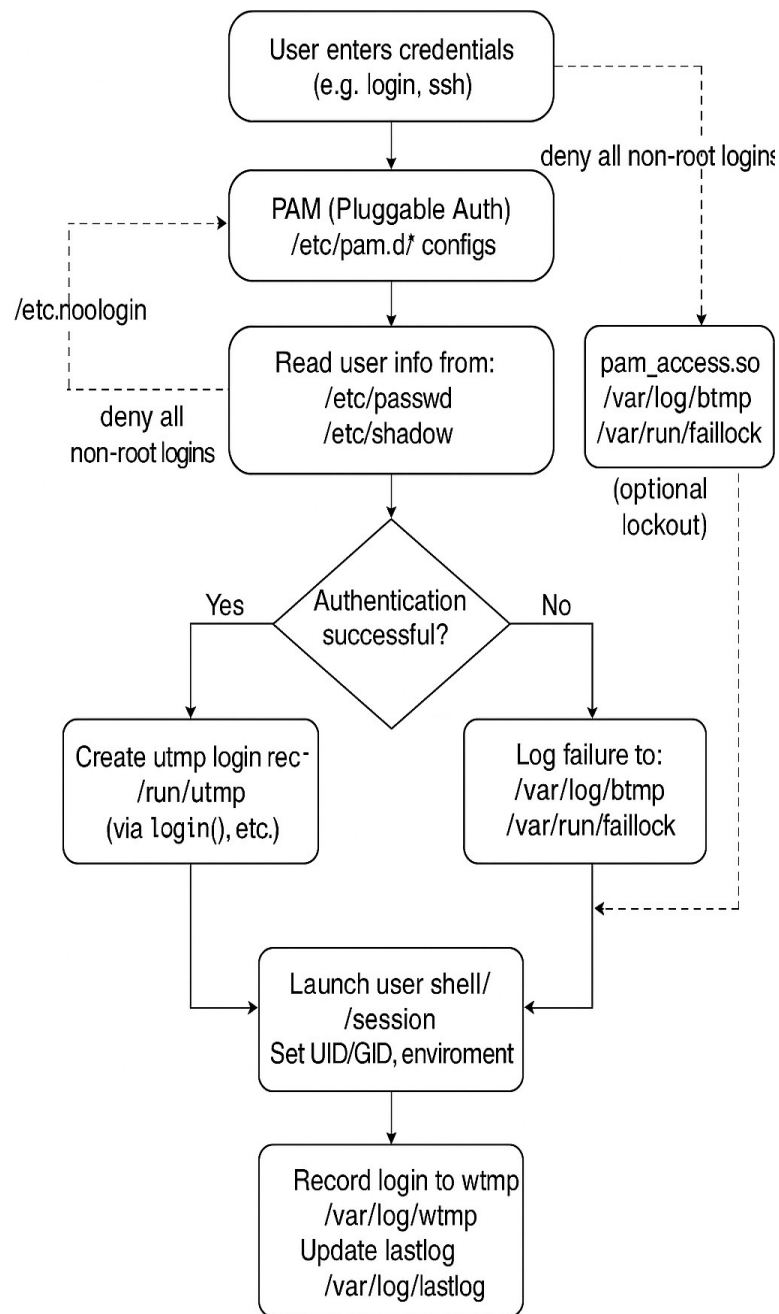


Fig 2. Login process

The following table lists the most commonly used program related to the login process.

Program	Category	Description
login	Login	Handles console login; prompts for user/password
passwd	Password Management	Changes user's password; updates /etc/shadow
su	Authentication	Switch user (usually to root); prompts for password
sudo	Privilege Escalation	Runs commands as another user (often root)
logout / exit	Logout	Ends the current shell session
who	User Accounting	Shows users currently logged in (utmp)
w	User Accounting	Shows logged-in users and what they are doing
last	Session History	Reads /var/log/wtmp for previous login sessions
lastlog	Login Info	Shows last login per user from /var/log/lastlog
utmpdump	Debug/Analysis	Dumps raw utmp, wtmp, or custom utmp files
whoami	Info Utility	Displays current effective username
id	Info Utility	Shows UID, GID, and groups for a user
getent	NSS Utility	Retrieves user, group, shadow, or host info from databases
pam-auth-update	PAM Config Tool	Enables/disables PAM modules on Debian-based systems
faillog	Security / Auditing	Shows login failures and controls fail counts
loginctl	Session Manager	Lists sessions via systemd-logind (modern alternative to who)

Tab 36. Login related commands

For further details about these commands, the user is encouraged to read the related manual pages.

The following table lists the files related to the login process.

File Path	Description	Used by
/etc/passwd	Basic user account information (username, UID, GID, home, shell)	login, su, id, getent
/etc/shadow	Secure password hashes, password aging and expiration settings	login, passwd, chage, crypt
/etc/group	Group definitions (GID, group members)	id, groups, getent
/etc/gshadow	Secure group passwords and admin info	groupadd, gpasswd
/run/utmp	Tracks current user sessions and logins	who, w, login, logout
/var/log/wtmp	Stores login/logout history across reboots	Last, login
/var/log/btmp	Records failed login attempts	lastb, faillog
/var/log/lastlog	Stores each user's most recent login time	lastlog, login
/var/log/auth.log	General authentication logs (PAM, sudo, ssh, etc.) (Debian-based OS)	syslog, rsyslog
/var/log/secure	Same auth.log (Red Hat-based OS)	syslog, rsyslog
/etc/login.defs	Configuration for login utilities (password aging, UID ranges)	login, passwd, useradd
/etc/nologin	If present, prevents non-root logins (used during shutdowns)	login
/etc/pam.d/	Directory with PAM (Pluggable Authentication Modules) configs for login, ssh, etc.	login, sudo, sshd
/var/run/faillock/	Tracks temporary lockouts after failed login attempts (used by PAM pam_faillock)	faillock, login
/etc/security/limits.conf	User resource limits (max logins, processes, etc.)	PAM modules
/etc/security/access.conf	Restrict login access by user or group	PAM pam_access

Tab 37. Files related to login

III.12.8 PAM functions summary

The table below provides an overview of the most commonly used PAM (Pluggable Authentication Modules) functions and their roles within the authentication framework.

Function	Description	Typical Use
<code>pam_start</code>	Initializes a PAM transaction with a specified service, user, and conversation function	Required at the start of PAM usage
<code>pam_end</code>	Terminates a PAM transaction and releases resources	Called at the end of PAM operations
<code>pam_authenticate</code>	Authenticates the user (e.g., checks password, biometrics, etc.)	login, sudo, etc.
<code>pam_acct_mgmt</code>	Performs account checks (e.g., expiration, access time restrictions)	Post-authentication account validation
<code>pam_setcred</code>	Establishes or deletes user credentials (e.g., Kerberos tickets, SELinux contexts)	After <code>pam_authenticate</code>
<code>pam_open_session</code>	Starts a session (e.g., mounts home dir, logs login).	When login is successful
<code>pam_close_session</code>	Ends a session (e.g., unmounts directories, removes session data)	On logout or exit
<code>pam_chauthtok</code>	Changes the user's authentication token (usually password)	<code>passwd</code> , password change flows
<code>pam_get_item</code>	Retrieves a PAM item (e.g., <code>PAM_USER</code> , <code>PAM_TTY</code>)	To read data from the PAM context
<code>pam_set_item</code>	Sets a PAM item (e.g., <code>PAM_RHOST</code> , <code>PAM_AUTHTOK</code>)	Before authentication or session
<code>pam_get_user</code>	Convenience wrapper for <code>pam_get_item(PAM_USER, ...)</code>	Retrieve username
<code>pam_get_authtok</code>	Retrieves the authentication token (e.g., password), possibly prompting the user	Used by modules, not applications
<code>pam_putenv</code>	Sets an environment variable in the PAM environment	During session setup
<code>pam_getenv</code>	Retrieves a specific environment variable from the PAM environment	During session
<code>pam_getenvlist</code>	Retrieves the entire list of environment variables set by PAM	After session init
<code>pam_strerror</code>	Returns a human-readable string for a PAM error code	For debugging and user messages
<code>pam_fail_delay</code>	Sets a delay time (in microseconds) before returning from failure	Login hardening

Tab 38. PAM common functions

List of tables

- Tab 1.** Clocks
- Tab 2.** Conversion specifier extensions of `strftime`
- Tab 3.** Error of `getdate_r`
- Tab 4.** Common environment variables
- Tab 5.** Locale settings
- Tab 6.** Locale categories
- Tab 7.** Locale functions
- Tab 8.** Values for the parameter `item` of `nl_langinfo`
- Tab 9.** Fields of the shadow file
- Tab 10.** Fields of the `gshadow` file
- Tab 11.** Files and commands for user and groups
- Tab 12.** Real, effective and saved GID
- Tab 13.** `setgid`, `setegid`, `setregid`, `setresgid`
- Tab 14.** Get and set user/group ID functions
- Tab 15.** The *hashed passphrase* fields
- Tab 16.** *Hashing methods* supported by `crypt`
- Tab 17.** Values for the `flags` argument
- Tab 18.** `utmp`, `wtmp` and `btmpt` files
- Tab 19.** `ut_type` values
- Tab 20.** Fields of the `utsname` structure
- Tab 21.** Files related to `uname`
- Tab 22.** File and settings related to the hostname
- Tab 23.** PAM authentication tasks
- Tab 24.** Values of the `flags` argument
- Tab 25.** `msg_style` values
- Tab 26.** Functions of the process environment and PAM environment
- Tab 27.** PAM modules and variables
- Tab 28.** PAM items
- Tab 29.** PAM services example
- Tab 30.** Auxiliary vector types
- Tab 31.** `intmax_t` and `uintmax_t` types
- Tab 32.** Groups commands
- Tab 33.** NSS files
- Tab 34.** NSS databases
- Tab 35.** Users commands
- Tab 36.** Login related commands
- Tab 37.** Files related to login
- Tab 38.** PAM common functions

List of figures

- Fig 1.** Process Execution Environment
- Fig 2.** Login process